

Lesson 4 of 5

Visualizing and Writing

Intro to R workshop, LU Skill School

Instructor:

Christopher Swader

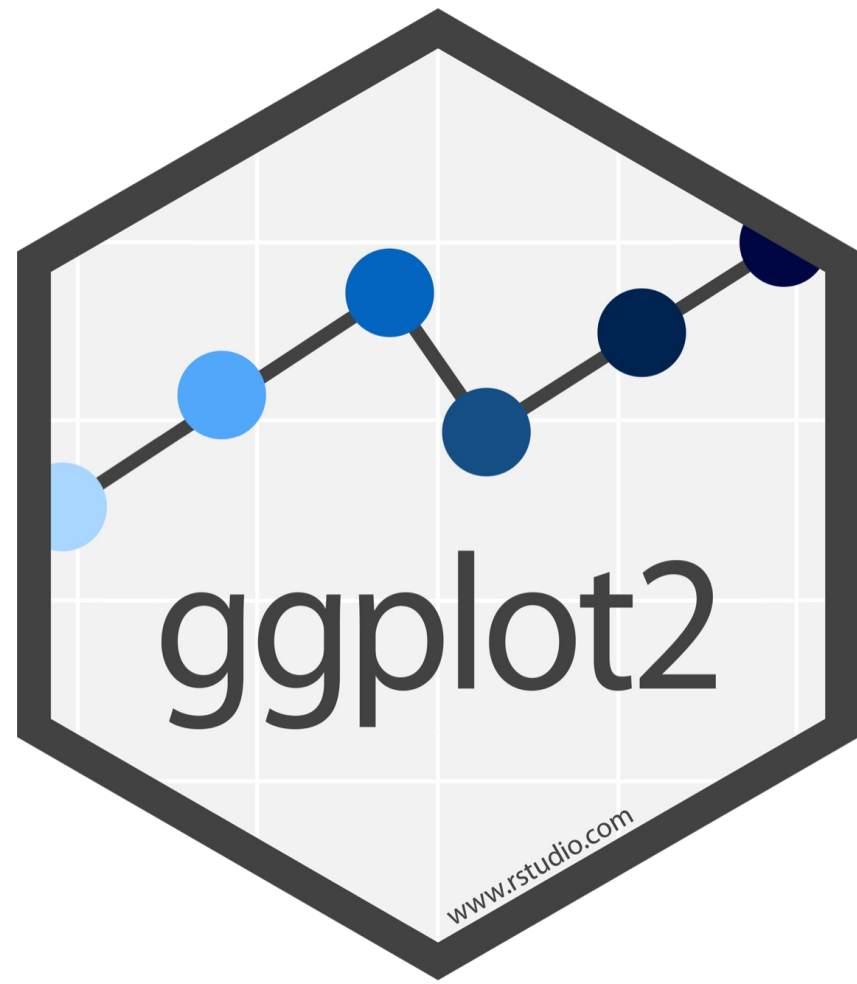
LU Sociology (Assoc. Prof) and LMU (Munich, Researcher)

Slides adapted from and inspired by:
Irina Vartanova, Institute for Future Studies, Stockholm

Today's agenda

- Visualizing with ggplot2, plotly, gganimate
- Rmarkdown

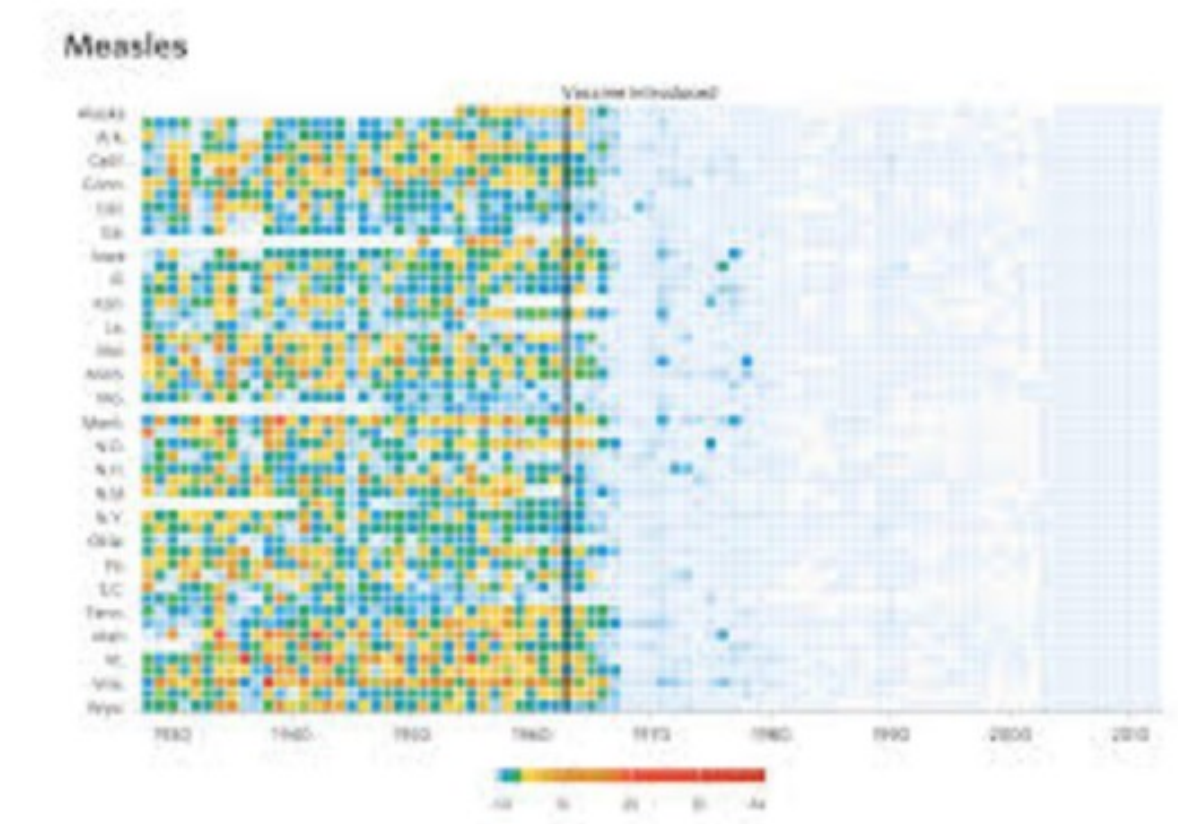
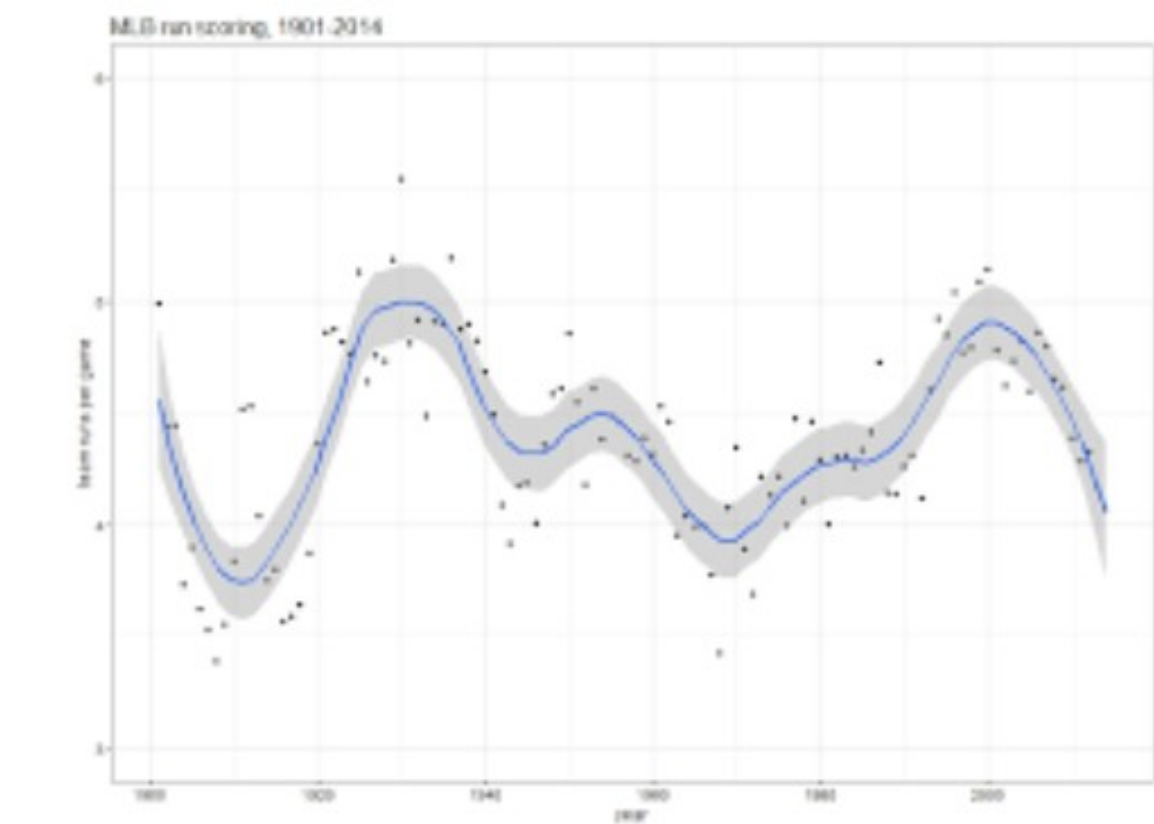
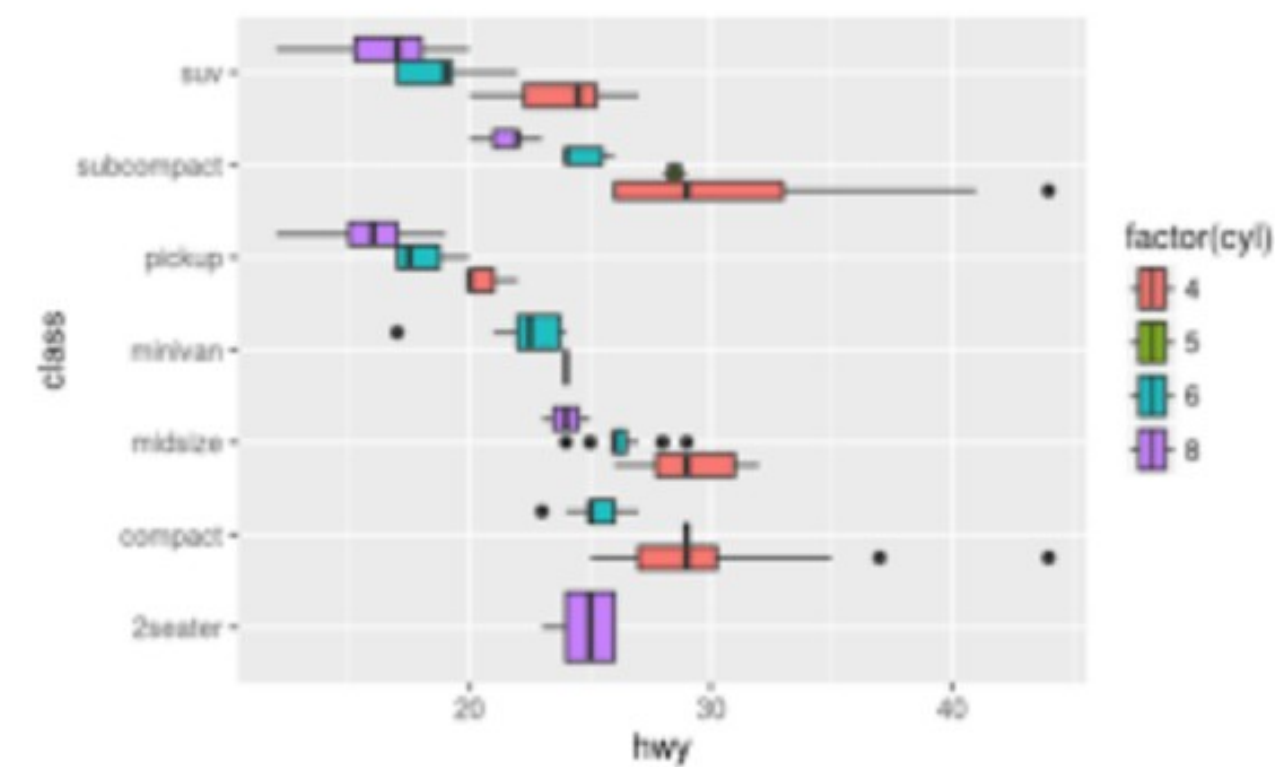
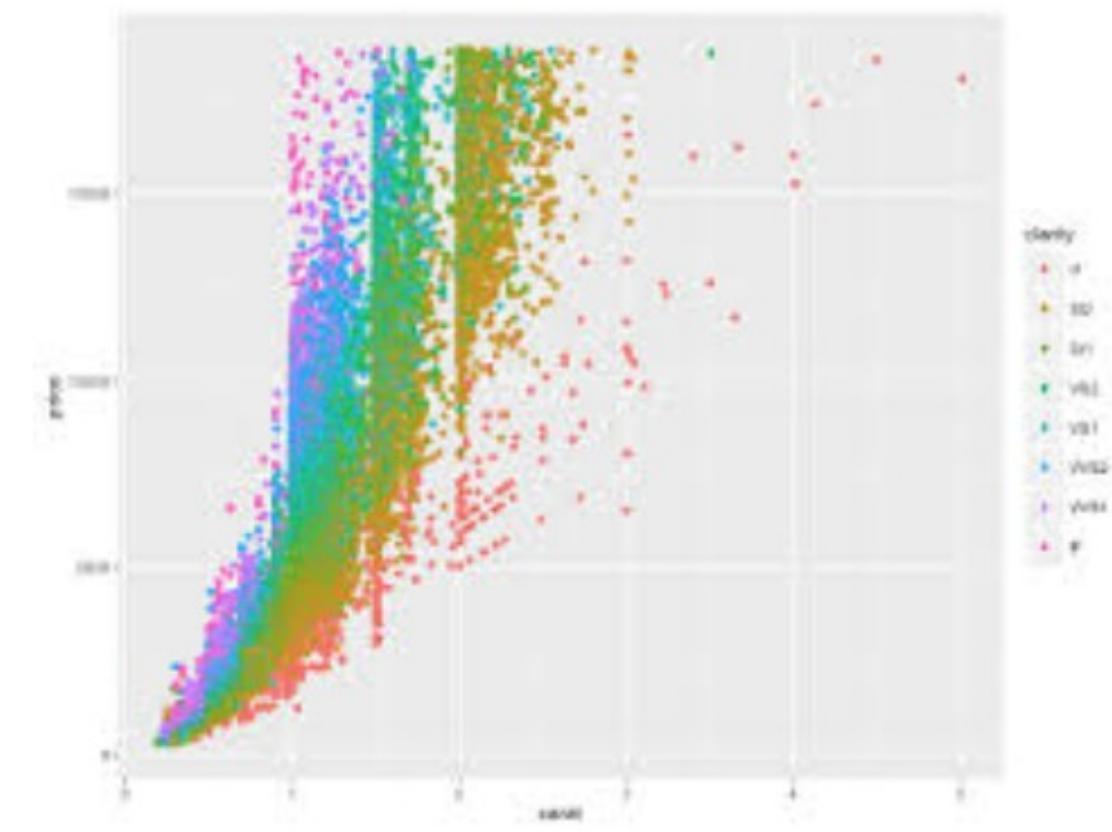
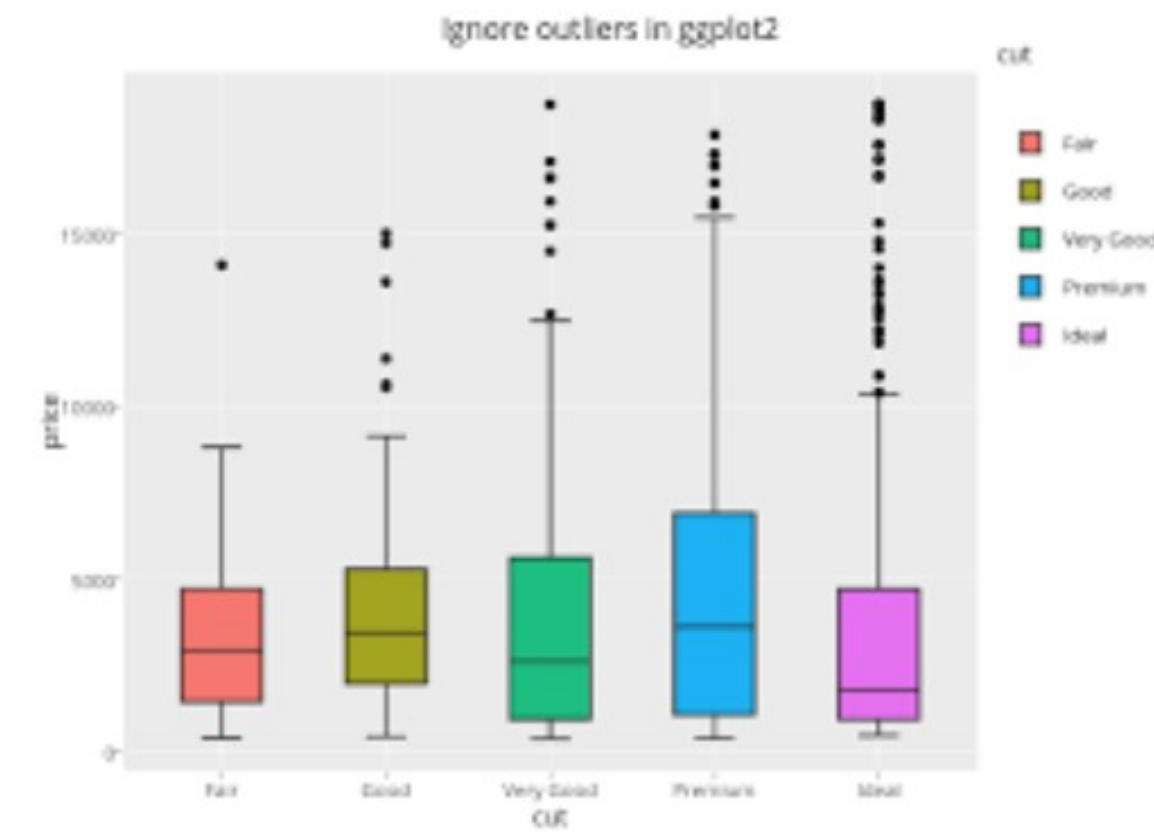
- Use the link to download the files we will be using today:
https://github.com/ChristopherSwader/R_introduction
- Download the Day 4 folder.

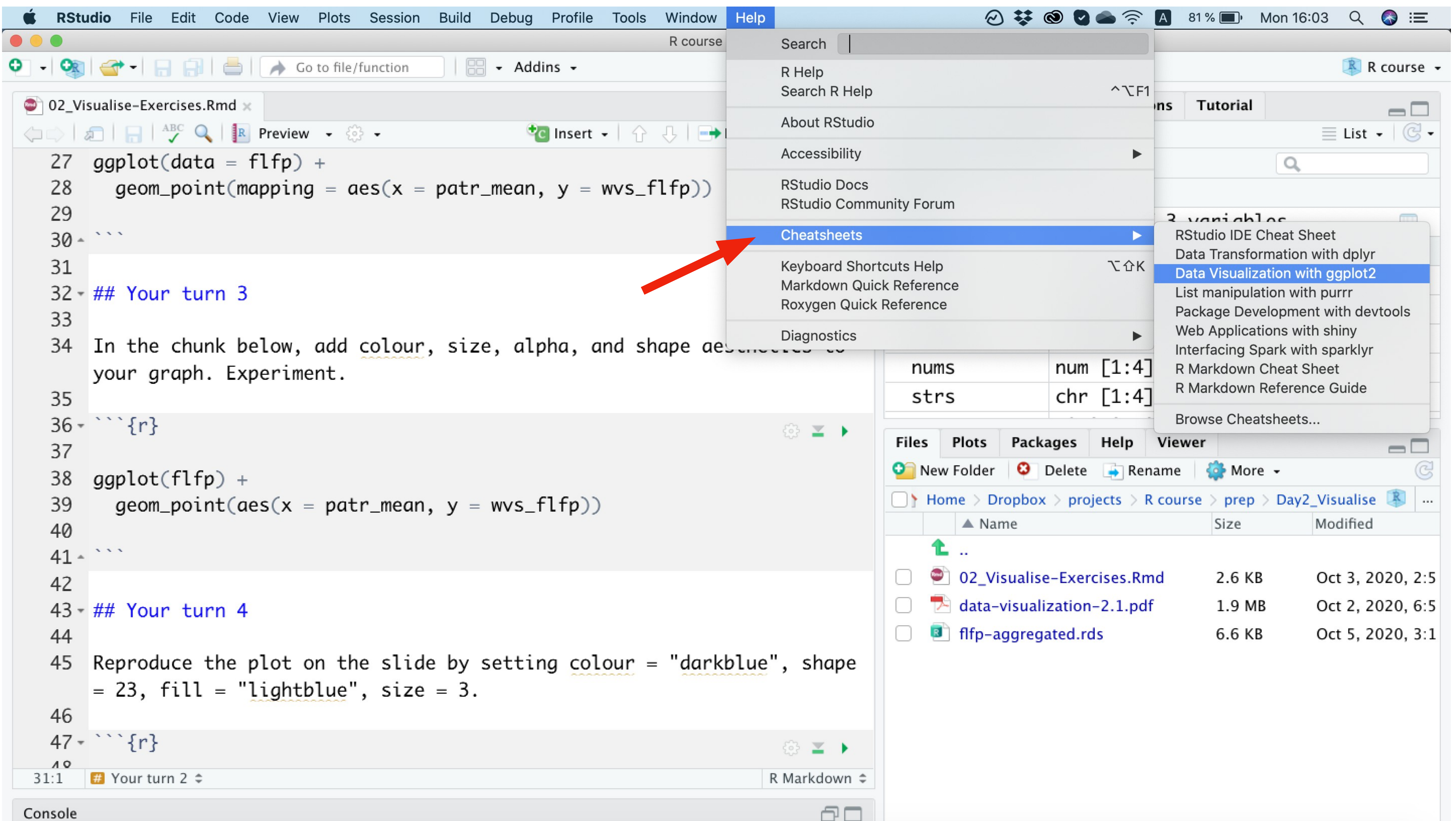


Basics of Visualisation in R's ggplot2 package

ggplot2: the grammar of graphics

A unified system for variety of graphs

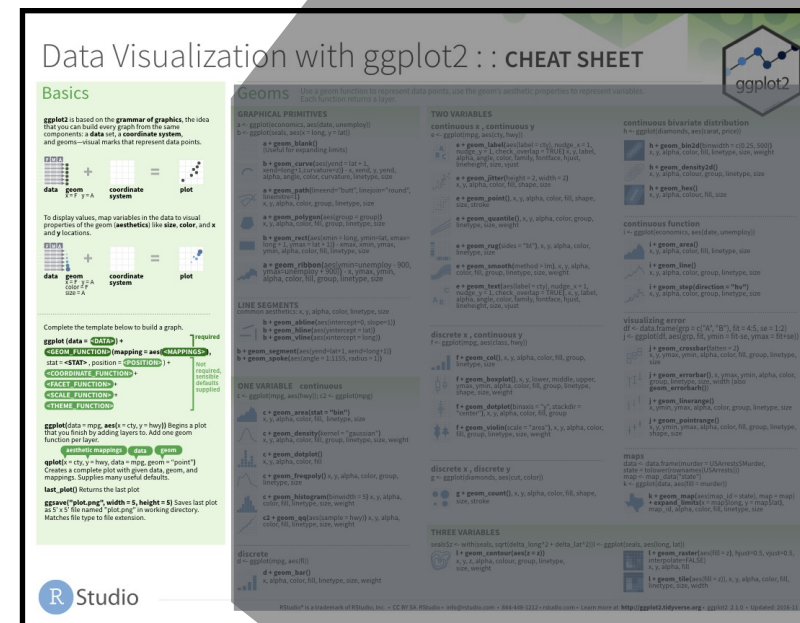




geom_functions

Different bases for
visualization

(Each requires a mapping
argument.)



Geoms

Use a geom function to represent data points, use the geom's aesthetic properties to represent variables.
Each function returns a layer.

GRAPHICAL PRIMITIVES

a <- ggplot(economics, aes(date, unemployment))
b <- ggplot(seals, aes(x = long, y = lat))

a + geom_blank()
(Useful for expanding limits)

b + geom_curve(aes(yend = lat + 1, xend=long+1, curvature=z)) - x, xend, y, yend, alpha, angle, color, curvature, linetype, size

a + geom_path(lineend="butt", linejoin="round", linemitre=1)
x, y, alpha, color, group, linetype, size

a + geom_polygon(aes(group = group))
x, y, alpha, color, fill, group, linetype, size

b + geom_rect(aes(xmin = long, ymin=lat, xmax=long + 1, ymax = lat + 1)) - xmax, xmin, ymax, ymin, alpha, color, fill, linetype, size

a + geom_ribbon(aes(ymin=unemployment - 900, ymax=unemployment + 900)) - x, ymax, ymin, alpha, color, fill, group, linetype, size

LINE SEGMENTS

common aesthetics: x, y, alpha, color, linetype, size

b + geom_abline(aes(intercept=0, slope=1))
b + geom_hline(aes(yintercept = lat))
b + geom_vline(aes(xintercept = long))

b + geom_segment(aes(yend=lat+1, xend=long+1))
b + geom_spoke(aes(angle = 1:1155, radius = 1))

ONE VARIABLE continuous

c <- ggplot(mpg, aes(hwy)); **c2** <- ggplot(mpg)

c + geom_area(stat = "bin")
x, y, alpha, color, fill, linetype, size

c + geom_density(kernel = "gaussian")
x, y, alpha, color, fill, group, linetype, size, weight

c + geom_dotplot()
x, y, alpha, color, fill

c + geom_freqpoly() x, y, alpha, color, group, linetype, size

c + geom_histogram(binwidth = 5) x, y, alpha, color, fill, linetype, size, weight

c2 + geom_qq(aes(sample = hwy)) x, y, alpha, color, fill, linetype, size, weight

discrete

d <- ggplot(mpg, aes(fl))

d + geom_bar()
x, alpha, color, fill, linetype, size, weight

TWO VARIABLES

continuous x, continuous y

e <- ggplot(mpg, aes(cty, hwy))

e + geom_label(aes(label = cty), nudge_x = 1, nudge_y = 1, check_overlap = TRUE) x, y, label, alpha, angle, color, family, fontface, hjust, lineheight, size, vjust

e + geom_jitter(height = 2, width = 2)
x, y, alpha, color, fill, shape, size

e + geom_point(x, y, alpha, color, fill, shape, size, stroke)

e + geom_quantile(x, y, alpha, color, group, linetype, size, weight)

e + geom_rug(sides = "bl", x, y, alpha, color, linetype, size)

e + geom_smooth(method = lm, x, y, alpha, color, fill, group, linetype, size, weight)

e + geom_text(aes(label = cty), nudge_x = 1, nudge_y = 1, check_overlap = TRUE), x, y, label, alpha, angle, color, family, fontface, hjust, lineheight, size, vjust

continuous bivariate distribution

h <- ggplot(diamonds, aes(carat, price))

h + geom_bin2d(binwidth = c(0.25, 500))
x, y, alpha, color, fill, linetype, size, weight

h + geom_density2d()
x, y, alpha, colour, group, linetype, size

h + geom_hex()
x, y, alpha, colour, fill, size

continuous function

i <- ggplot(economics, aes(date, unemployment))

i + geom_area()
x, y, alpha, color, fill, linetype, size

i + geom_line()
x, y, alpha, color, group, linetype, size

i + geom_step(direction = "hv")
x, y, alpha, color, group, linetype, size

visualizing error

df <- data.frame(grp = c("A", "B"), fit = 4:5, se = 1:2)
j <- ggplot(df, aes(grp, fit, ymin = fit-se, ymax = fit+se))

j + geom_crossbar(fatten = 2)
x, y, ymax, ymin, alpha, color, fill, group, linetype, size

j + geom_errorbar() x, ymax, ymin, alpha, color, group, linetype, size, width (also **geom_errorbarh**())

j + geom_linerange()
x, ymin, ymax, alpha, color, group, linetype, size

j + geom_pointrange()
x, y, ymin, ymax, alpha, color, fill, group, linetype, shape, size

discrete x, continuous y

f <- ggplot(mpg, aes(class, hwy))

f + geom_col() x, y, alpha, color, fill, group, linetype, size

f + geom_boxplot() x, y, lower, middle, upper, ymax, ymin, alpha, color, fill, group, linetype, shape, size, weight

f + geom_dotplot(binaxis = "y", stackdir = "center"), x, y, alpha, color, fill, group

f + geom_violin(scale = "area"), x, y, alpha, color, fill, group, linetype, size, weight

discrete x, discrete y

g <- ggplot(diamonds, aes(cut, color))

g + geom_count() x, y, alpha, color, fill, shape, size, stroke

THREE VARIABLES

seals\$z <- with(seals, sqrt(delta_long^2 + delta_lat^2)); **l** <- ggplot(seals, aes(long, lat))

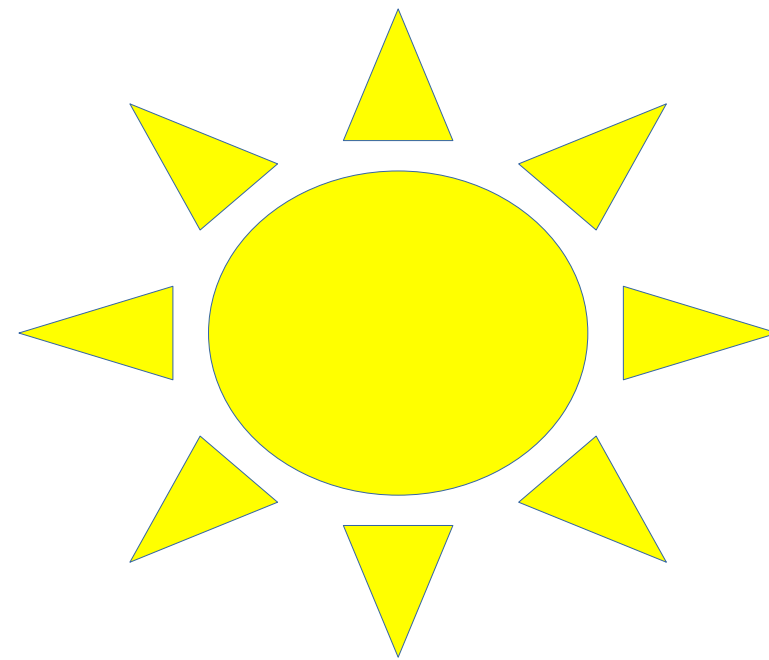
l + geom_contour(aes(z = z))
x, y, z, alpha, colour, group, linetype, size, weight

l + geom_raster(aes(fill = z), hjust=0.5, vjust=0.5, interpolate=FALSE)
x, y, alpha, fill

l + geom_tile(aes(fill = z)), x, y, alpha, color, fill, linetype, size, width

Graph Gallery!

<https://r-graph-gallery.com/>



To make a graph

1. Pick a **data** set

[template]

```
ggplot(data = <DATA>) +  
  <GEOM_FUNCTION>(mapping = aes(<MAPPINGS>))
```

2. Choose a **geom** to display cases

3. **Map** aesthetic properties to variables

Histograms and Bar Charts

Histogram

data

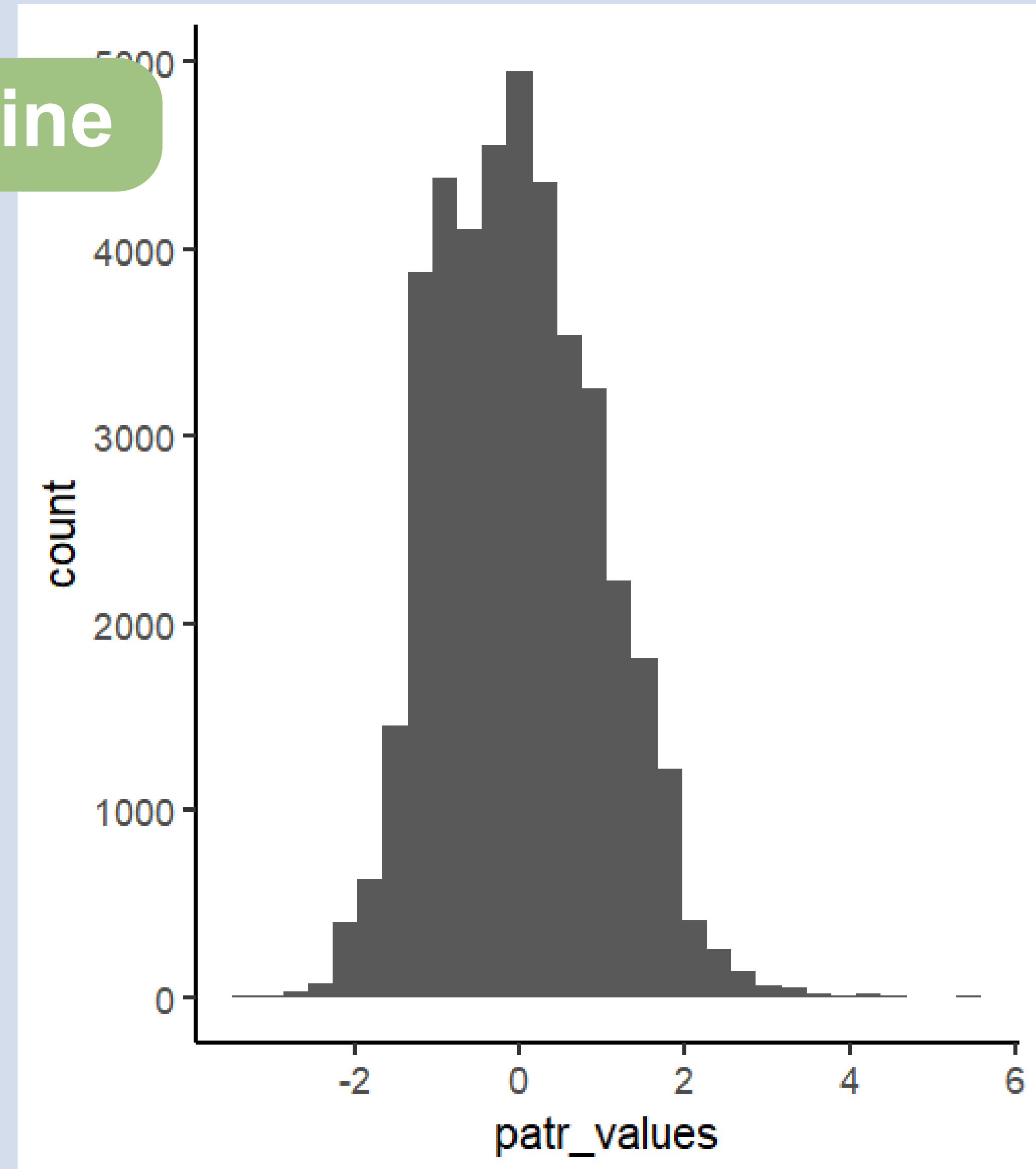
Aesthetic mappings

+ before new line

```
ggplot(flp, aes(pat_ values)) +  
  geom_histogram()
```

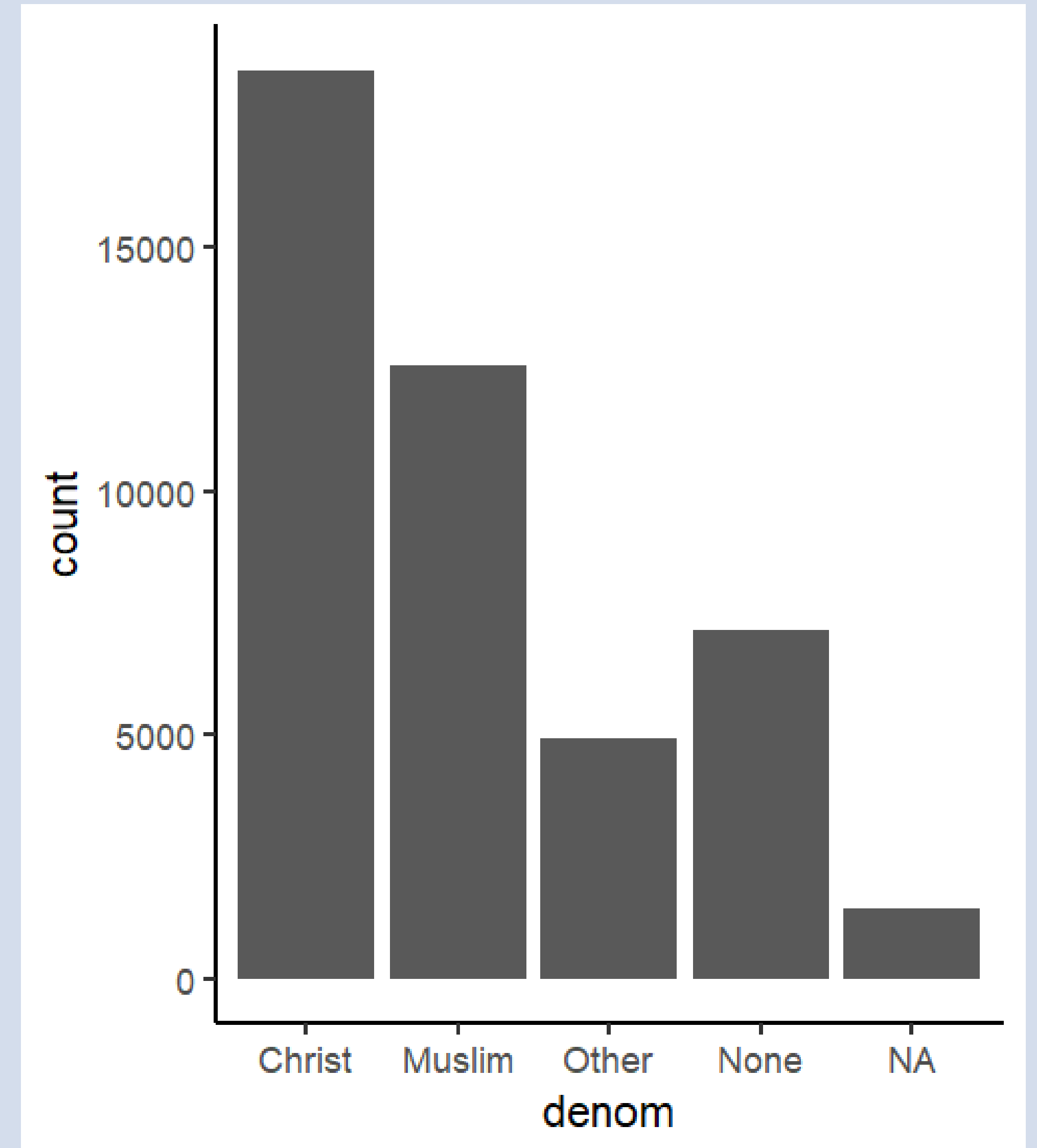
type of layer

The geom
(style, shape)
of the graph



Bar chart - frequencies

```
ggplot(flp, aes(denom)) +  
  geom_bar()
```



Scatterplots

data

+ before new line

x variable

```
ggplot(data = flfp_agg) +  
  geom_point(mapping = aes(x = patr_mean,  
                           y = wvs_flfp))
```

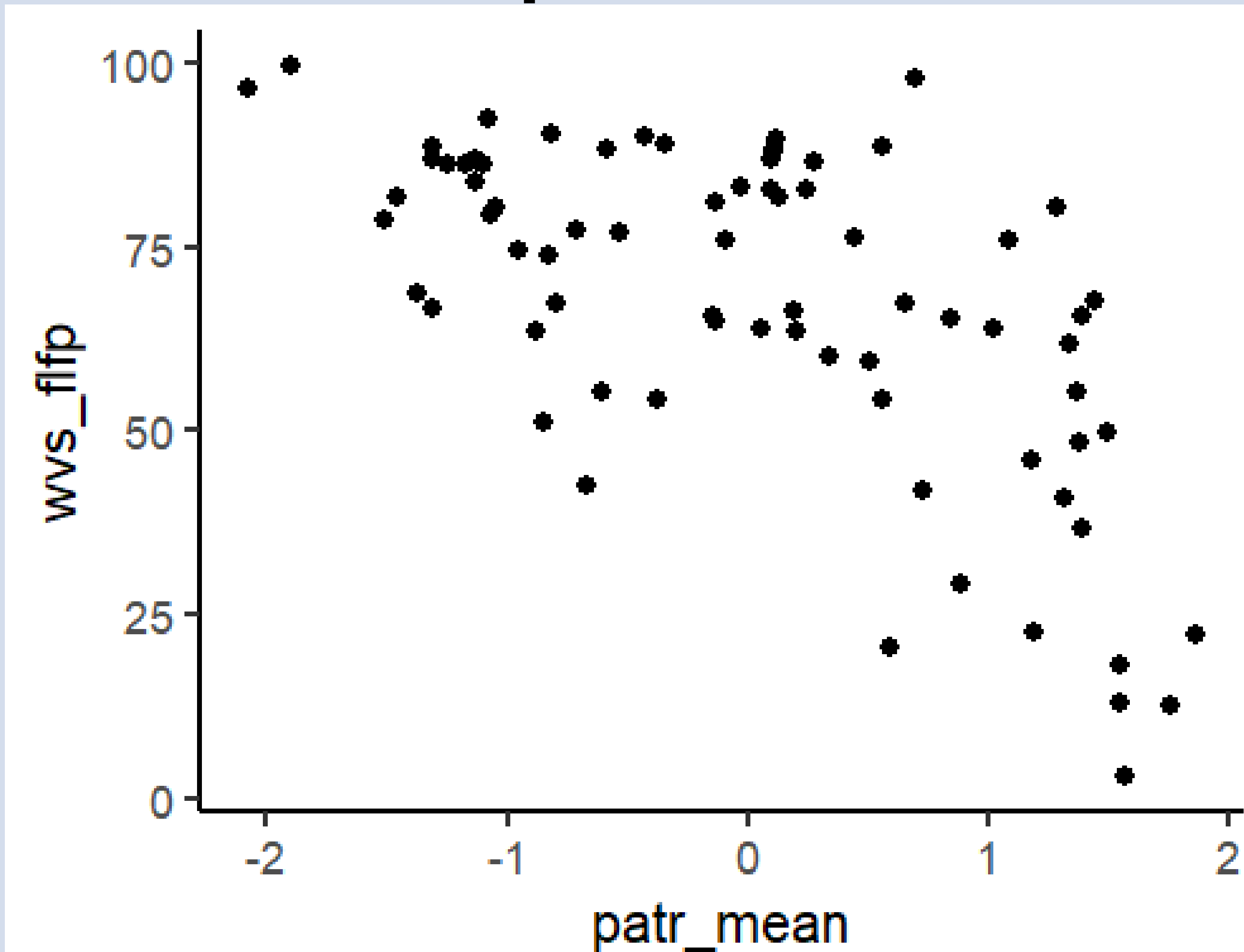
**type of layer,
geom**

**aesthetic
mapping**

y variable

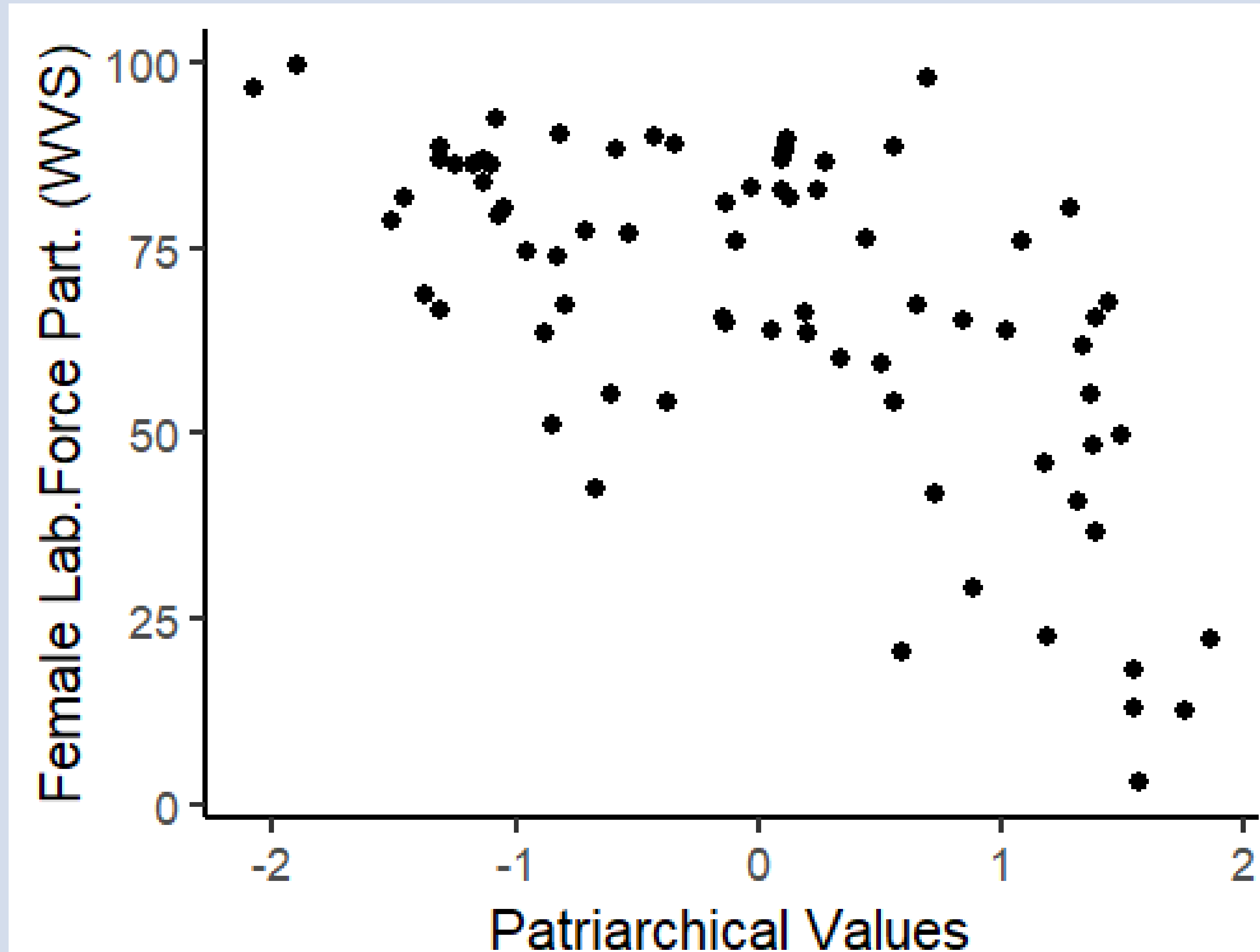
Basic Scatterplot

```
ggplot(flfp_agg, aes(x=patr_mean,  
y=wvs_flfp)) +  
  geom_point()
```



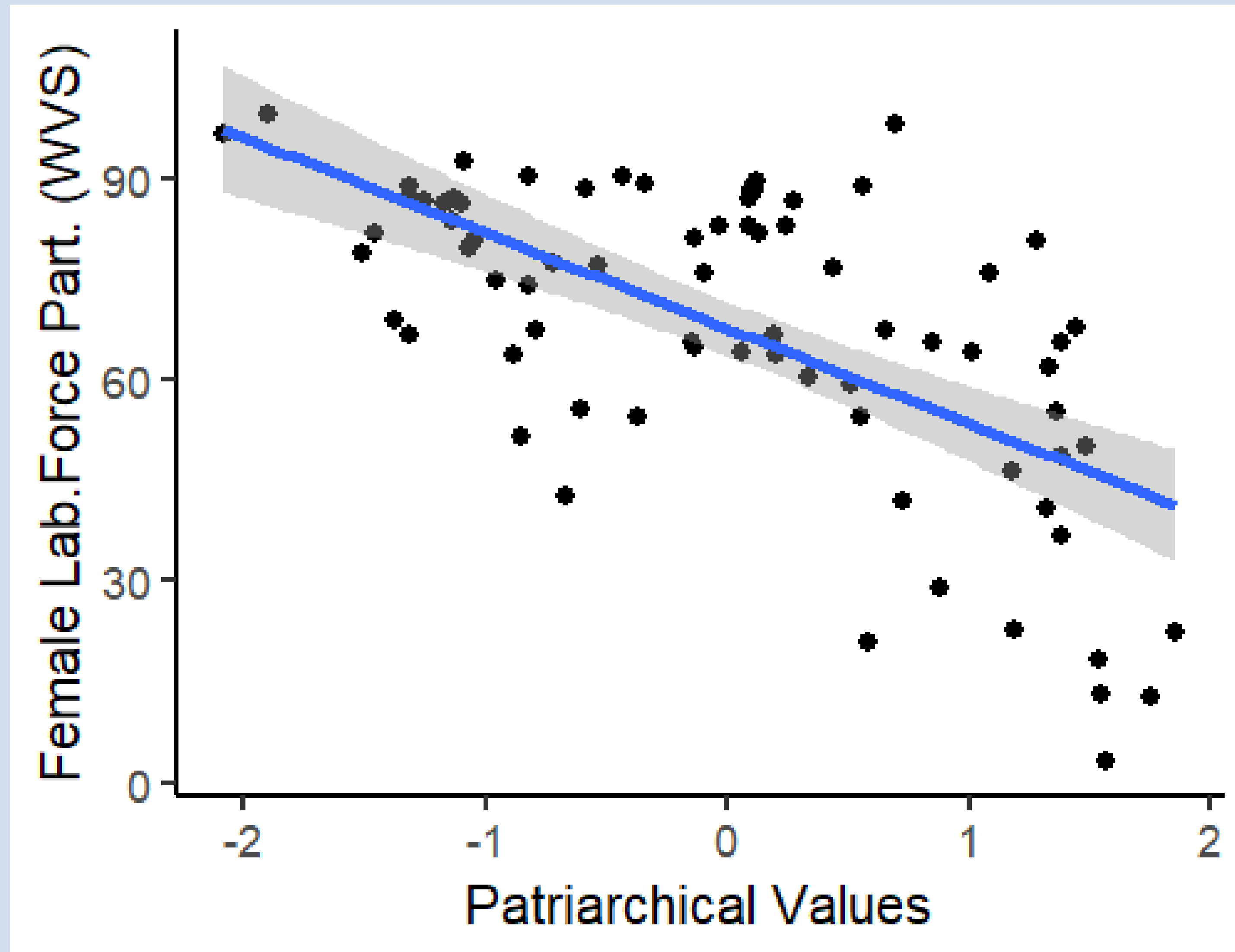
Fix axis labels

```
ggplot(flfp_agg, aes(x=patr_mean,  
y=wvs_flfp)) +  
  geom_point()+  
  labs(y= "Female Lab.Force Part.  
(WVS)", x = "Patriarchal Values")
```



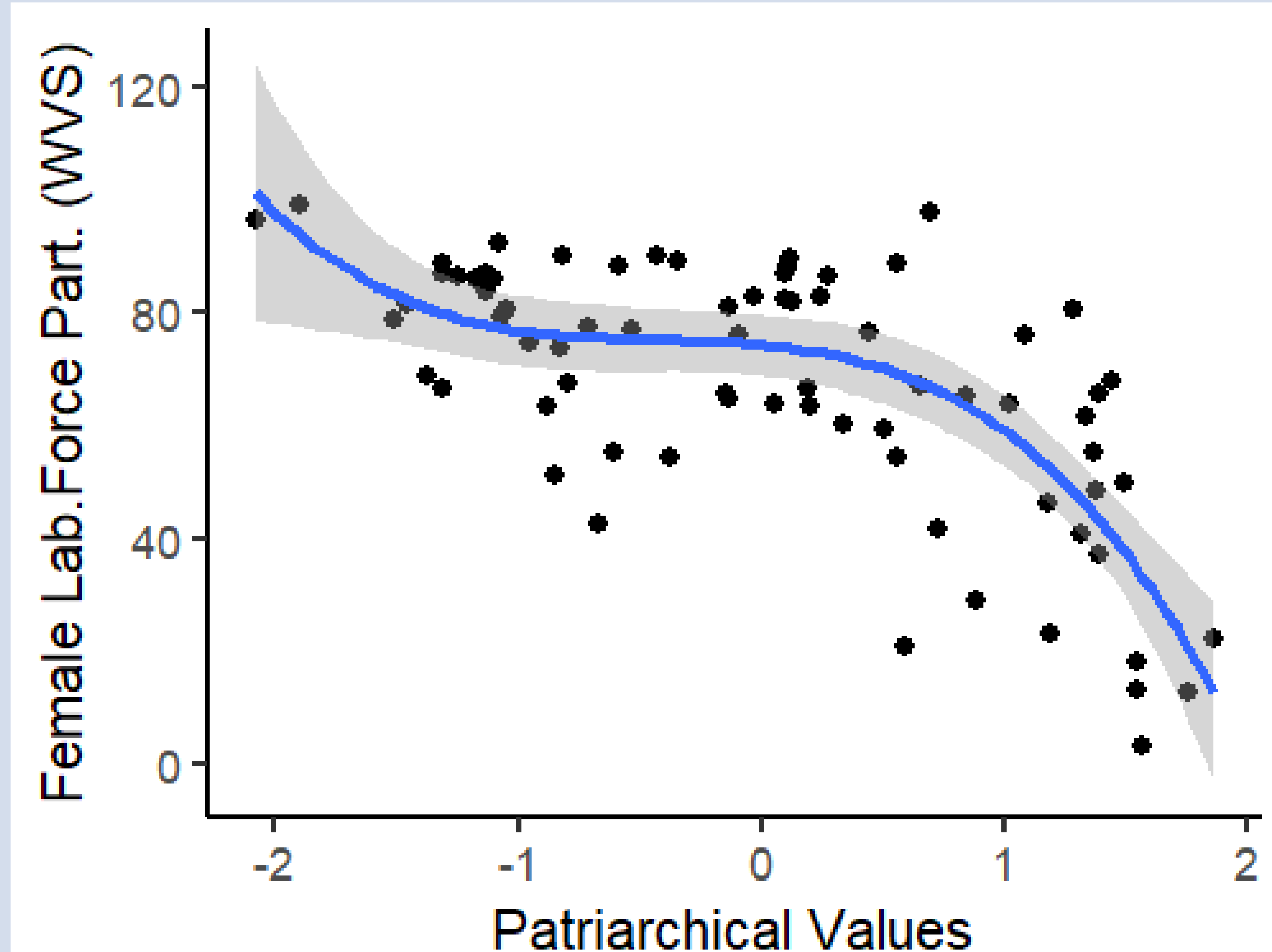
Add a regression line (with confidence interval)

```
ggplot(flfp_agg, aes(x=patr_mean,  
y=wvs_flfp)) +  
  geom_point()+  
  labs(y= "Female Lab.Force Part.  
(WVS)", x = "Patriarchal Values")  
+  
  geom_smooth(method = "lm", se  
= T)
```



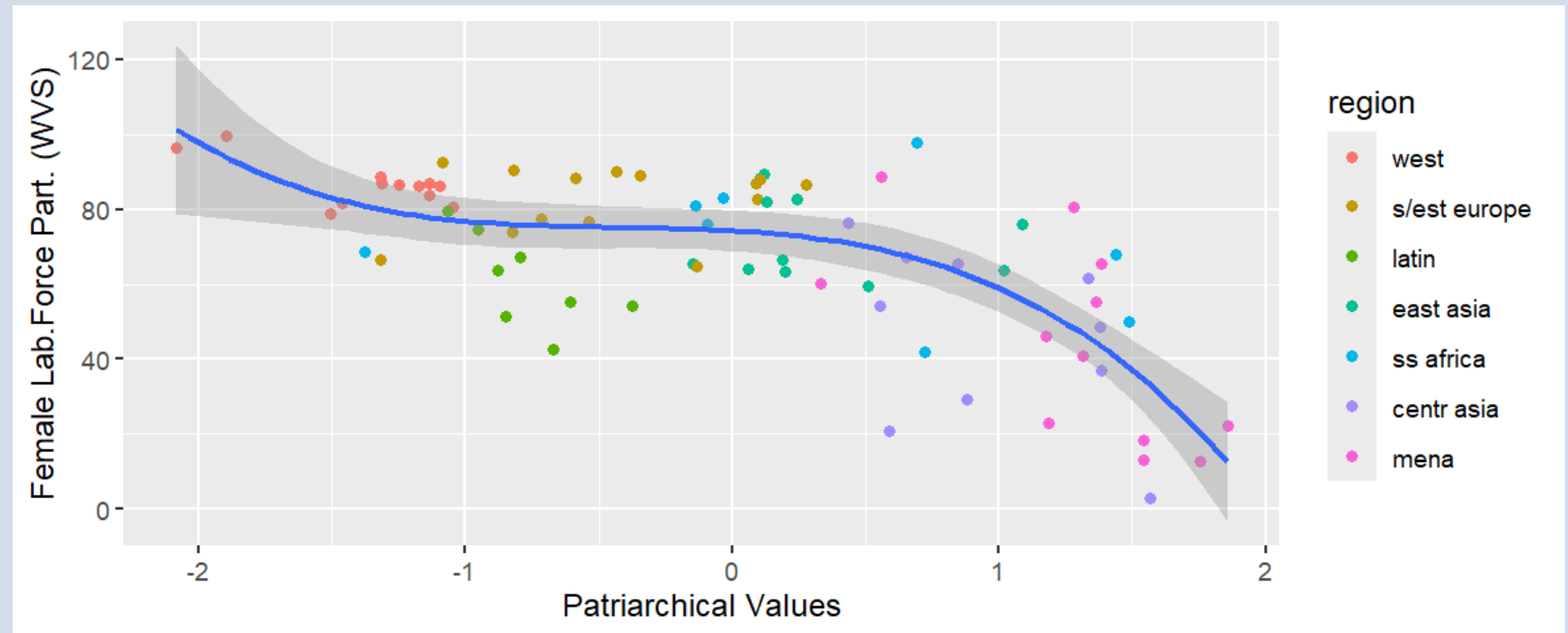
Add a curvier regression line

```
ggplot(flfp_agg, aes(x=patr_mean,  
y=wvs_flfp)) +  
  geom_point()+  
  labs(y= "Female Lab.Force Part.  
(WVS)", x = "Patriarchal Values")  
+  
  geom_smooth(method = "lm", se  
= T, formula = y ~ poly(x, 3))
```

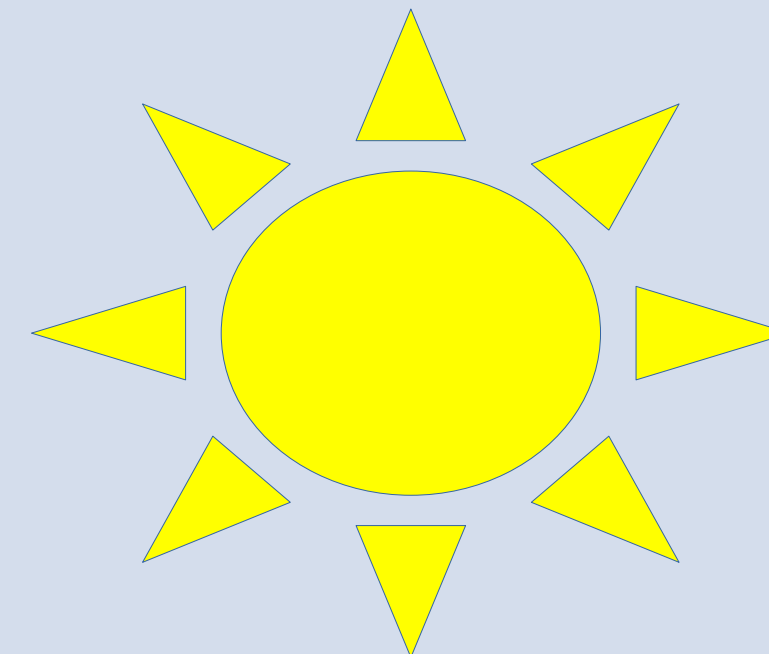


Highlight Region Differences

```
ggplot(flfp_agg, aes(x=patr_mean,  
y=wvs_flfp)) +  
  geom_point(aes(colour = region))  
+  
  labs(y= "Female Lab.Force Part.  
(WVS)", x = "Patriarchal Values")  
+  
  geom_smooth(method = "lm", se  
= T, formula = y ~ poly(x, 3))
```

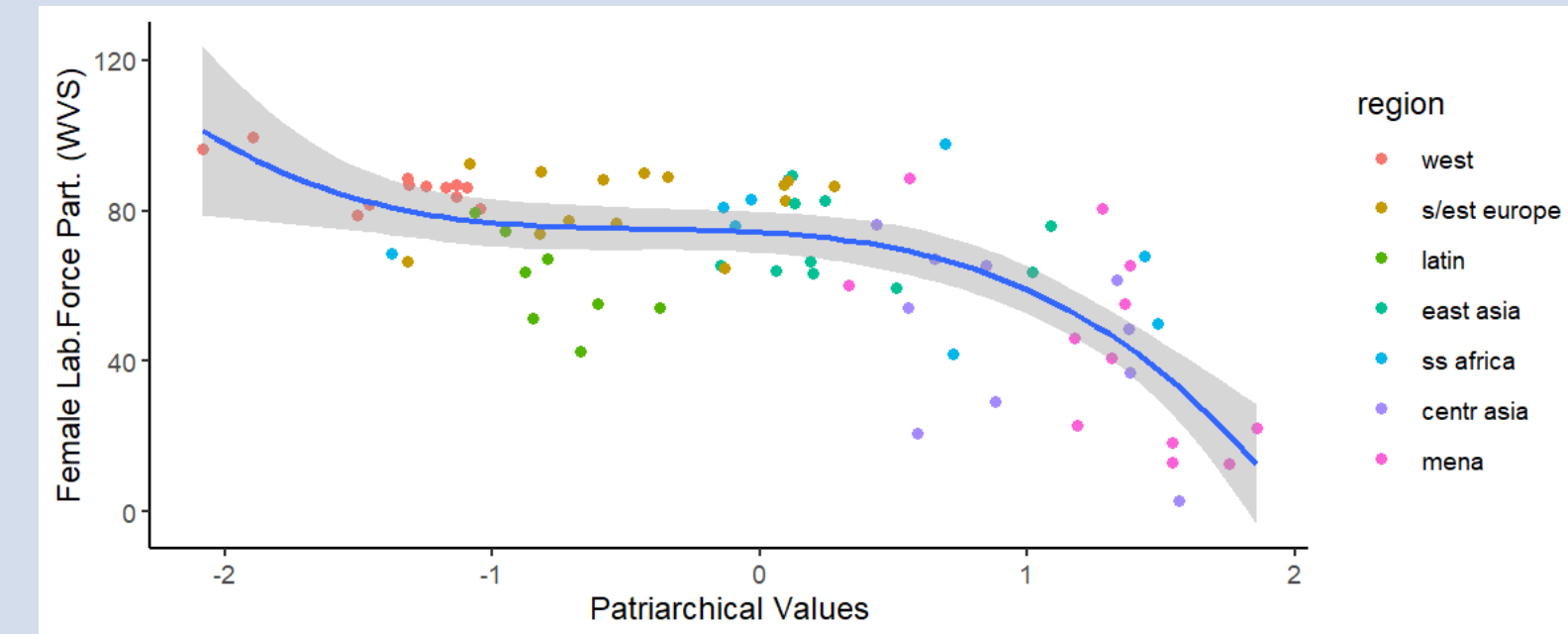
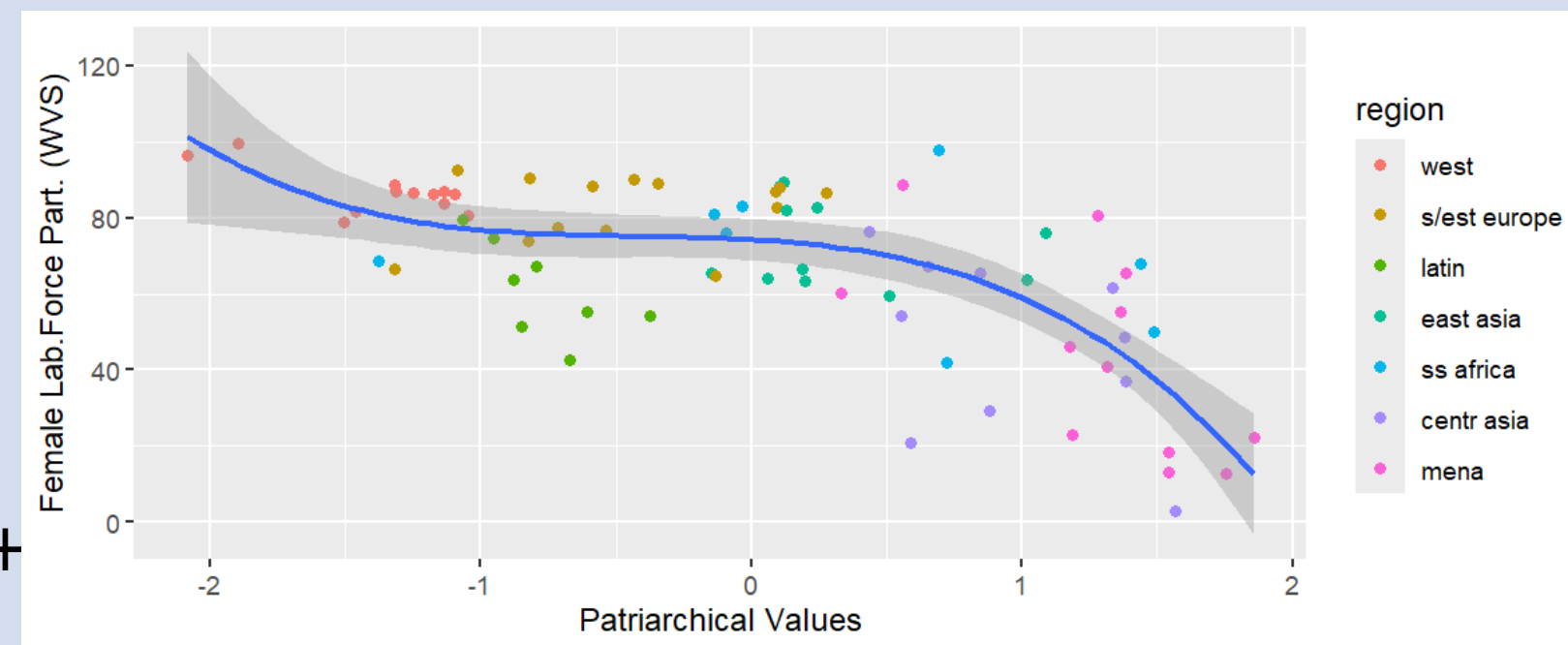


We had to add this aesthetic to geom_point! If we would add it to ggplot() on top, it would impact EVERYTHING, including the regression line, thus splitting it.

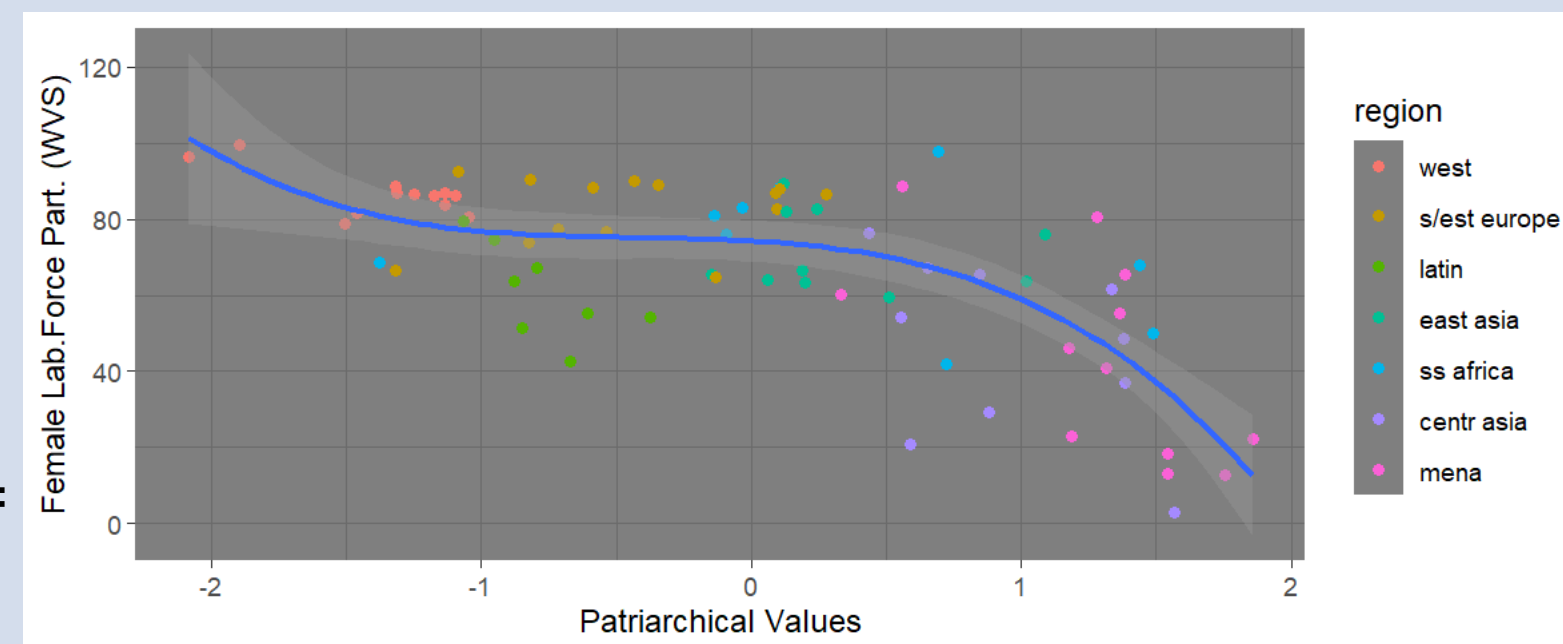


Change stylistic *theme*

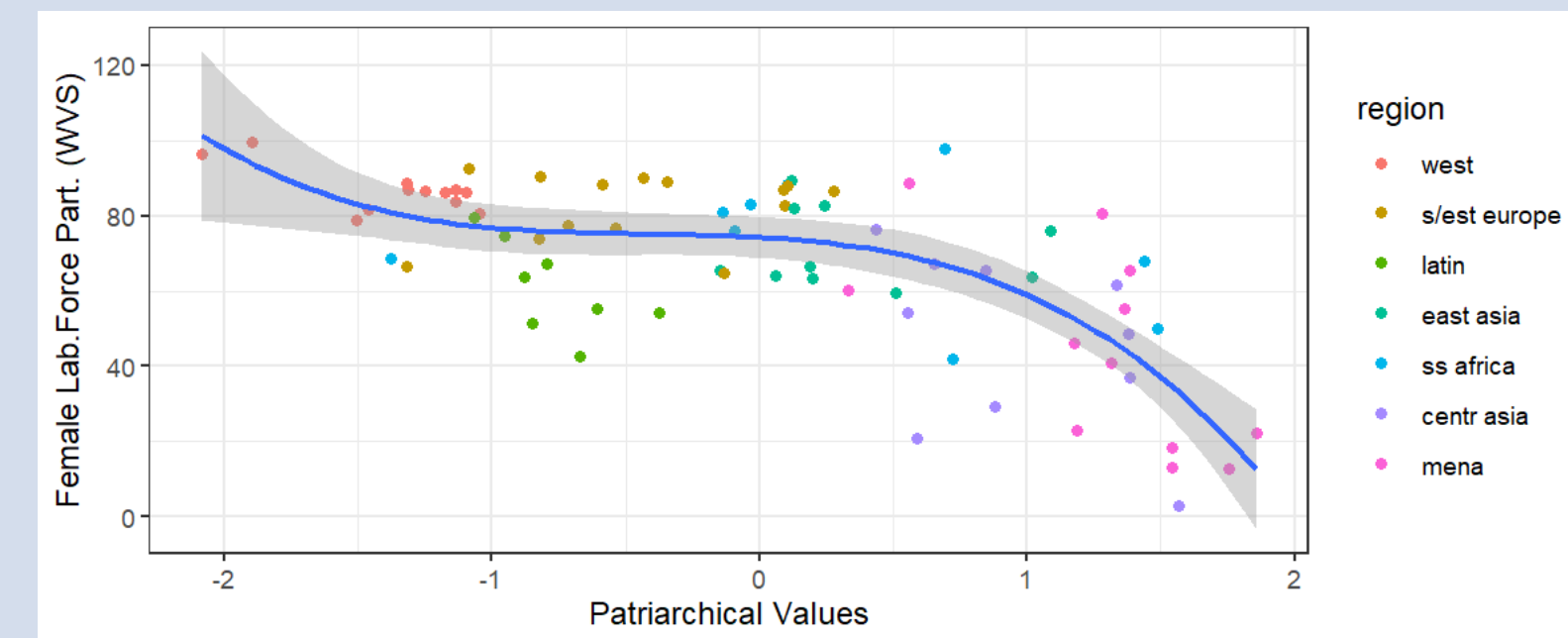
```
ggplot(flp_agg, aes(x=patr_mean, y=wvs_flfp)) +
  geom_point(aes(colour = region))+
  theme_classic() +
  labs(y= "Female Lab.Force Part. (WVS)",
       x = "Patriarchal Values") +
  geom_smooth(method = "lm", se = T, formula =
y ~ poly(x, 3))
```



`theme_classic()`



`theme_dark()`



`theme_bw()`

Lots of others can be found online. There are also ways to specify all of these manually

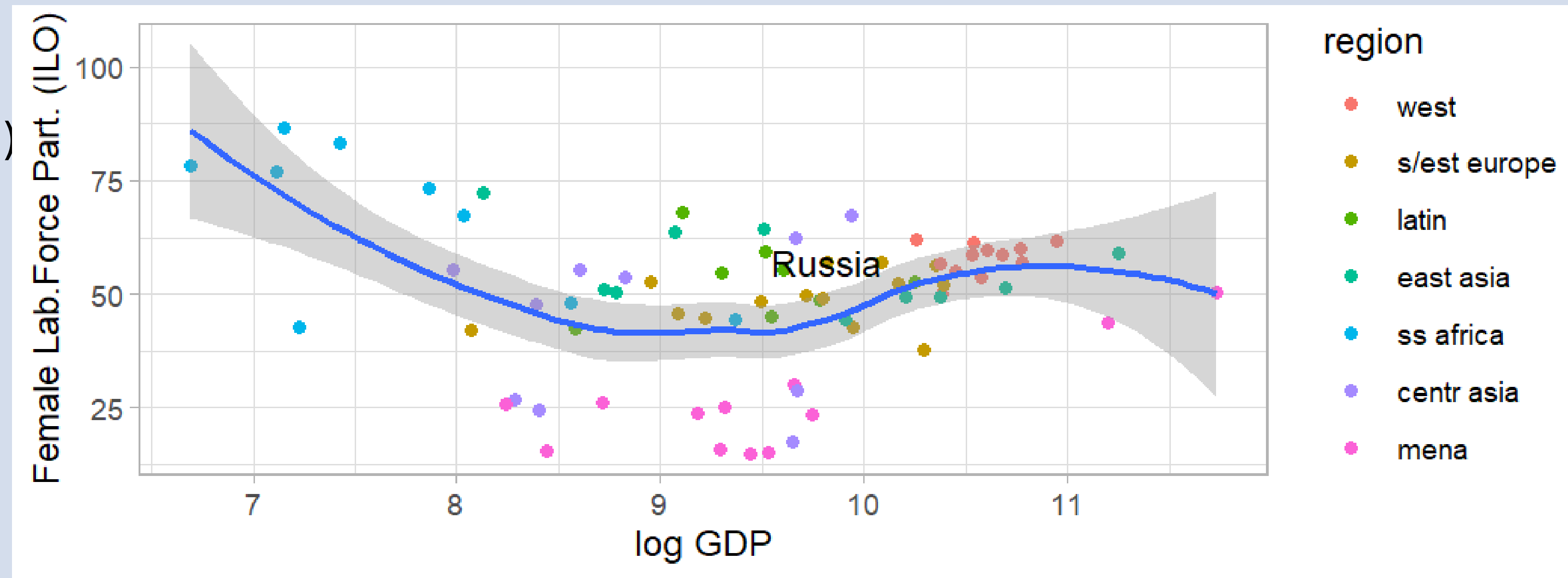
Your Turn

- Use `ggplot` with `flfp_agg` to make a scatterplot with `log_gdp` on the x-axis and `ilo_flfp` on the y-axis. Add an unspecified smoothed line (no method), with a confidence interval. Split by region, and use the `theme_light()`. Adapt your axis labels. Add a special label to the to a country name of your choosing (*cntry*) using `geom_text()`.

Highlight Region Differences

```
ggplot(flfp_agg, aes(x=log_gdp,  
y=ilo_flfp)) +  
  geom_point(aes(colour = region)) +  
  theme_light() +  
  labs(y= "Female Lab.Force Part.  
(ILO)", x = "log GDP") +  
  geom_smooth(se = T)+  
  geom_text(data=filter(flfp_agg,  
cntry=="Russia"), aes(label=cntry))
```

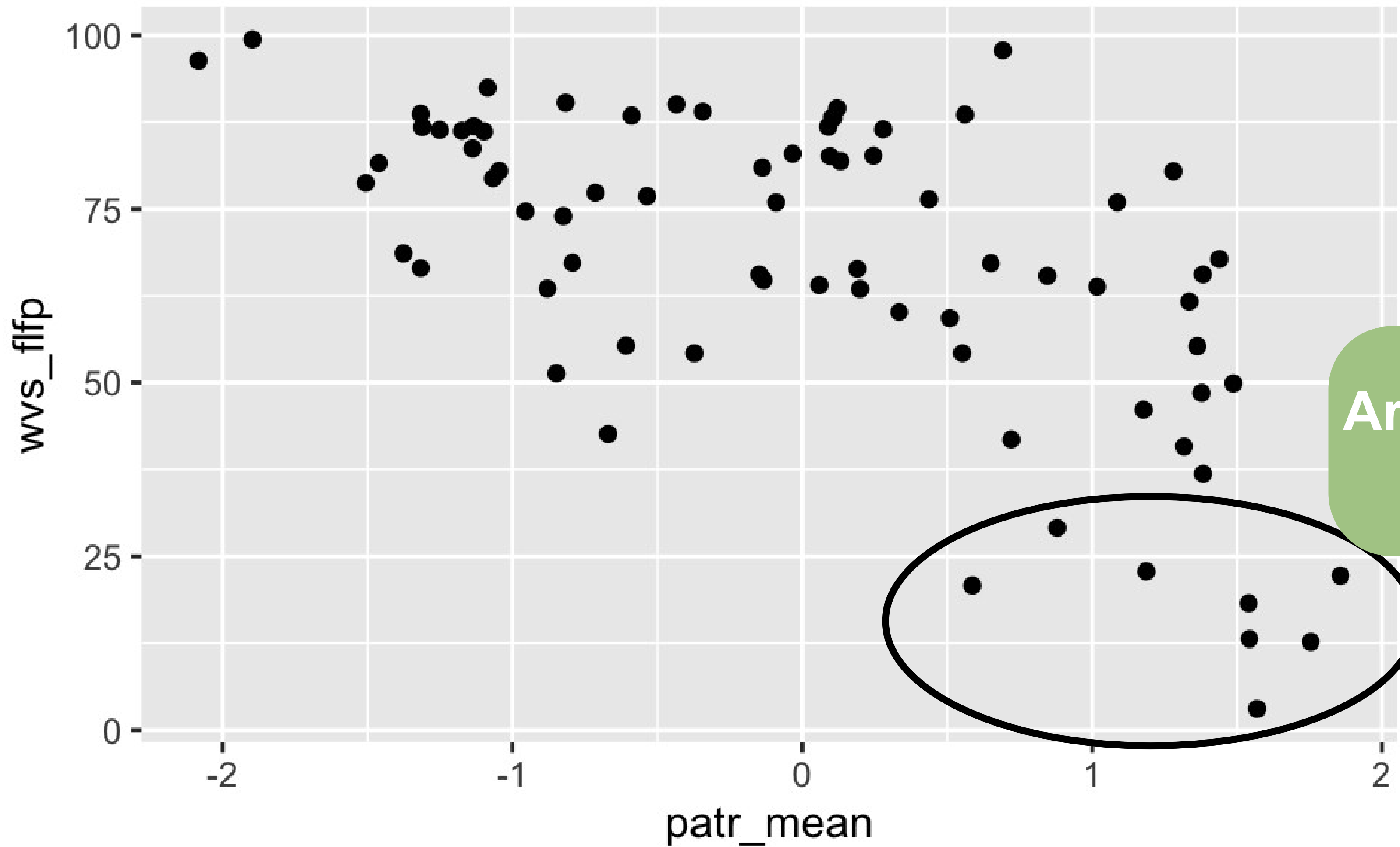
Solution is to use a combination of geom_text, filter() to limit the data under consideration, and aes(label=cntry) to add the label(s).



Layering

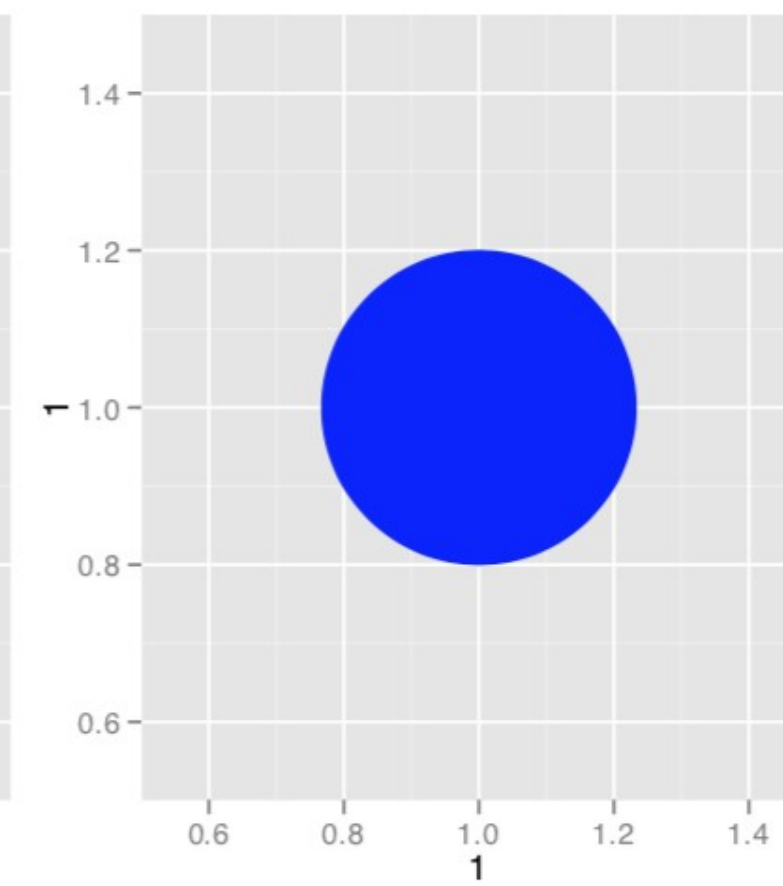
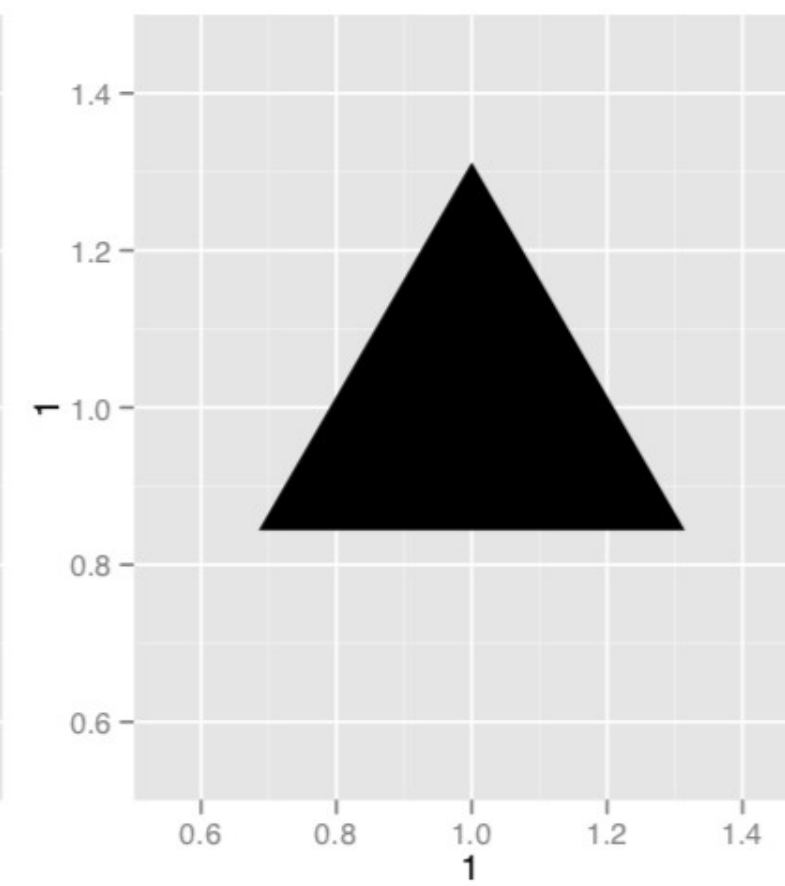
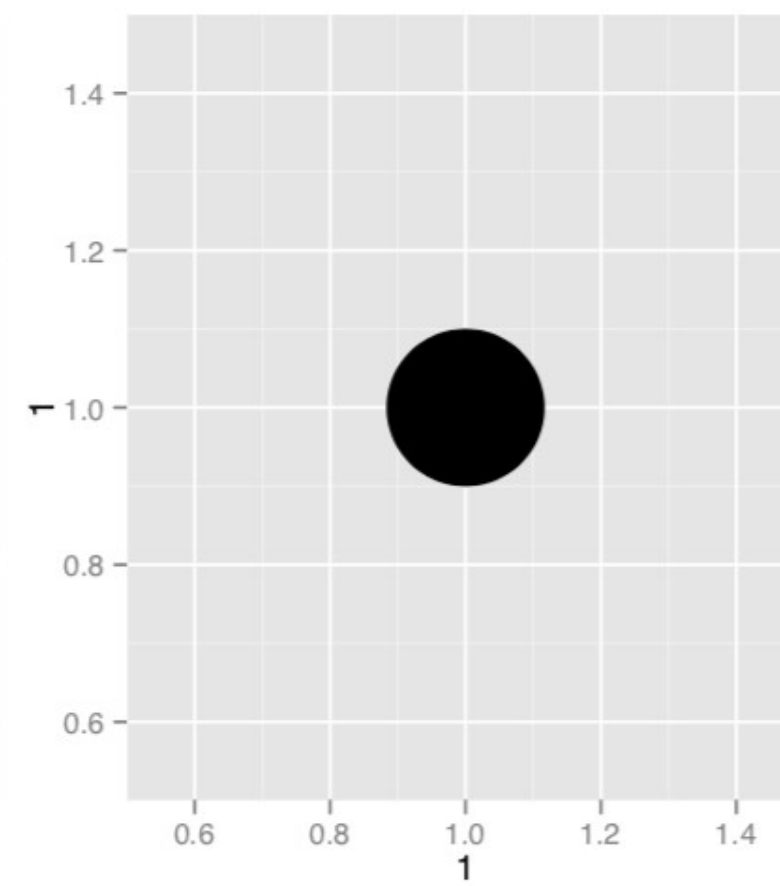
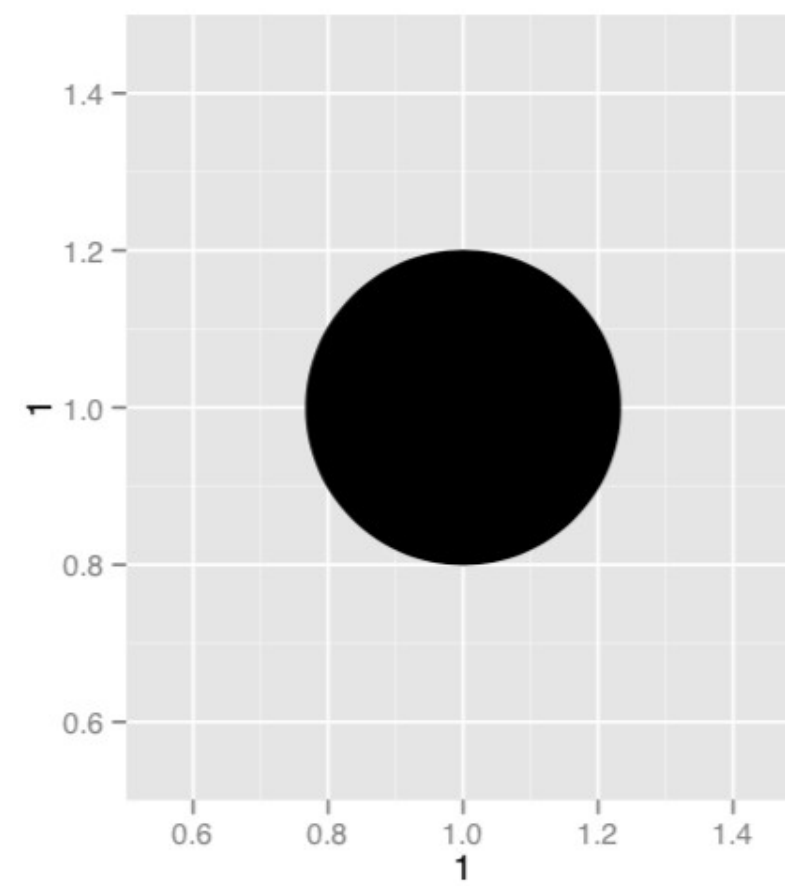
Using this same method, we can adapt ANYTHING and everything about this plot. e.g. the font, where the label appears in relation to the point. We can connect the two with a line, offset them so they do not overlap. Change the legend...etc etc .etc

Mappings



```
ggplot(flfp_agg) +  
  geom_point(aes(x = patr_mean, y = wvs_flfp))
```

Aesthetics

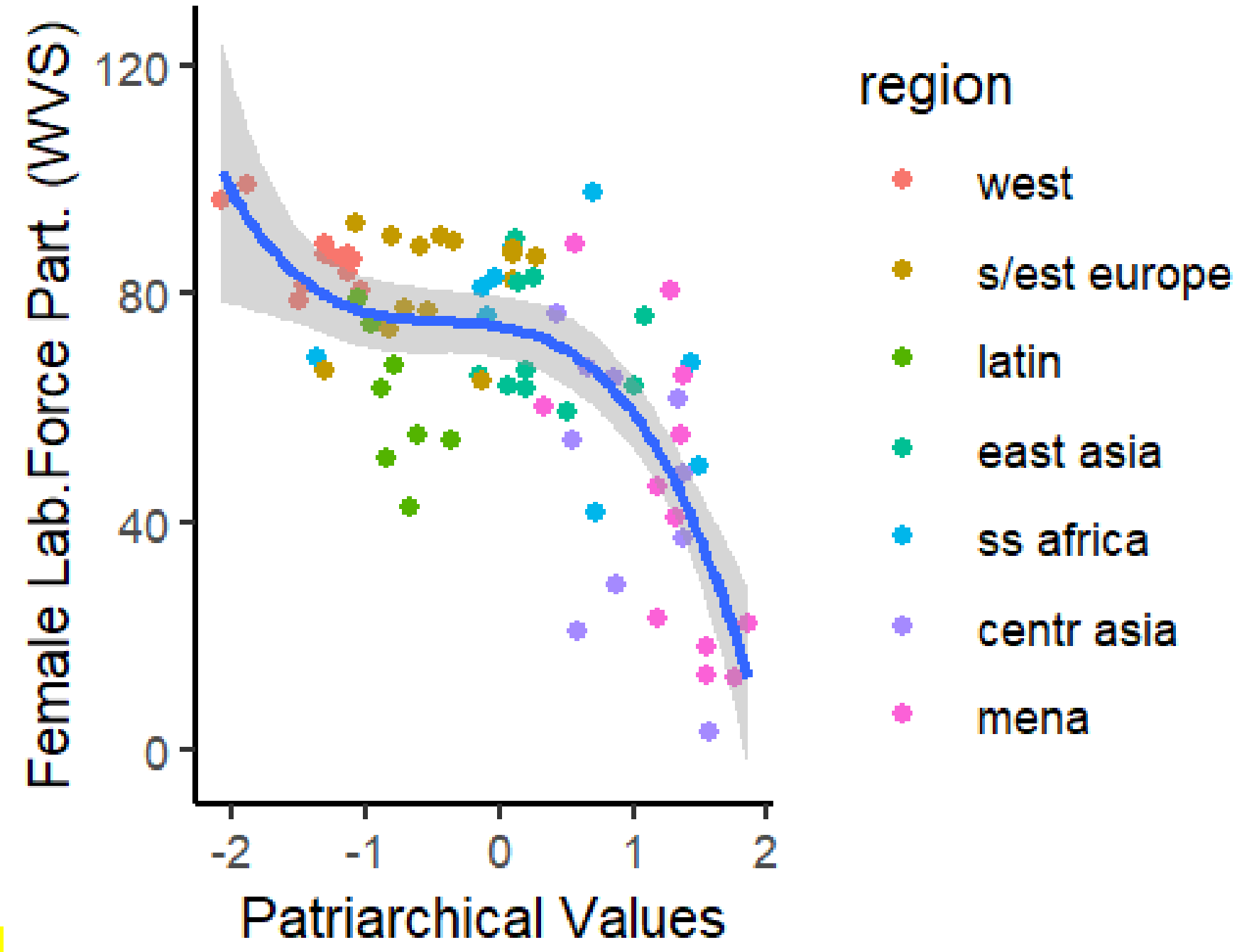


Aesthetics

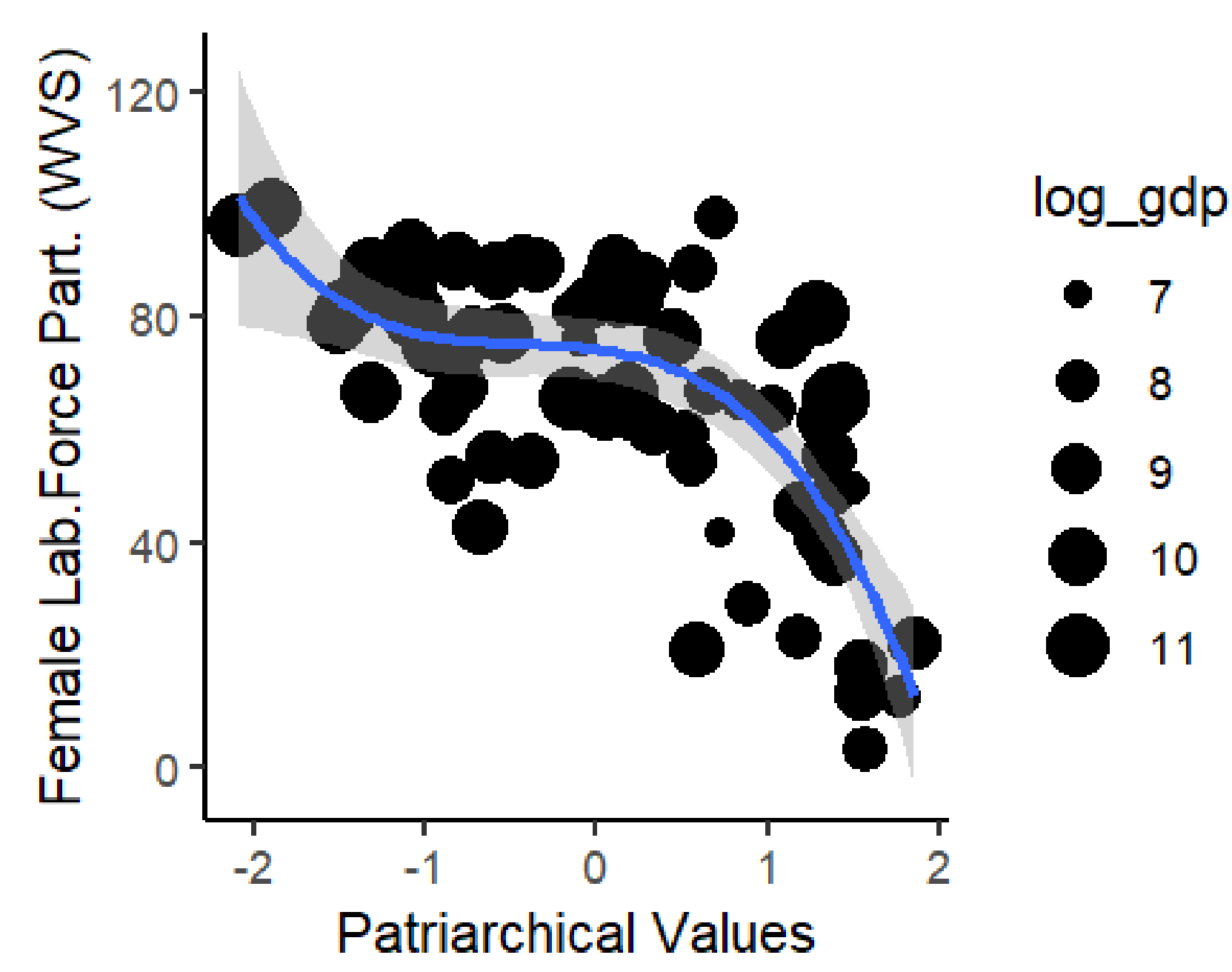
aesthetic
property

variable to
map it to

```
ggplot(flfp_agg) + geom_point(aes(x = patr_mean, y = wvs_flfp, color = region))  
ggplot(flfp_agg) + geom_point(aes(x = patr_mean, y = wvs_flfp, size = log_gdp))  
ggplot(flfp_agg) + geom_point(aes(x = patr_mean, y = wvs_flfp, shape = region))  
ggplot(flfp_agg) + geom_point(aes(x = patr_mean, y = wvs_flfp, alpha = log_gdp))
```



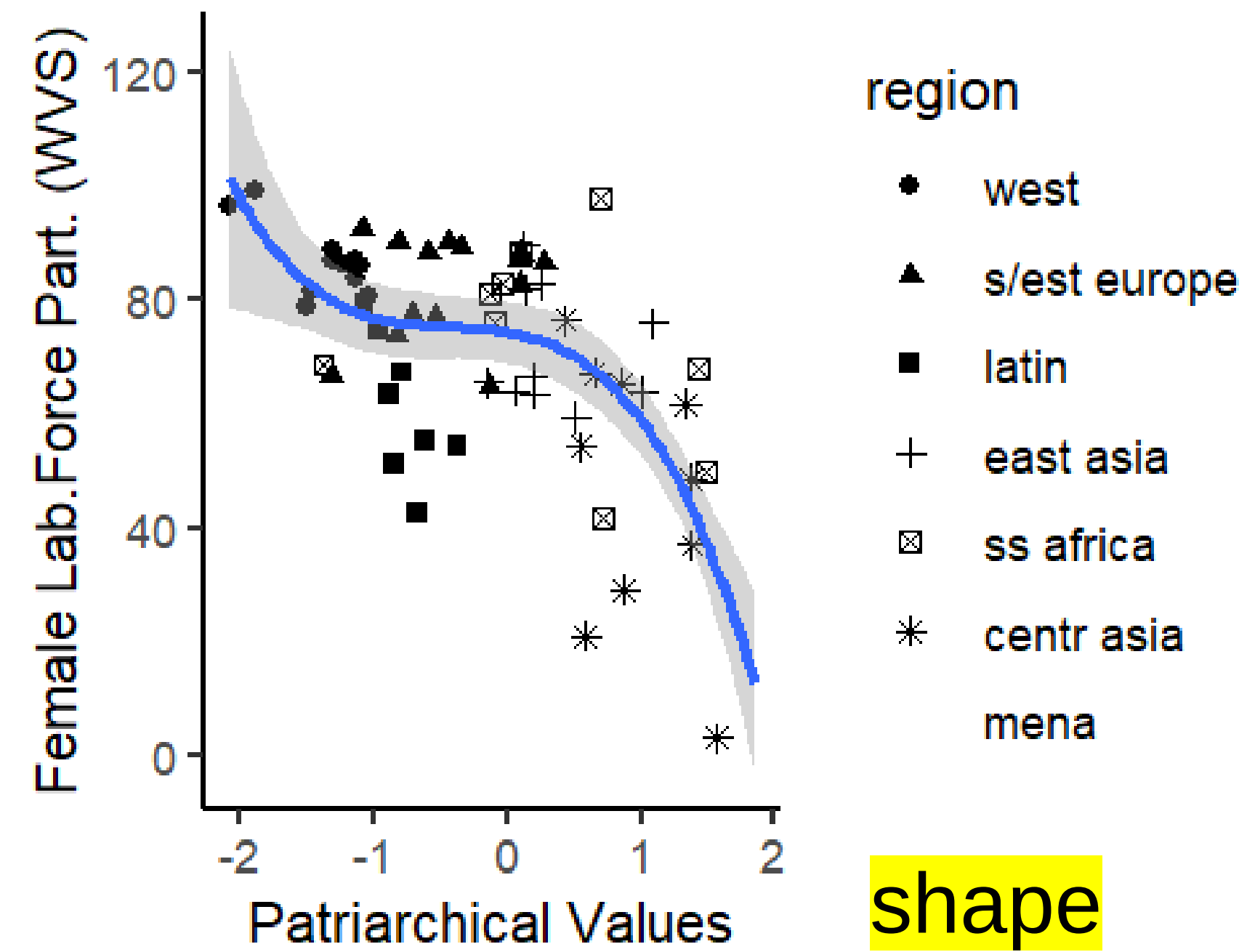
color



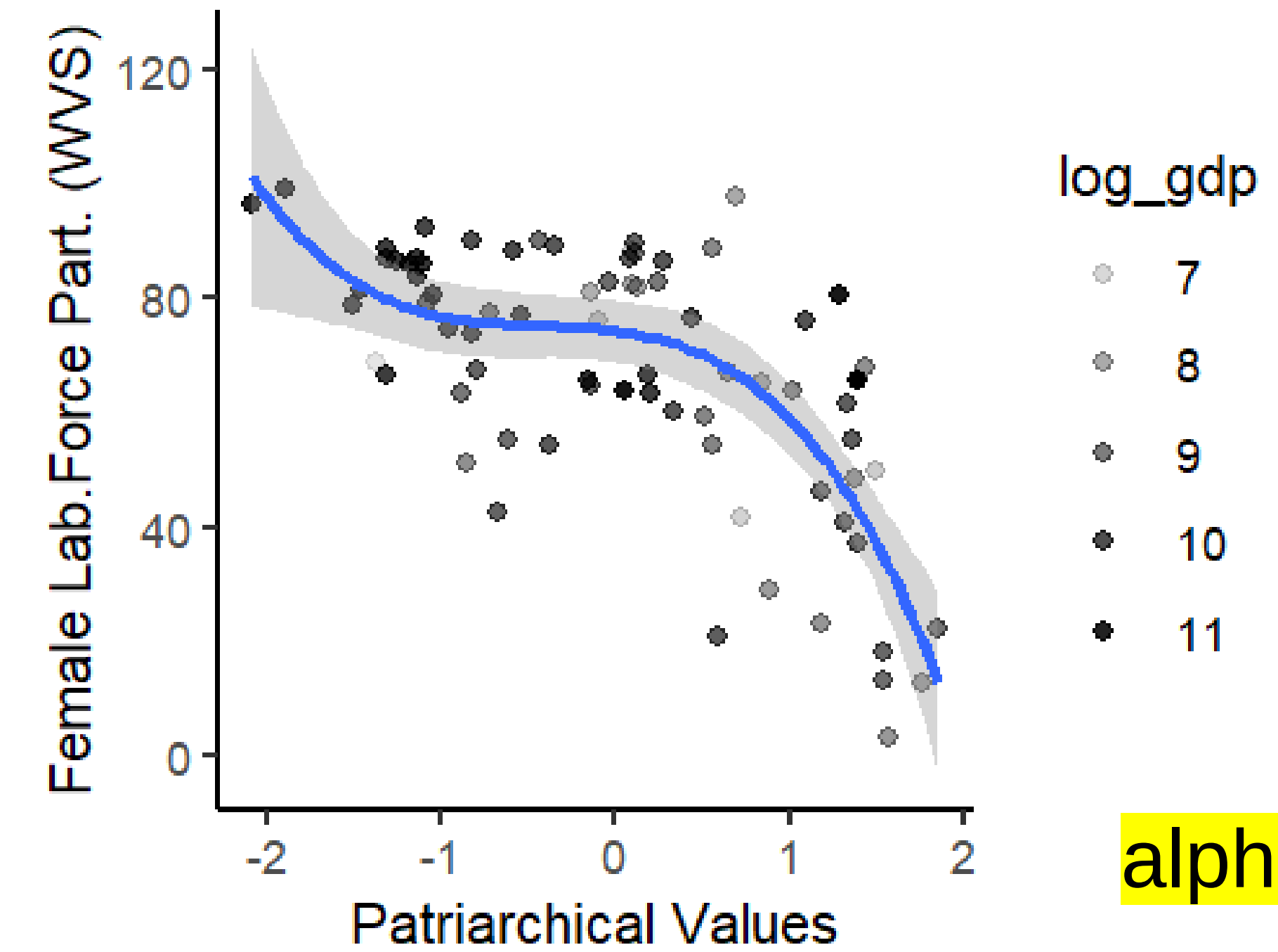
size

Legends added automatically

size and alpha more appropriate for non-categorical variables



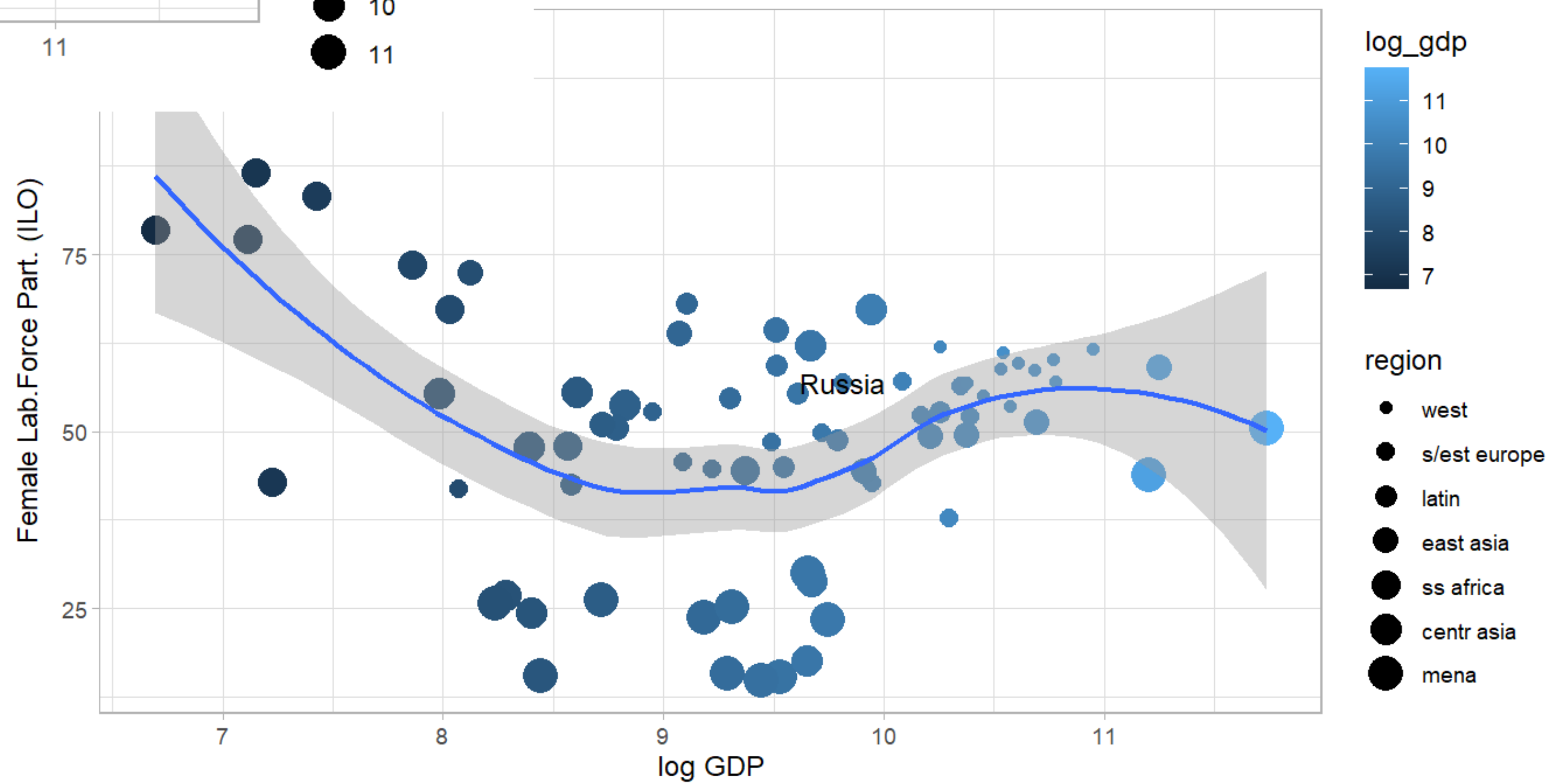
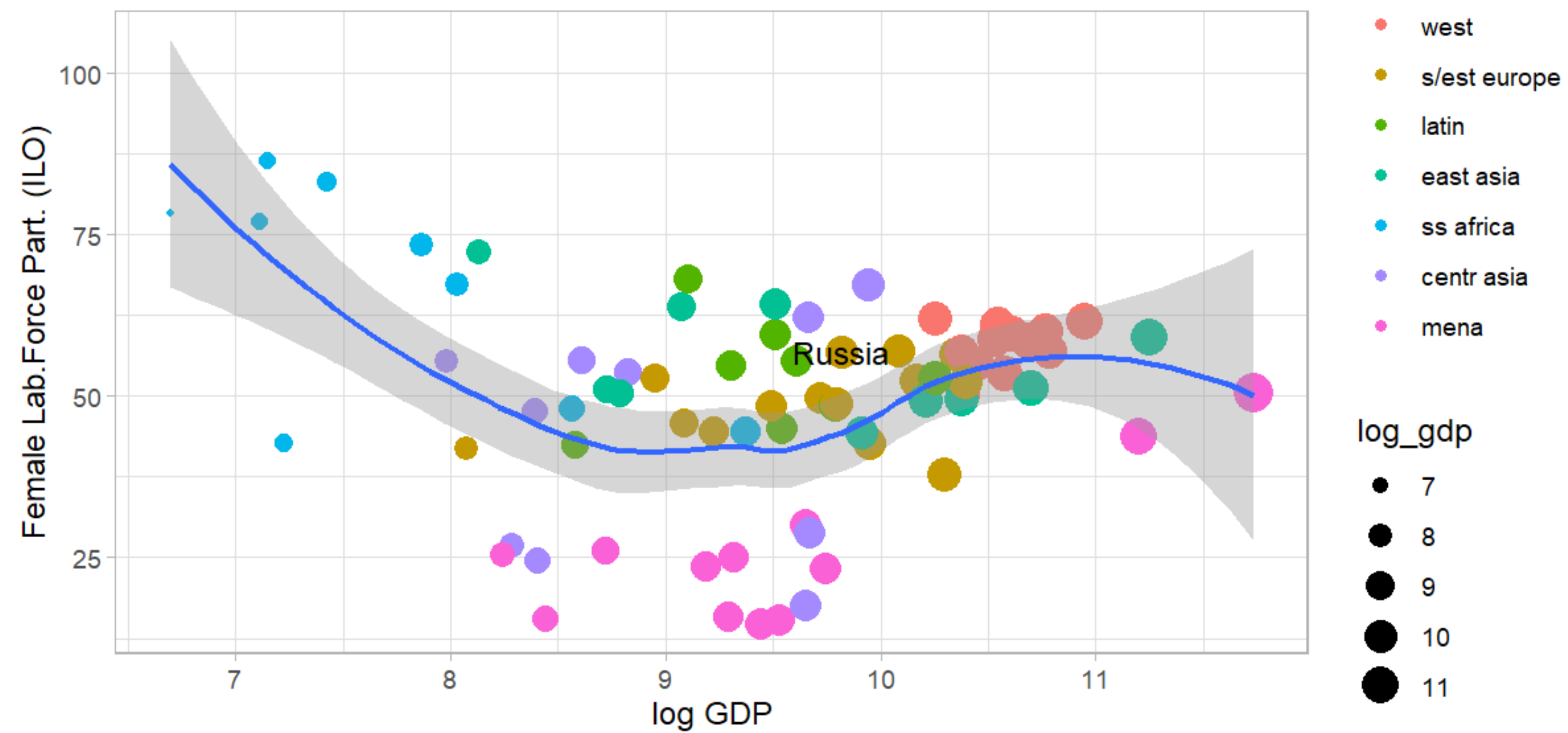
shape



alpha

Your turn

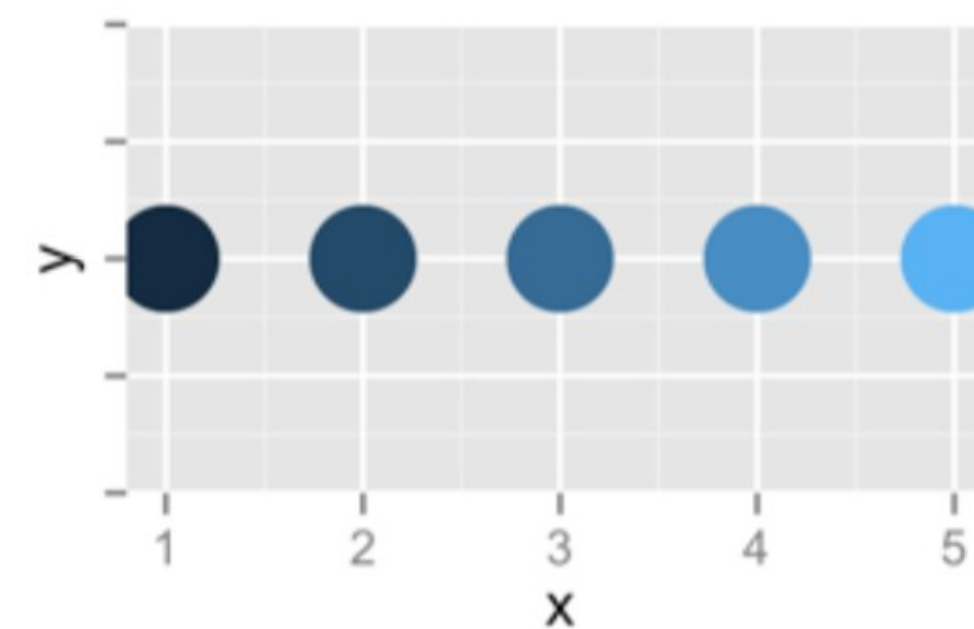
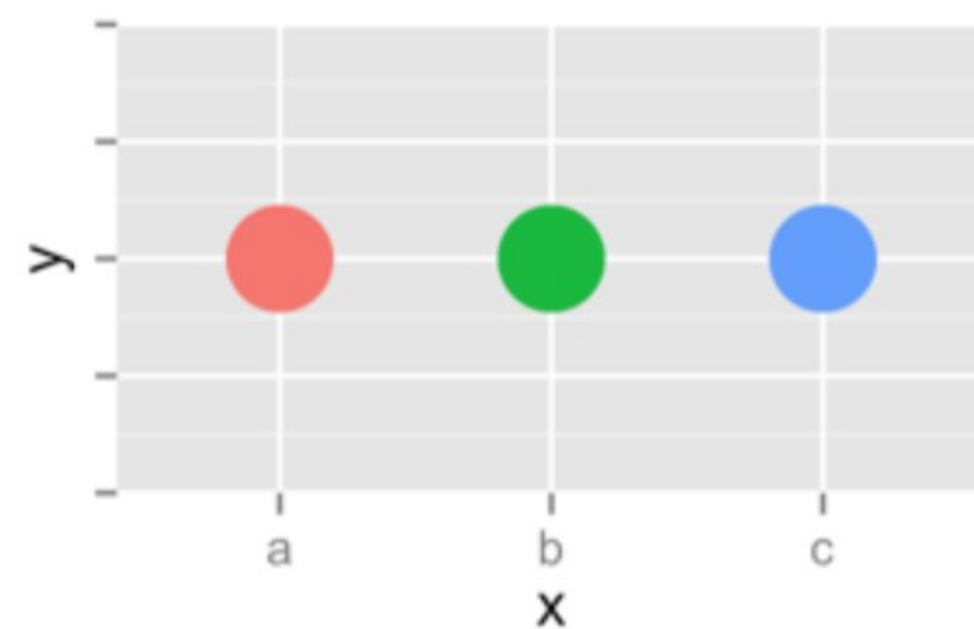
- In the next chunk, add colour, size, alpha, and shape aesthetics to your graph from your last individual exercise. Experiment.
- What happens when you map aesthetics to discrete and continuous variables?
- What happens when you use more than one aesthetic?



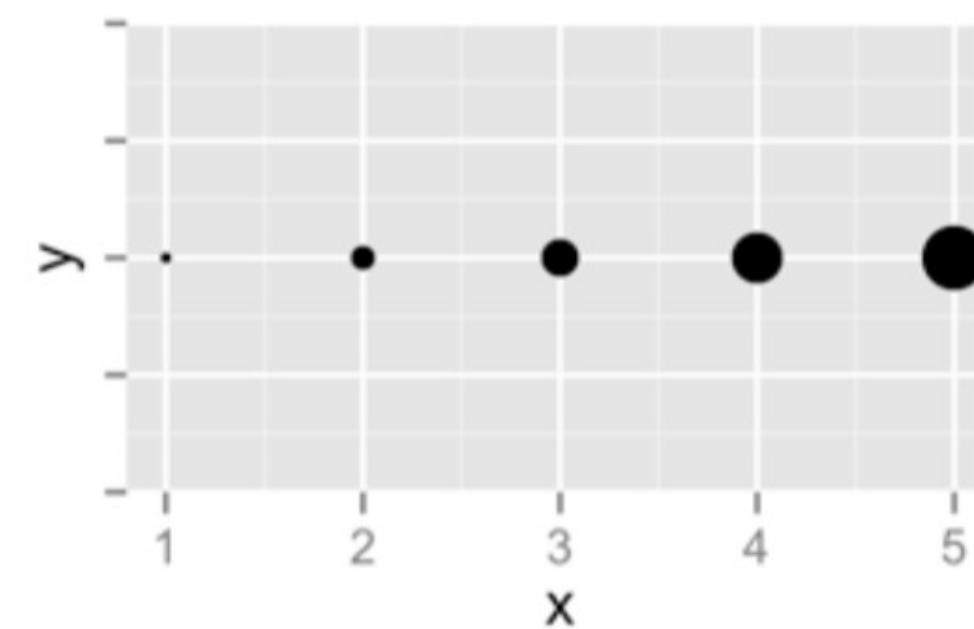
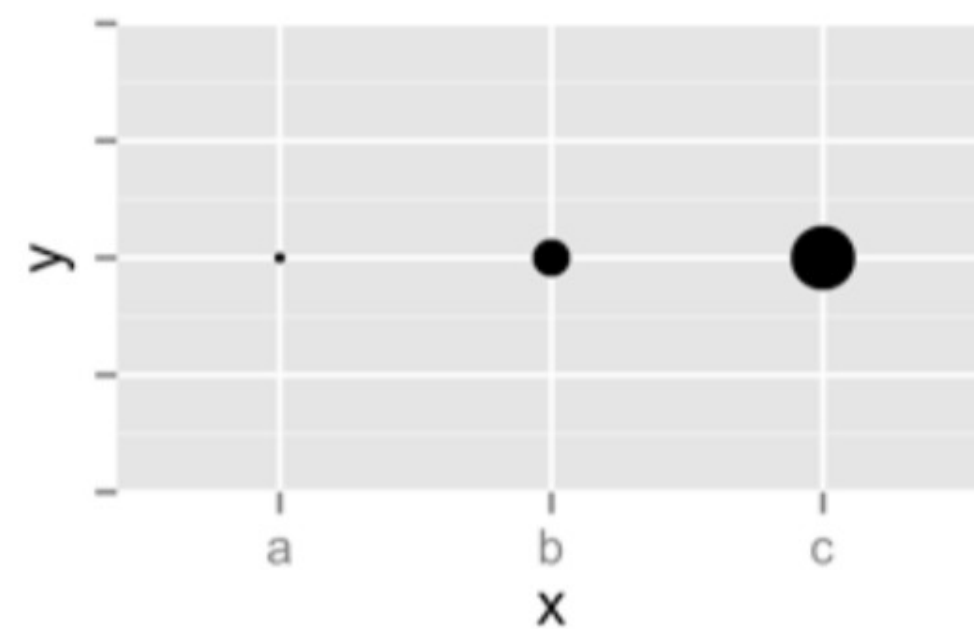
Discrete

Continuous

Color



Size

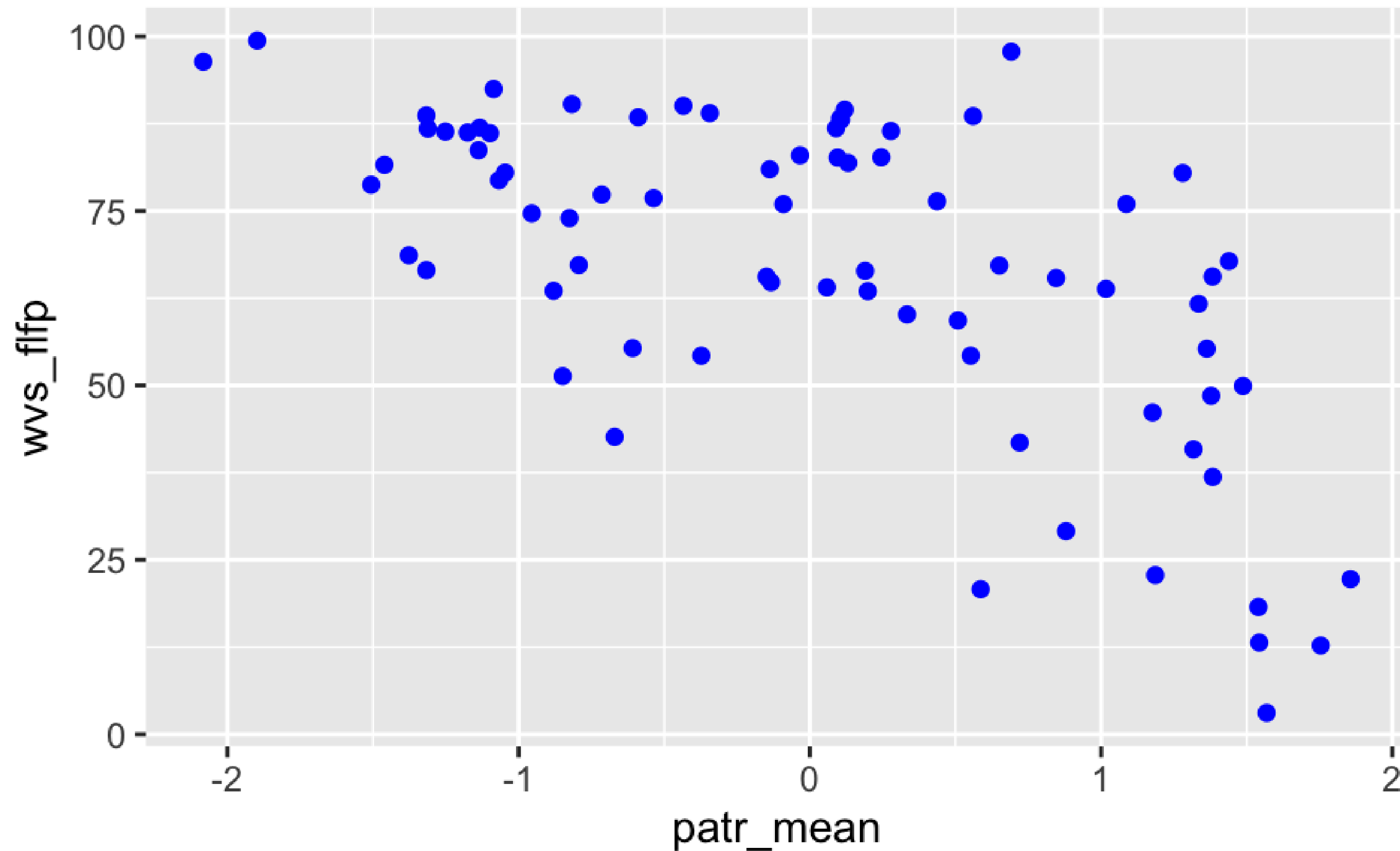


Shape

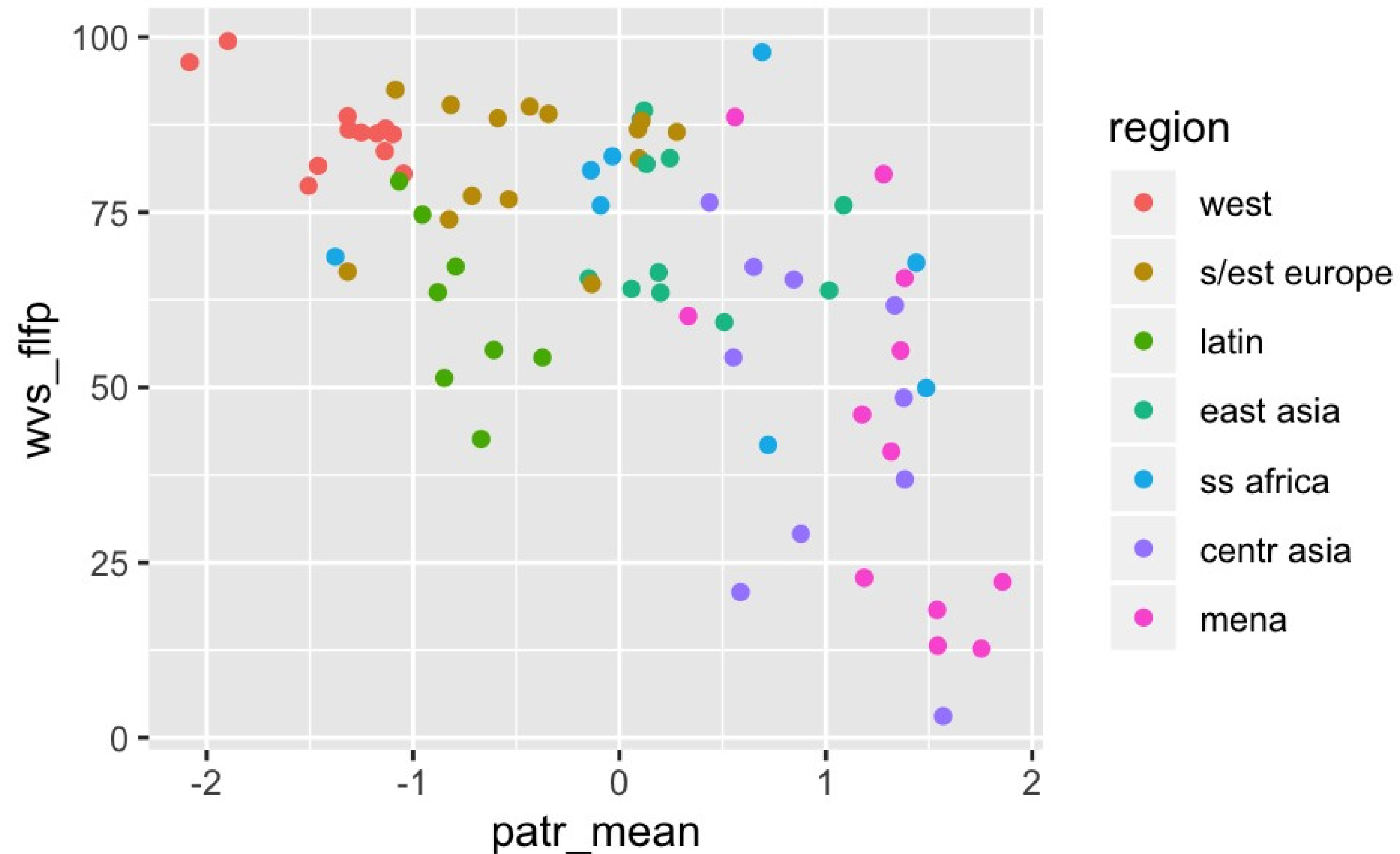


set vs. map

How do you make this plot?



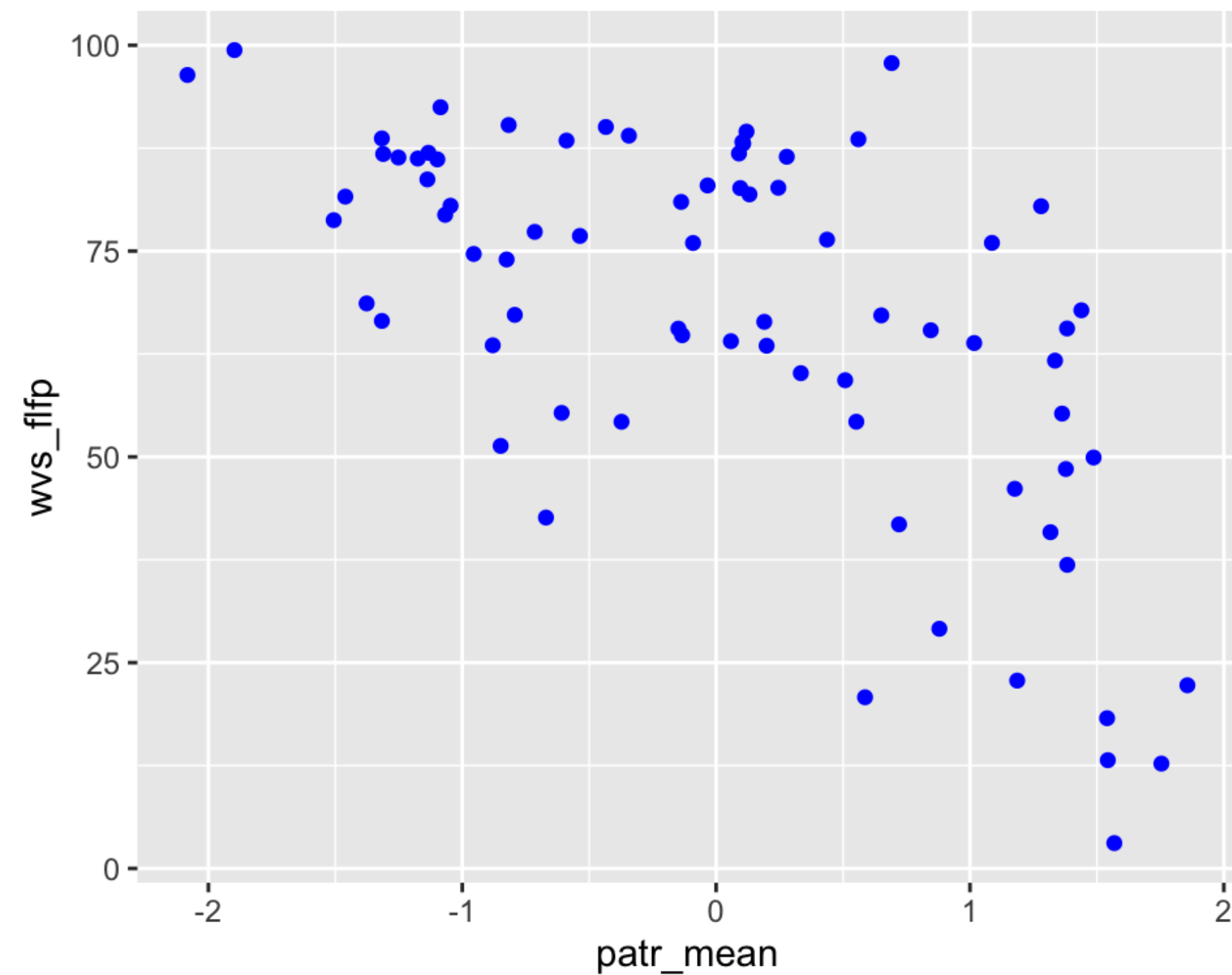
Map variable



**Inside of aes():
maps an
aesthetic to a
variable**

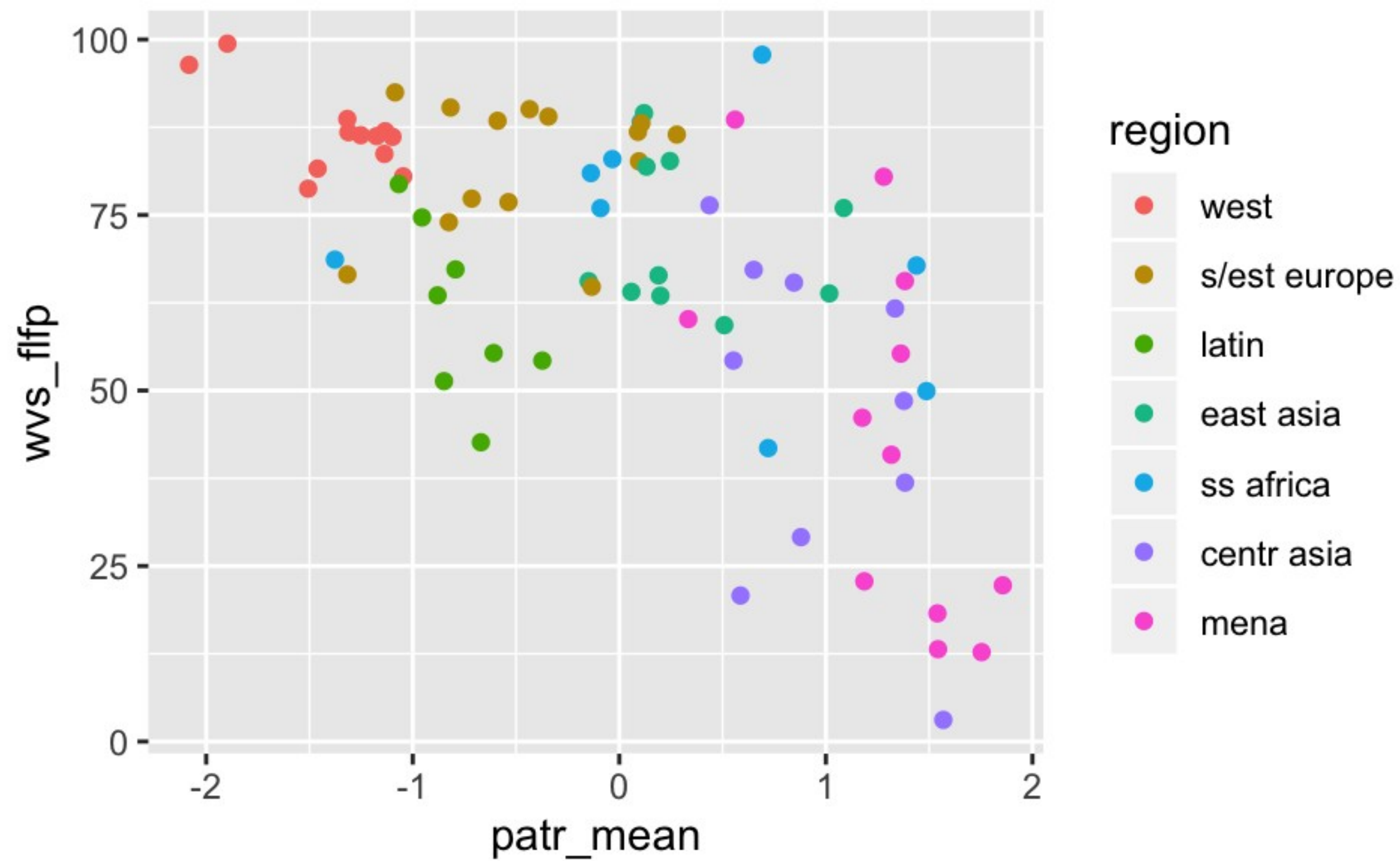
```
ggplot(flfp_agg) +  
  geom_point(aes(x = patr_mean, y = wvs_flfp, color = region))
```

Set values

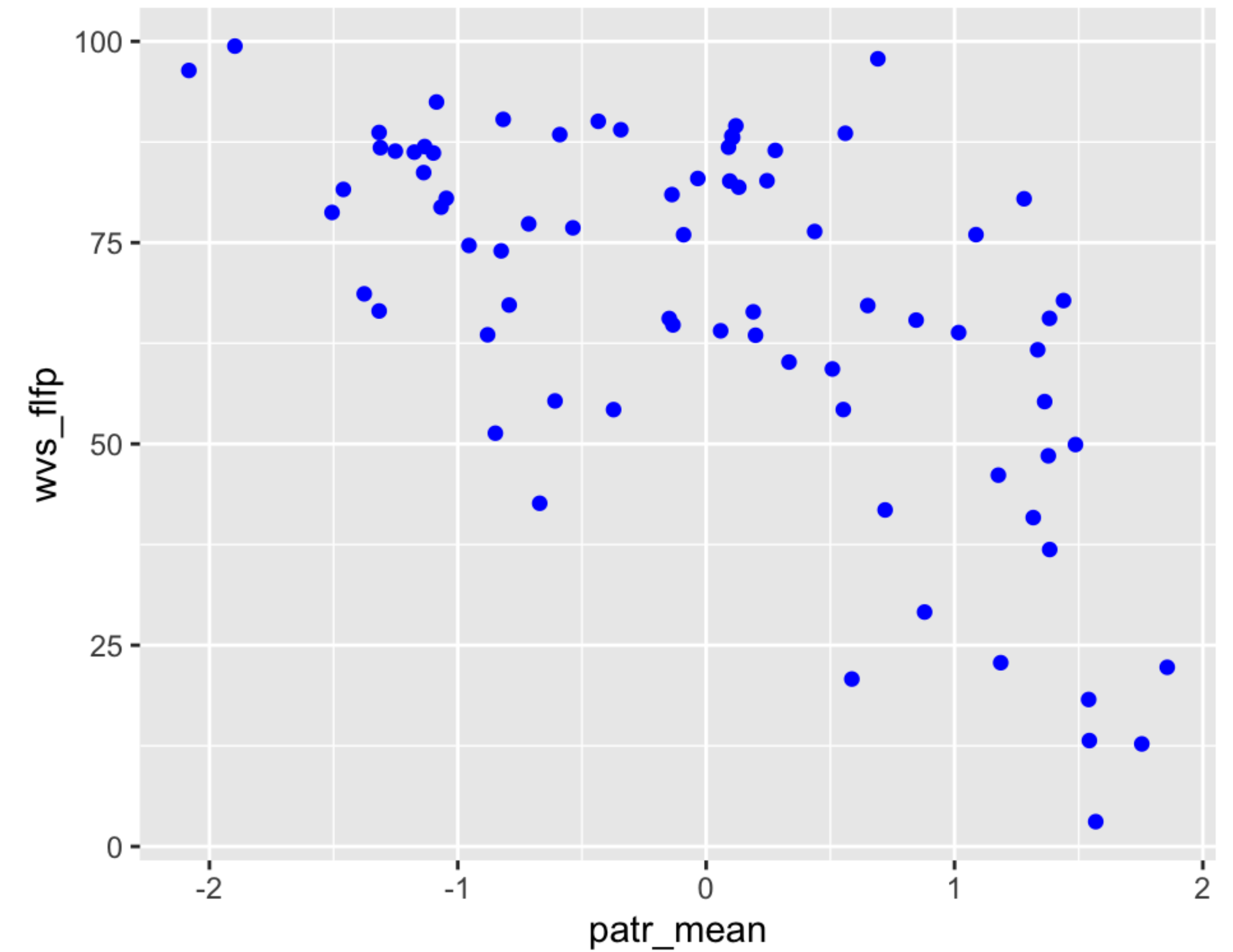


**Outside of aes():
sets an aesthetic to a value**

```
ggplot(flfp_agg) +  
  geom_point(mapping = aes(x = patr_mean, y = wvs_flfp),  
             colour = "blue")
```



```
ggplot(flfp_agg) +
  geom_point(aes(x = patr_mean,
                 y = wvs_flfp,
                 colour = region))
```

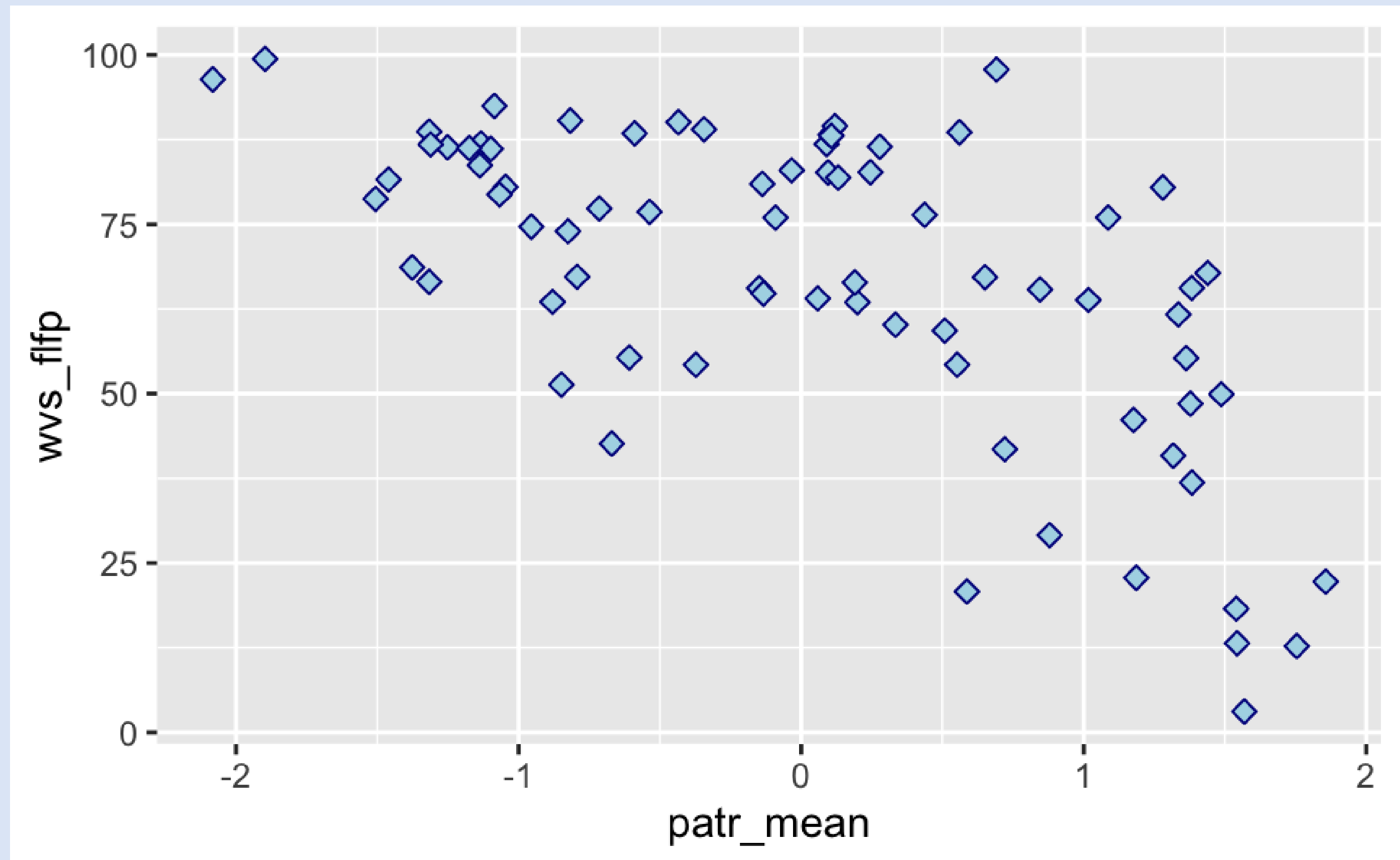


```
ggplot(flfp_agg) +
  geom_point(aes(x = patr_mean,
                 y = wvs_flfp),
            colour = "blue")
```

Your turn

Reproduce the plot by setting:

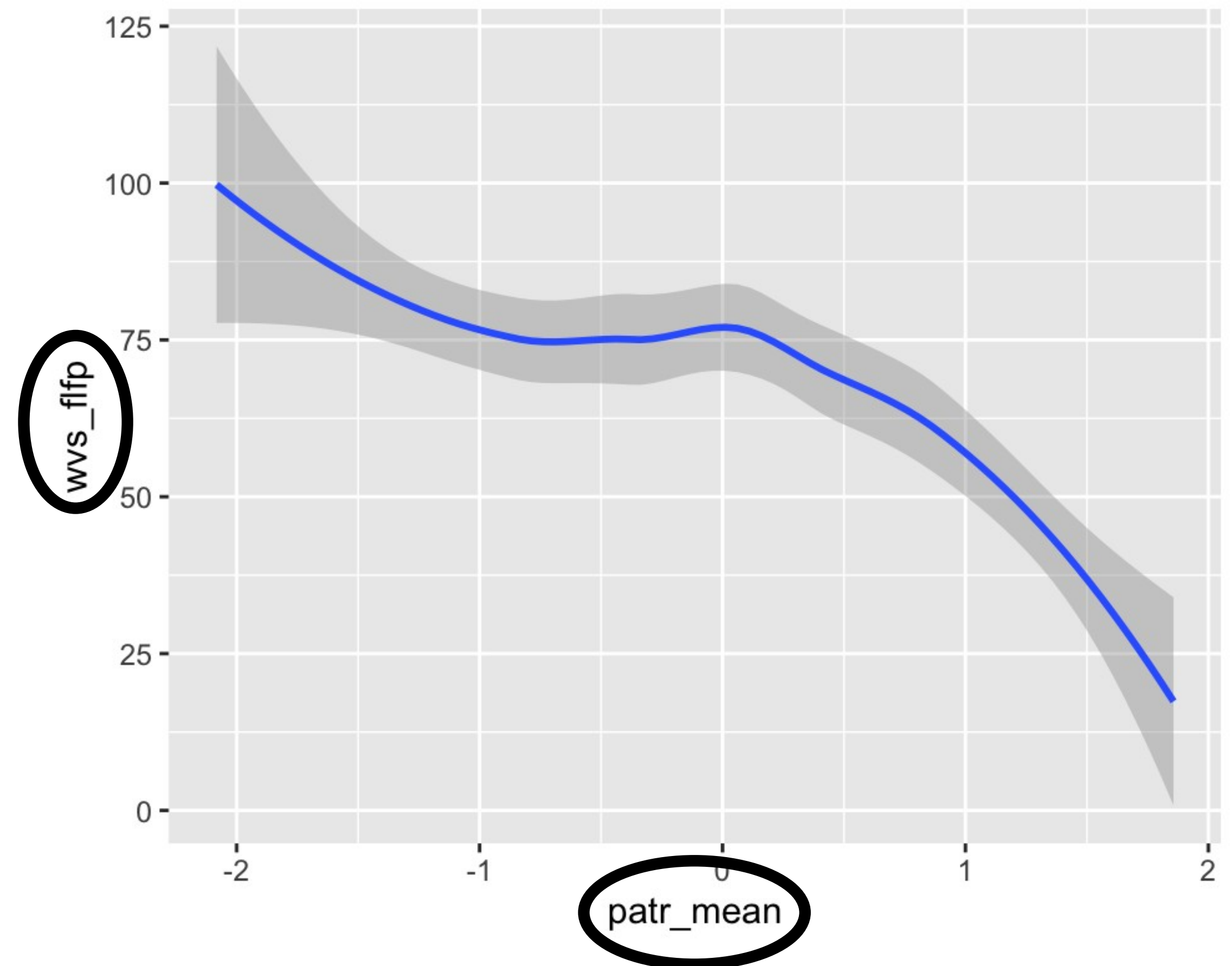
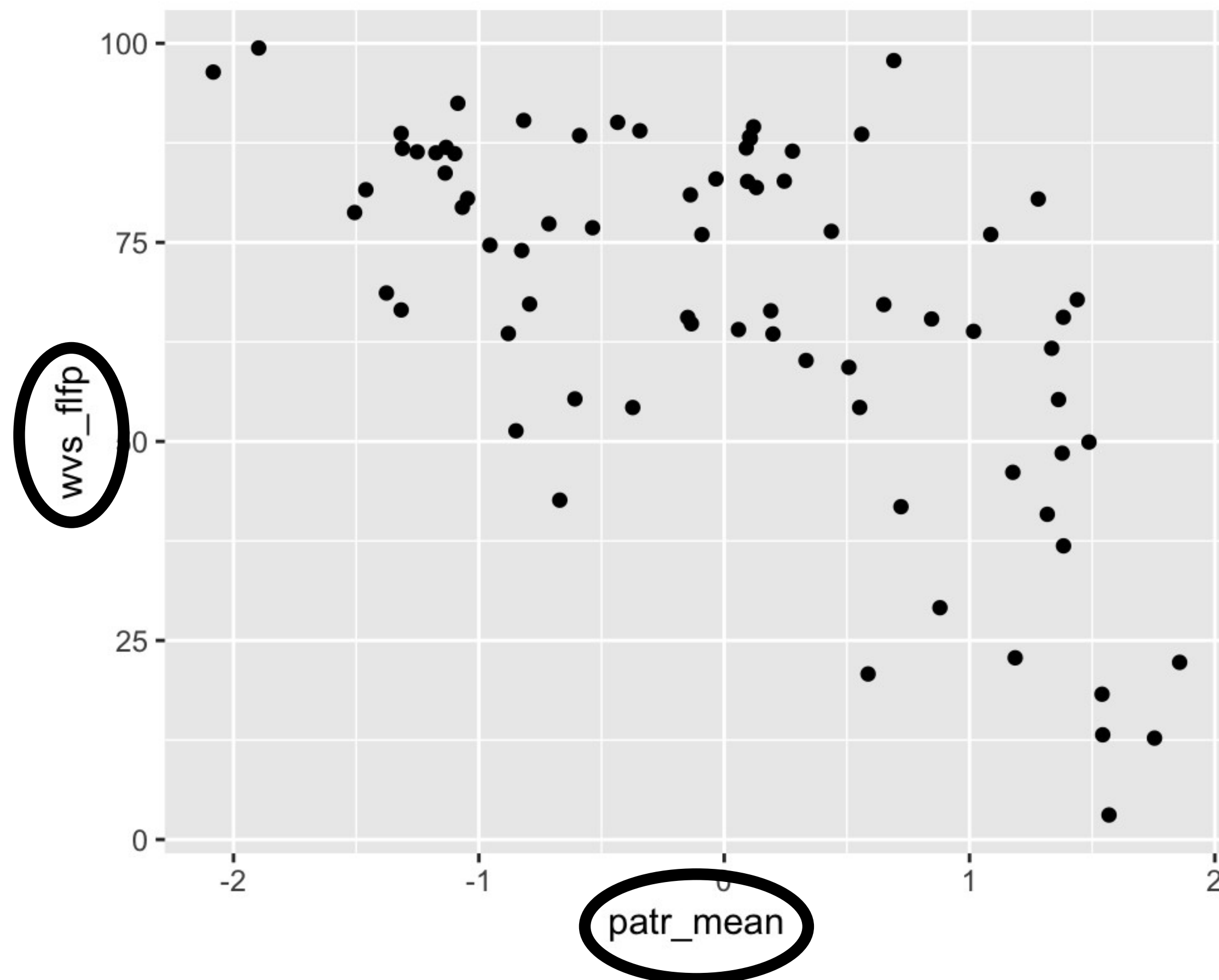
- colour = "darkblue"
- shape = 23
- fill = "lightblue"
- size = 3



Geoms

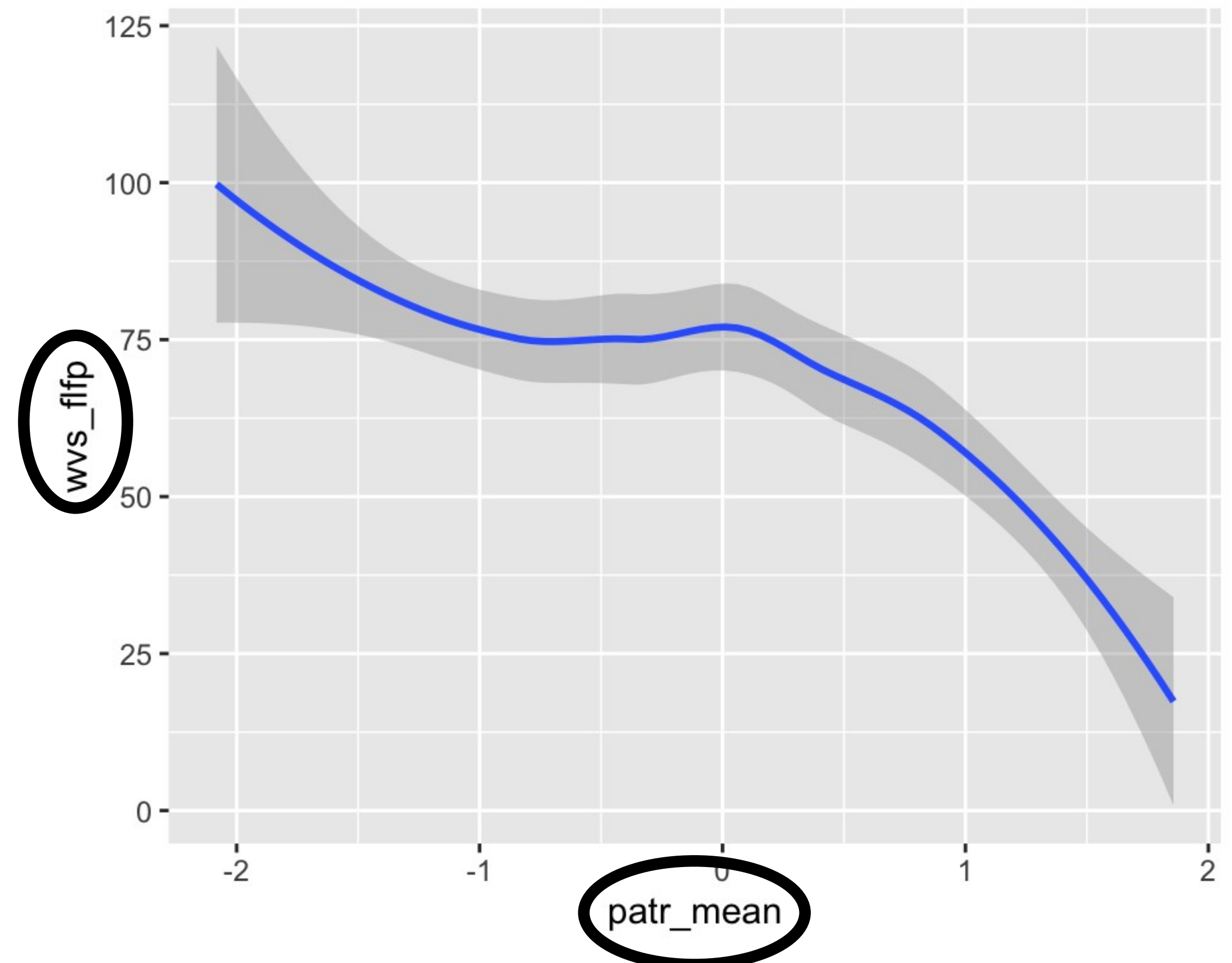
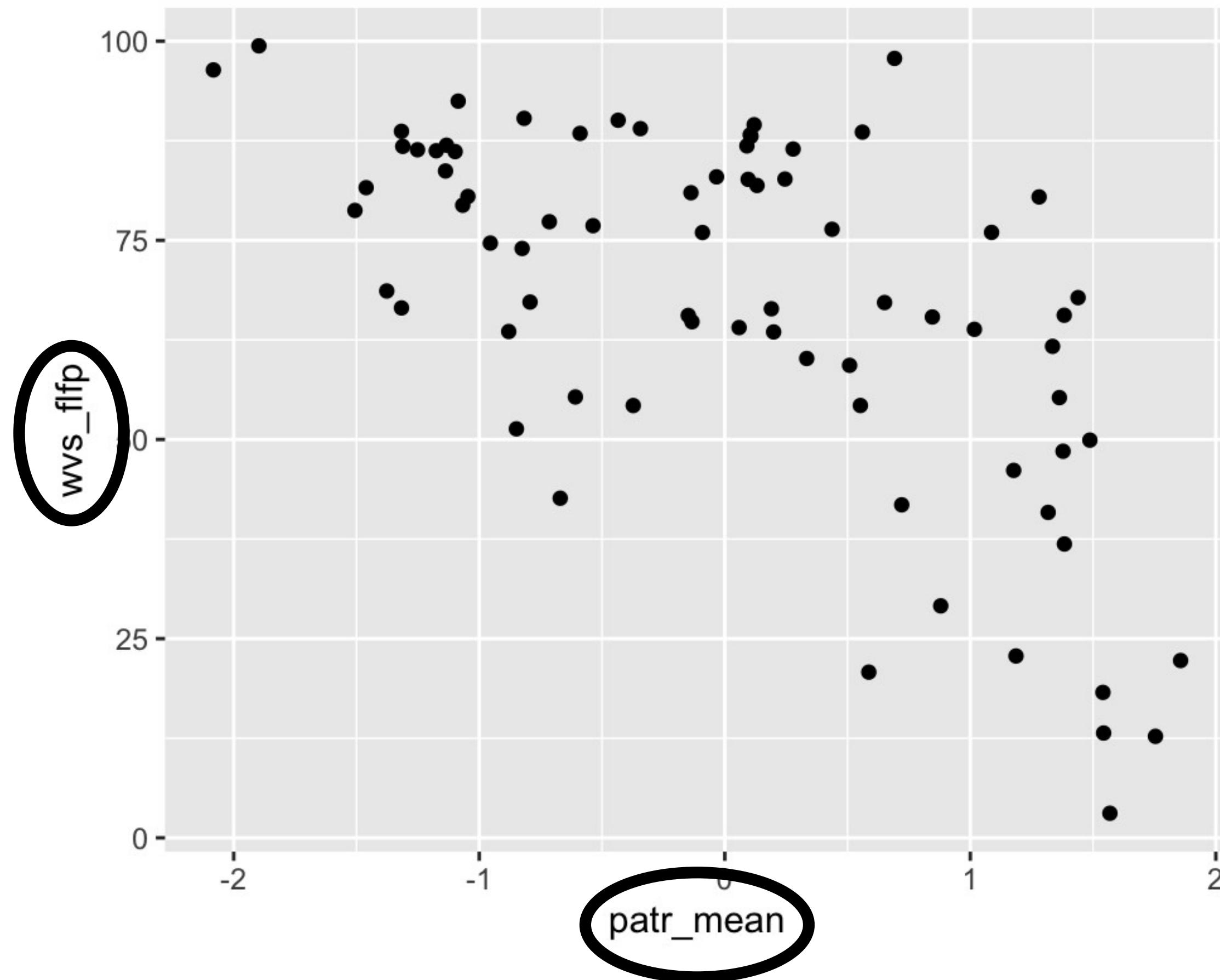
How are these
plots similar?

Same: x var, y var,
data



How are these
plots different?

Different: geometric
shape/object
(geom function)

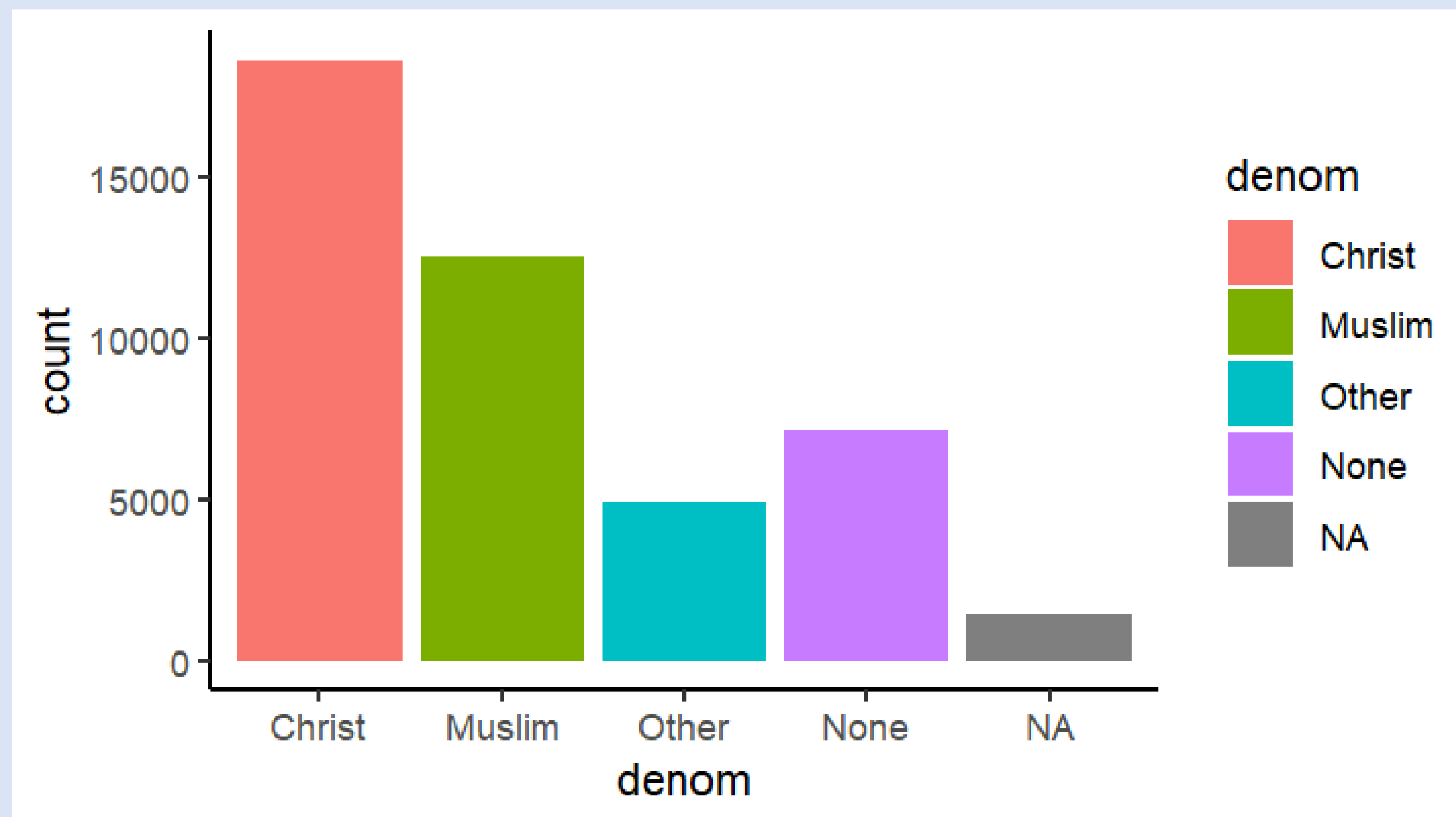


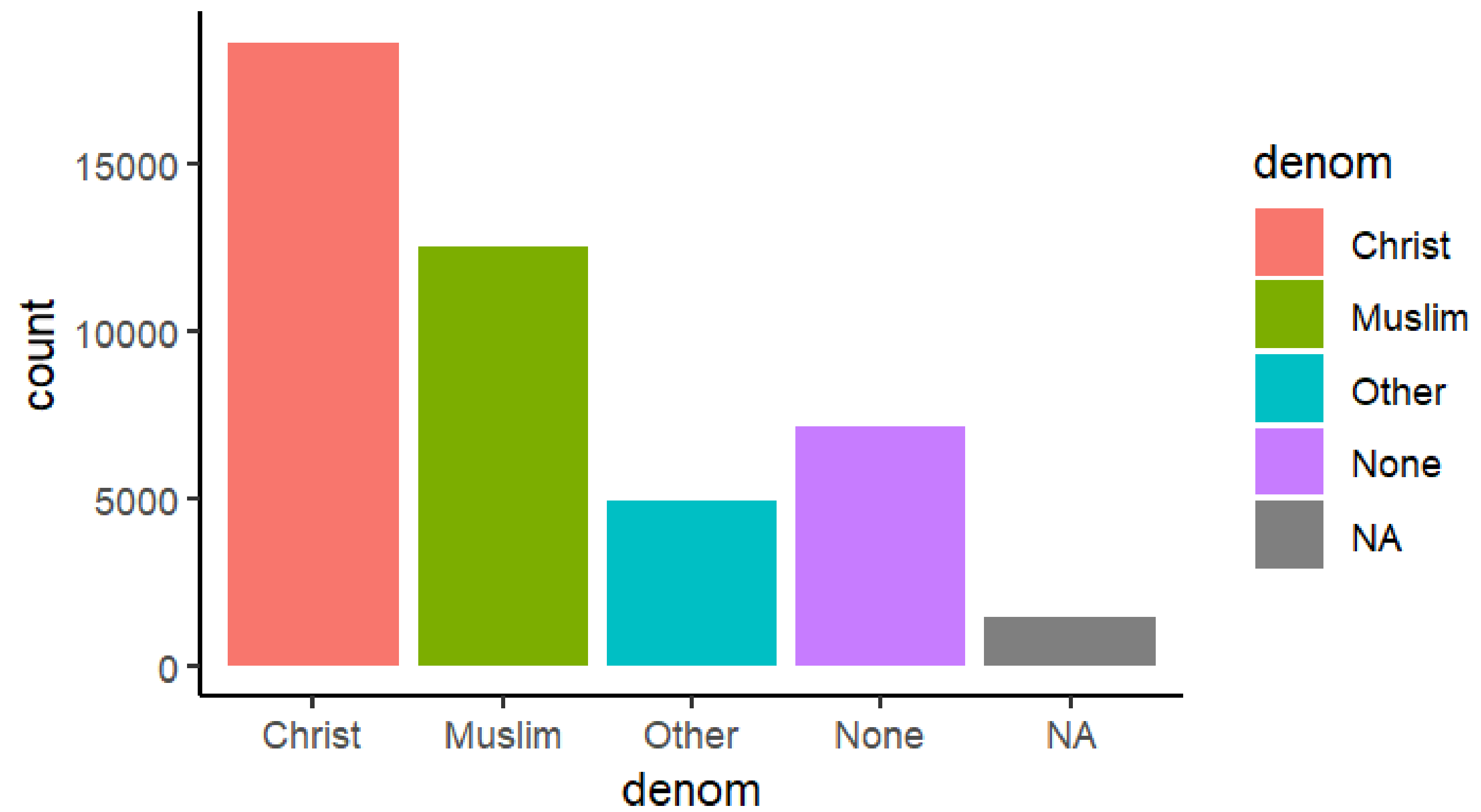
geoms

```
ggplot(data = <DATA>) +  
  <GEOM_FUNCTION>(mapping = aes(<MAPPINGS>))
```

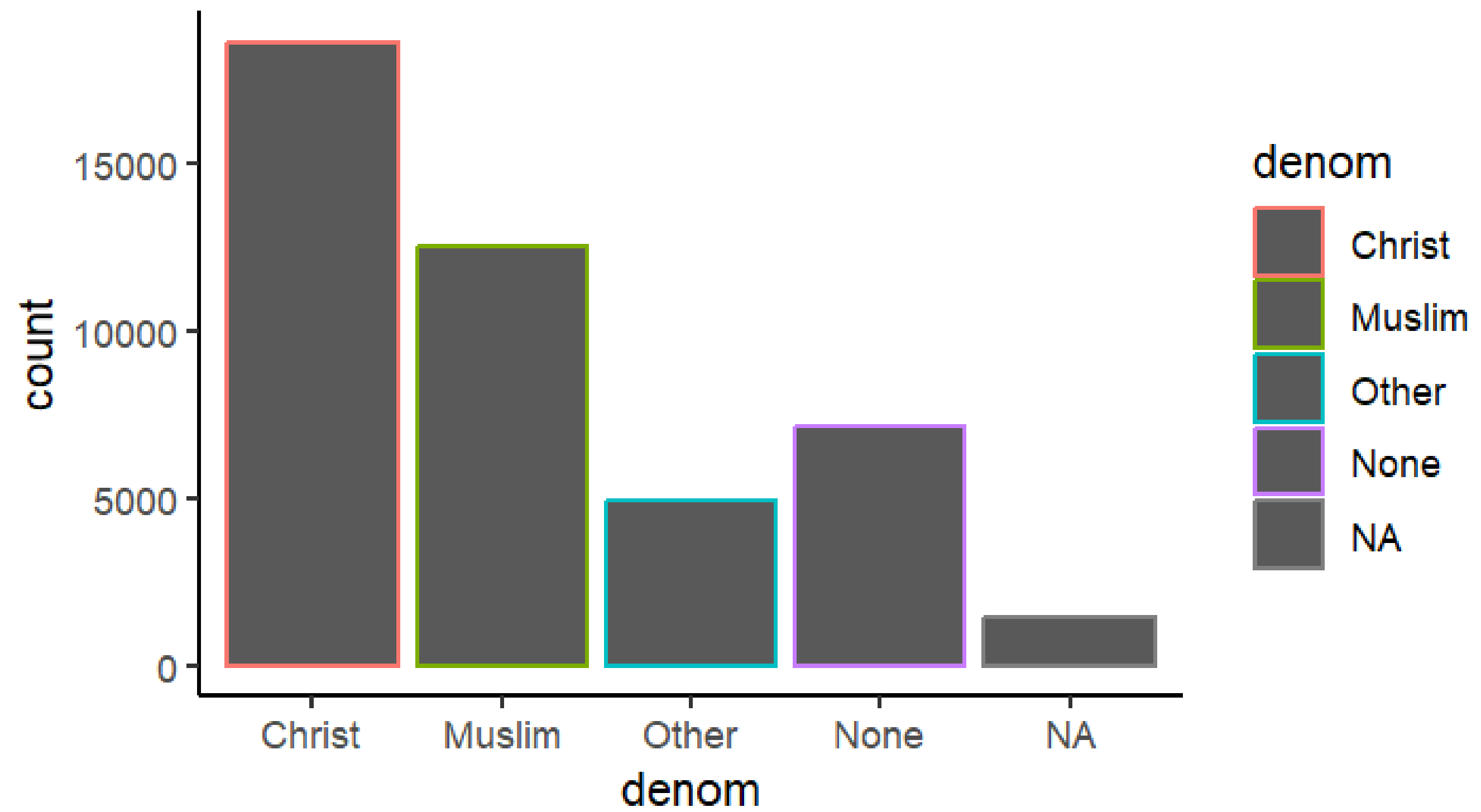
Your turn

- Make the bar-plot of **denom** from the flfp df which you made earlier today, coloured by denom, like below.





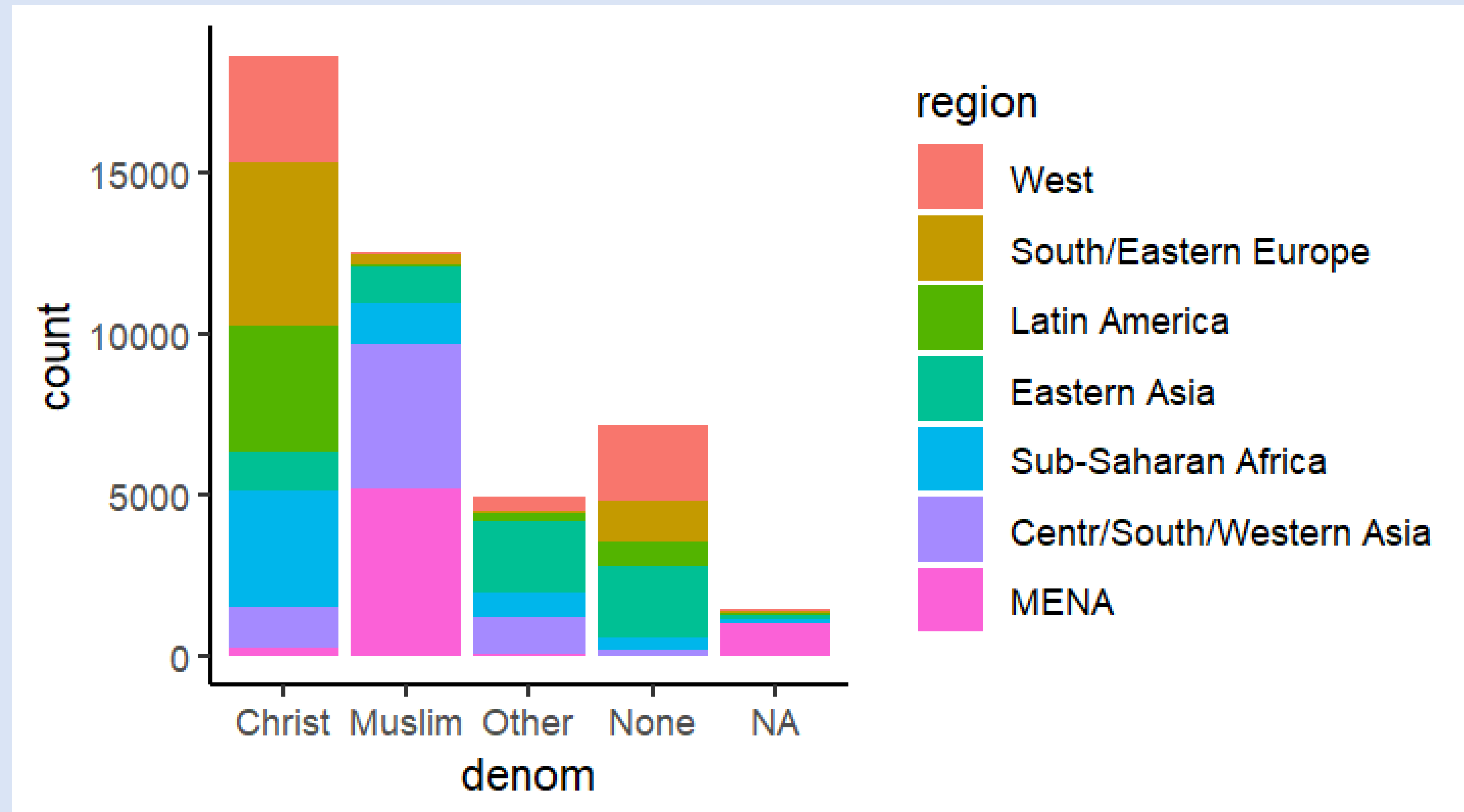
```
ggplot(flfp, aes(denom)) +  
  geom_bar(aes(fill=denom)) +  
  theme_classic()
```

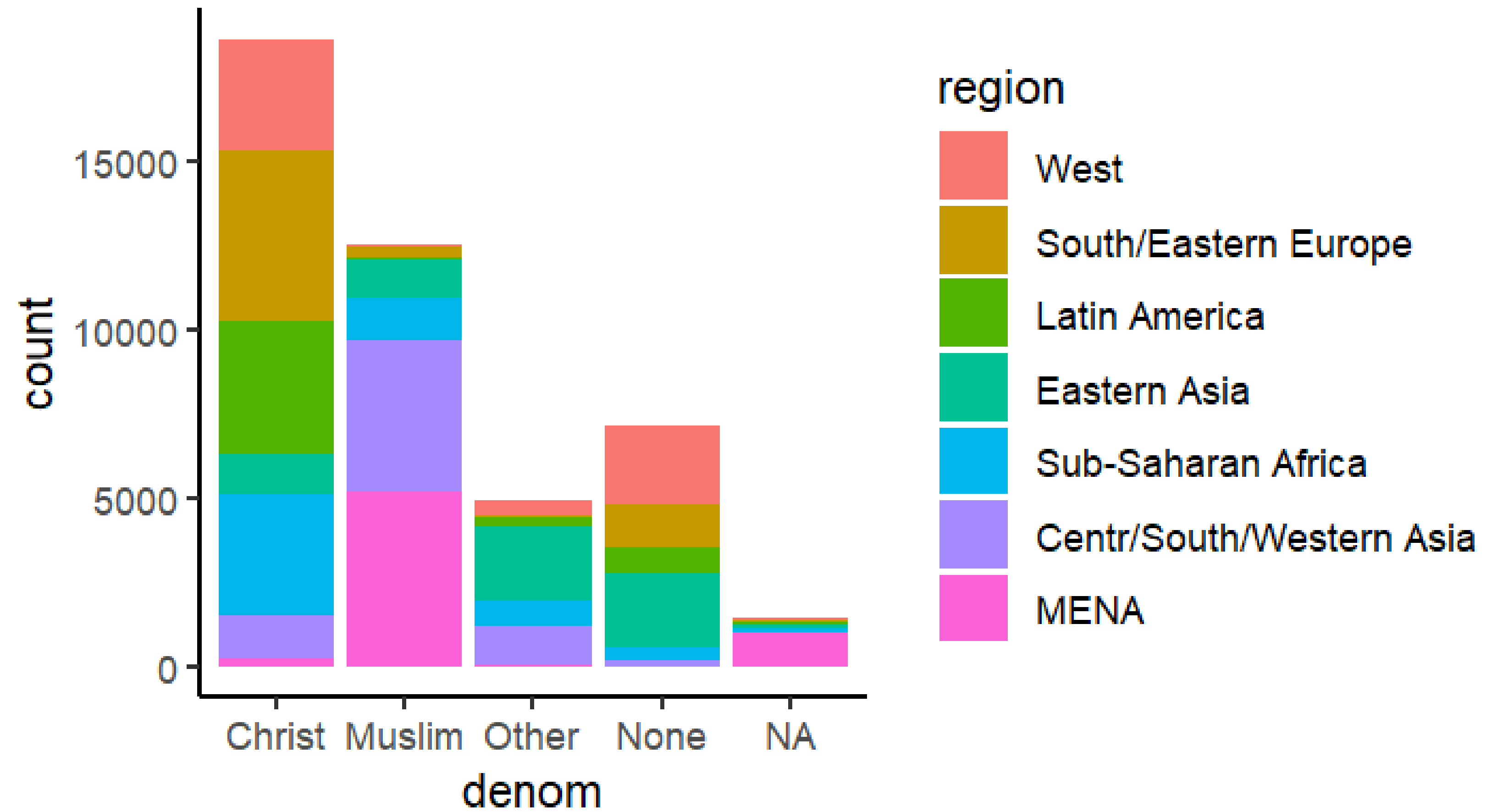


```
ggplot(flfp, aes(denom)) +  
  geom_bar(aes(color=denom)) +  
  theme_classic()
```

Your turn

- How to get this graph?



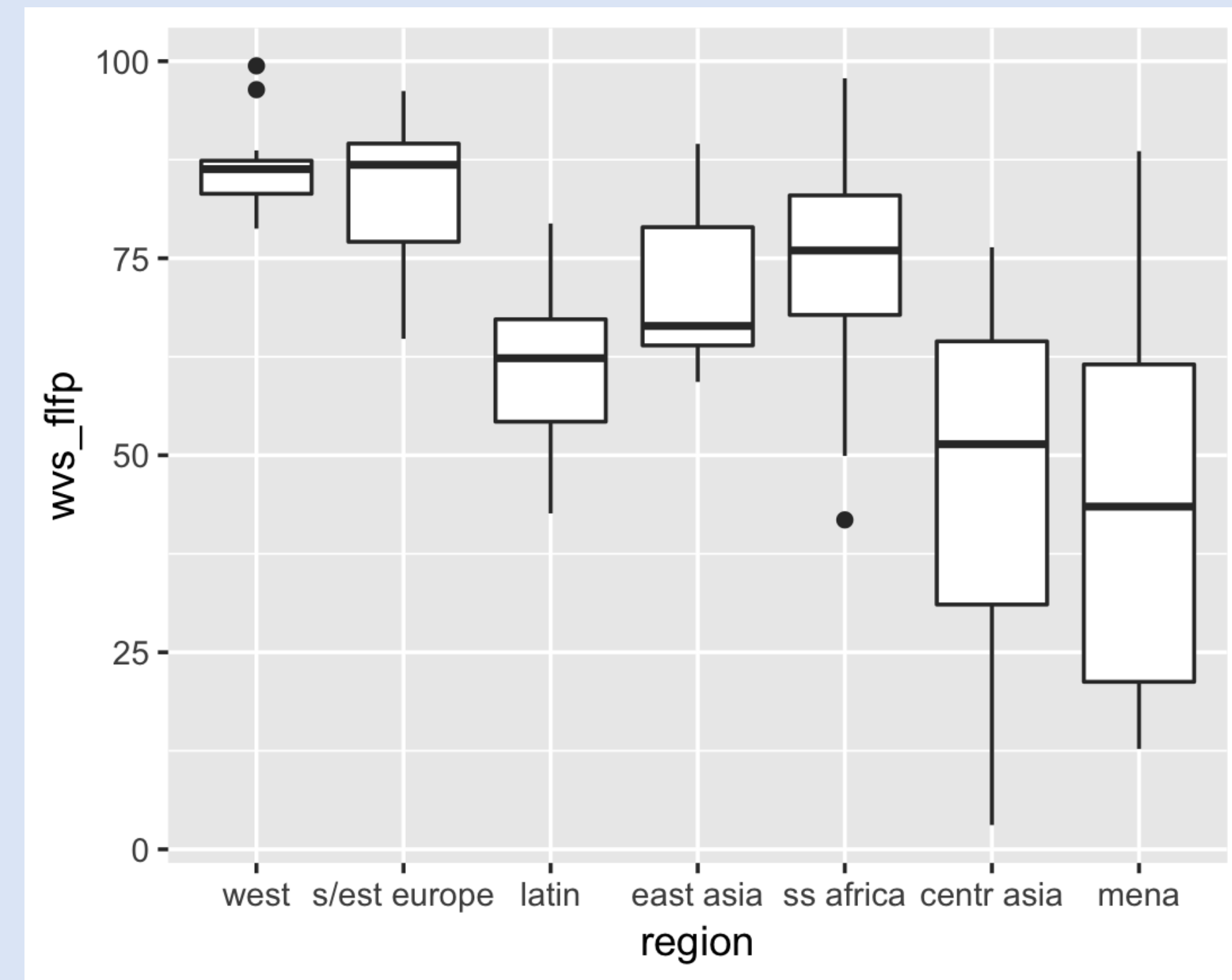
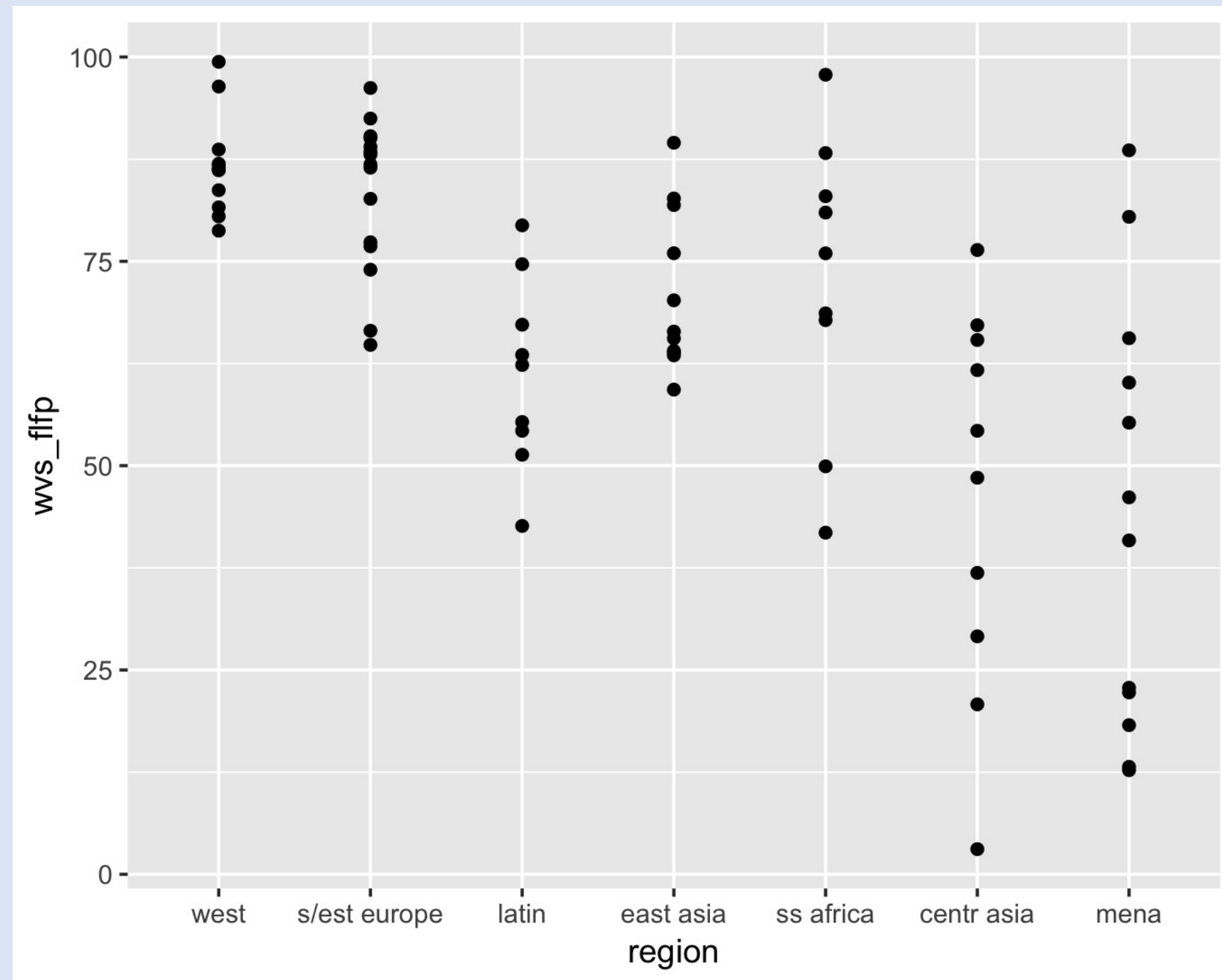


```
ggplot(flfp, aes(x=denom)) +  
  geom_bar(aes(fill=region)) +  
  theme_classic()
```

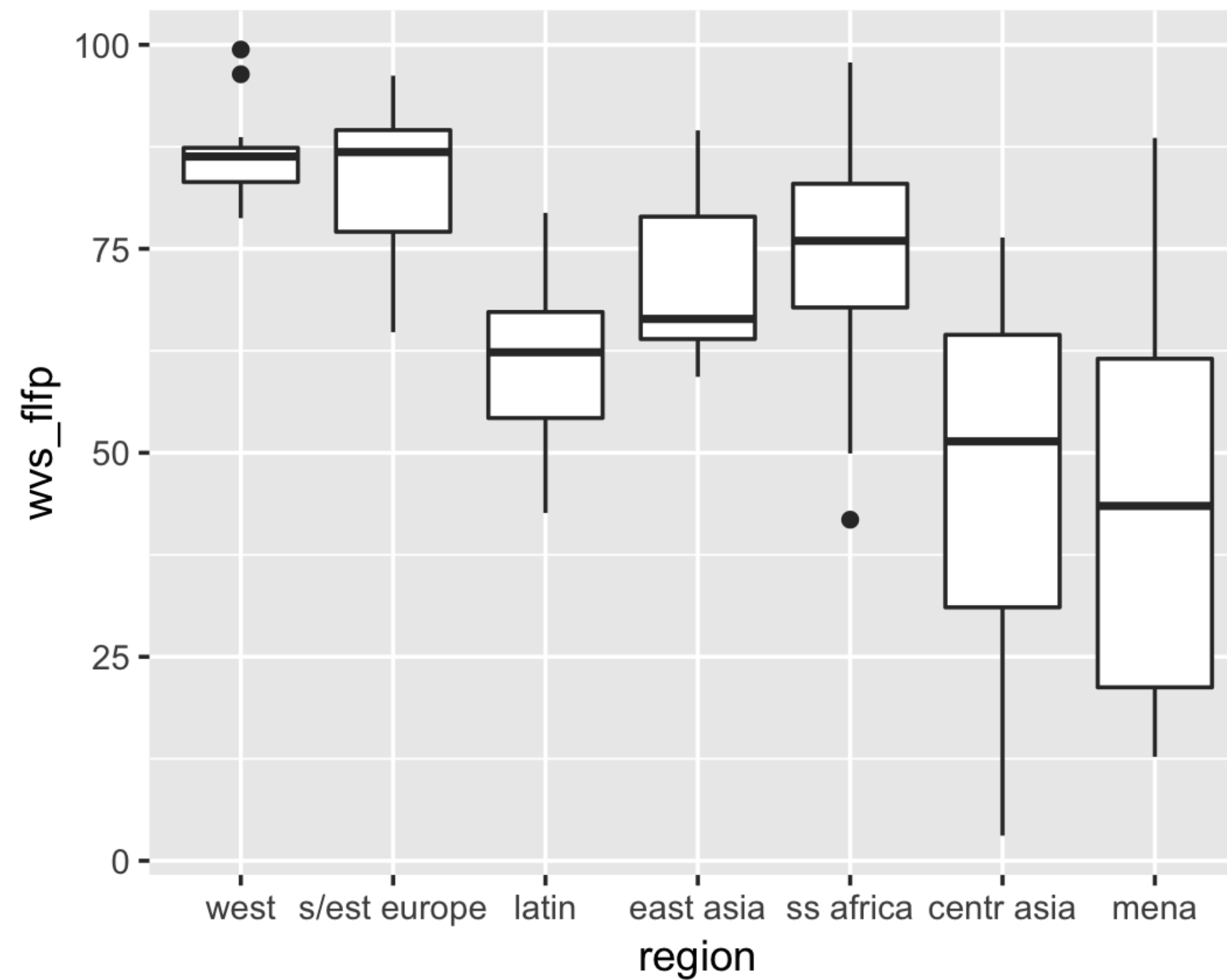
boxplots

Your turn

- Use the cheatsheet/ google and decide how to replace the scatterplot on the left with a box-plot



```
ggplot(flfp_agg) +  
  geom_point(aes(x = region, y = wvs_flfp)) +  
  theme_bw()
```



```
ggplot(flfp_agg) +  
geom_boxplot(aes(x = region, y = wvs_flfp)) +  
theme_bw()
```

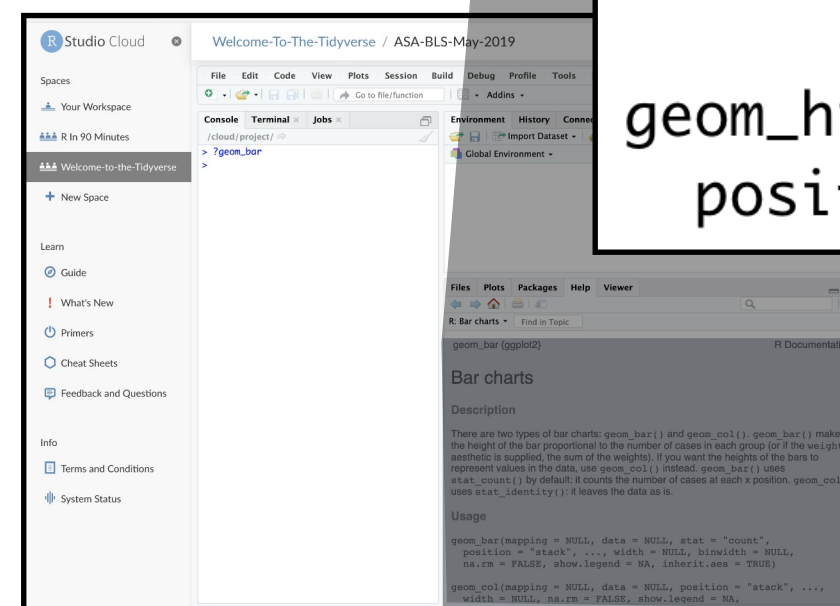
"Help" pages

To open the documentation for a function, type

`?geom_histogram`

?

function name (no parentheses)



`geom_freqpoly {ggplot2}`

R Documentation

Histograms and frequency polygons

Description

Visualise the distribution of a single continuous variable by dividing the x axis into bins and counting the number of observations in each bin. Histograms (`geom_histogram()`) display the counts with bars; frequency polygons (`geom_freqpoly()`) display the counts with lines. Frequency polygons are more suitable when you want to compare the distribution across the levels of a categorical variable.

Usage

```
geom_freqpoly(mapping = NULL, data = NULL, stat = "bin",  
              position = "identity", ..., na.rm = FALSE, show.legend = NA,  
              inherit.aes = TRUE)
```

```
geom_histogram(mapping = NULL, data = NULL, stat = "bin",  
               position = "stack", ..., binwidth = NULL, bins = NULL,
```

Tips

- **scan** page for relevant info, especially the **arguments**
- **ignore** things that don't make sense
- **try** out the **examples**

geom_freqpoly {ggplot2}

R Documentation

Histograms and frequency polygons

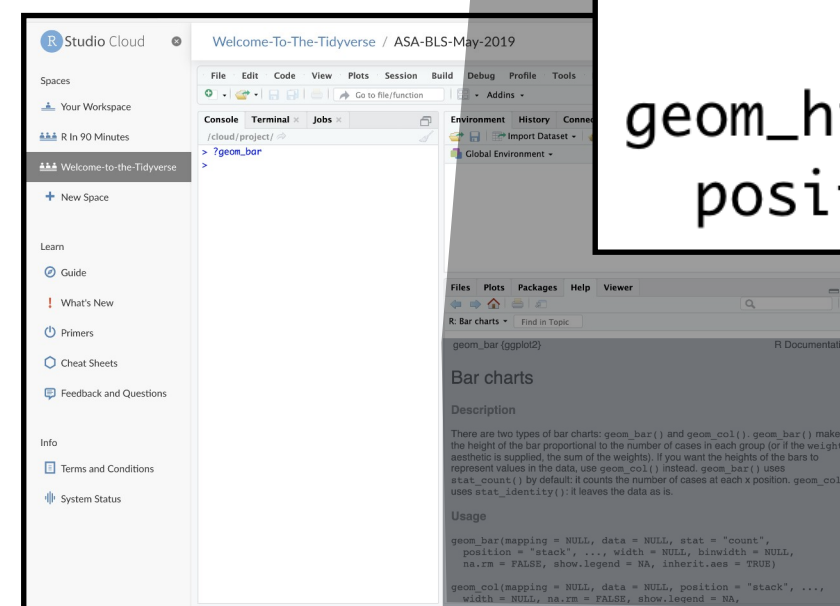
Description

Visualise the distribution of a single continuous variable by dividing the x axis into bins and counting the number of observations in each bin. Histograms (`geom_histogram()`) display the counts with bars; frequency polygons (`geom_freqpoly()`) display the counts with lines. Frequency polygons are more suitable when you want to compare the distribution across the levels of a categorical variable.

Usage

```
geom_freqpoly(mapping = NULL, data = NULL, stat = "bin",  
              position = "identity", ..., na.rm = FALSE, show.legend = NA,  
              inherit.aes = TRUE)
```

```
geom_histogram(mapping = NULL, data = NULL, stat = "bin",  
               position = "stack", ..., binwidth = NULL, bins = NULL,
```

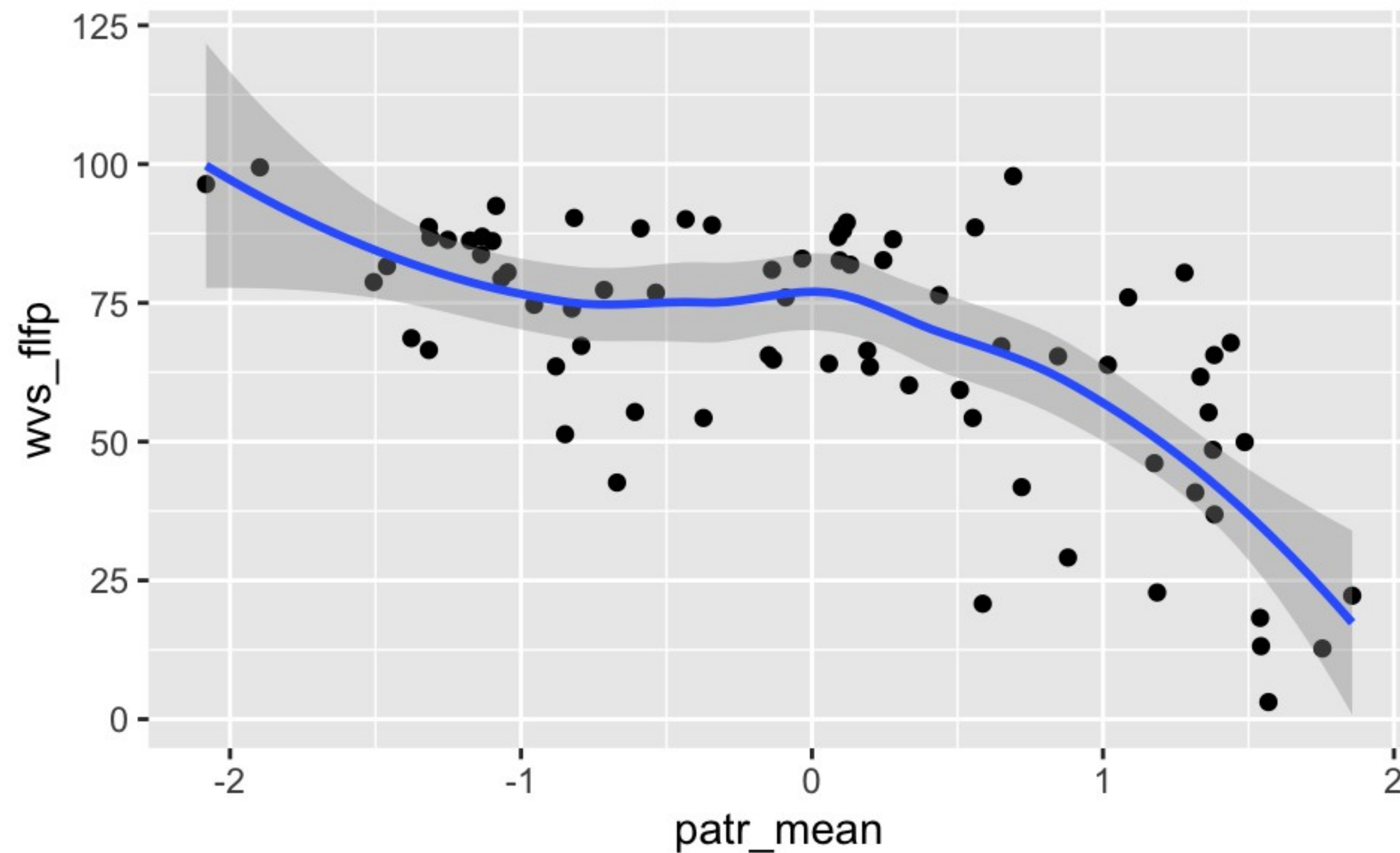


Your turn

- Try to predict what this code will do. Then run it.

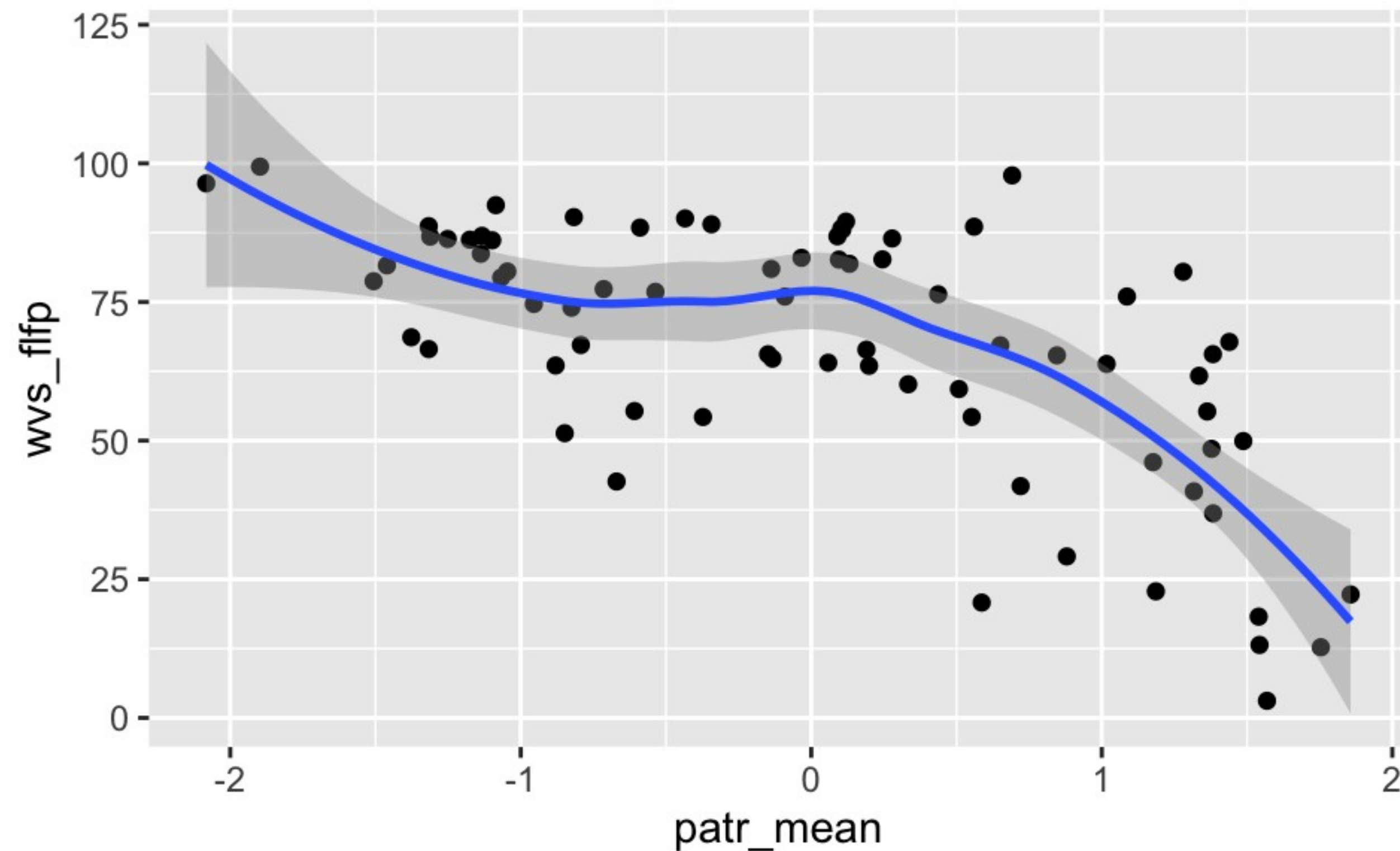
```
ggplot(flfp_agg) +  
  geom_point(aes(x = patr_mean, y = wvs_flfp)) +  
  geom_smooth(aes(x = patr_mean, y = wvs_flfp)) +  
  theme_bw()
```

Multiple geoms on the same plot



```
ggplot(flfp_agg) +  
  geom_point(aes(x = patr_mean, y = wvs_flfp)) +  
  geom_smooth(aes(x = patr_mean, y = wvs_flfp))
```

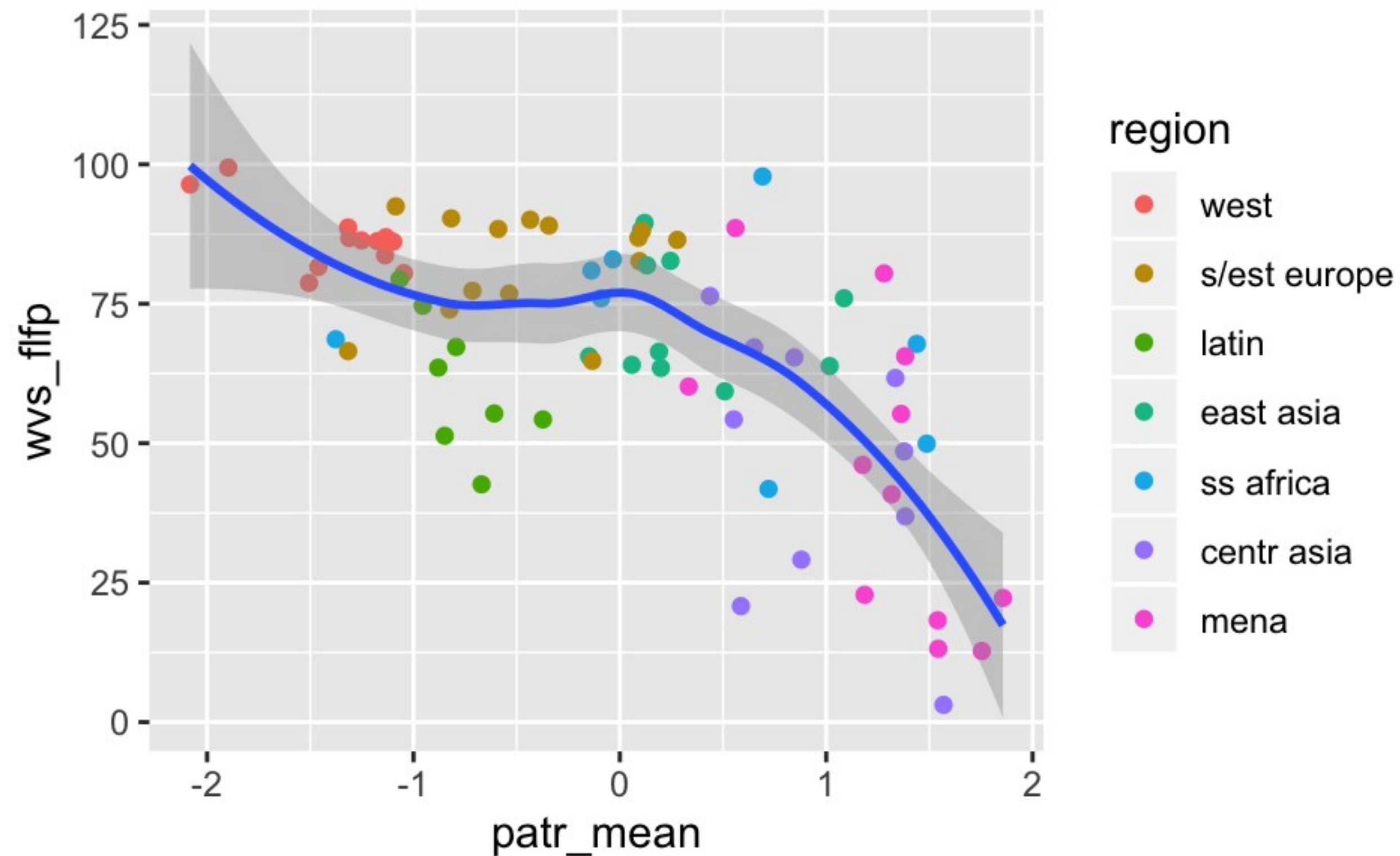
Global mapping



Mappings (and data) that appear in `ggplot()` will apply globally to every layer

```
ggplot(flfp, aes(x = patr_mean, y = wvs_flfp)) +  
  geom_point() +  
  geom_smooth()
```

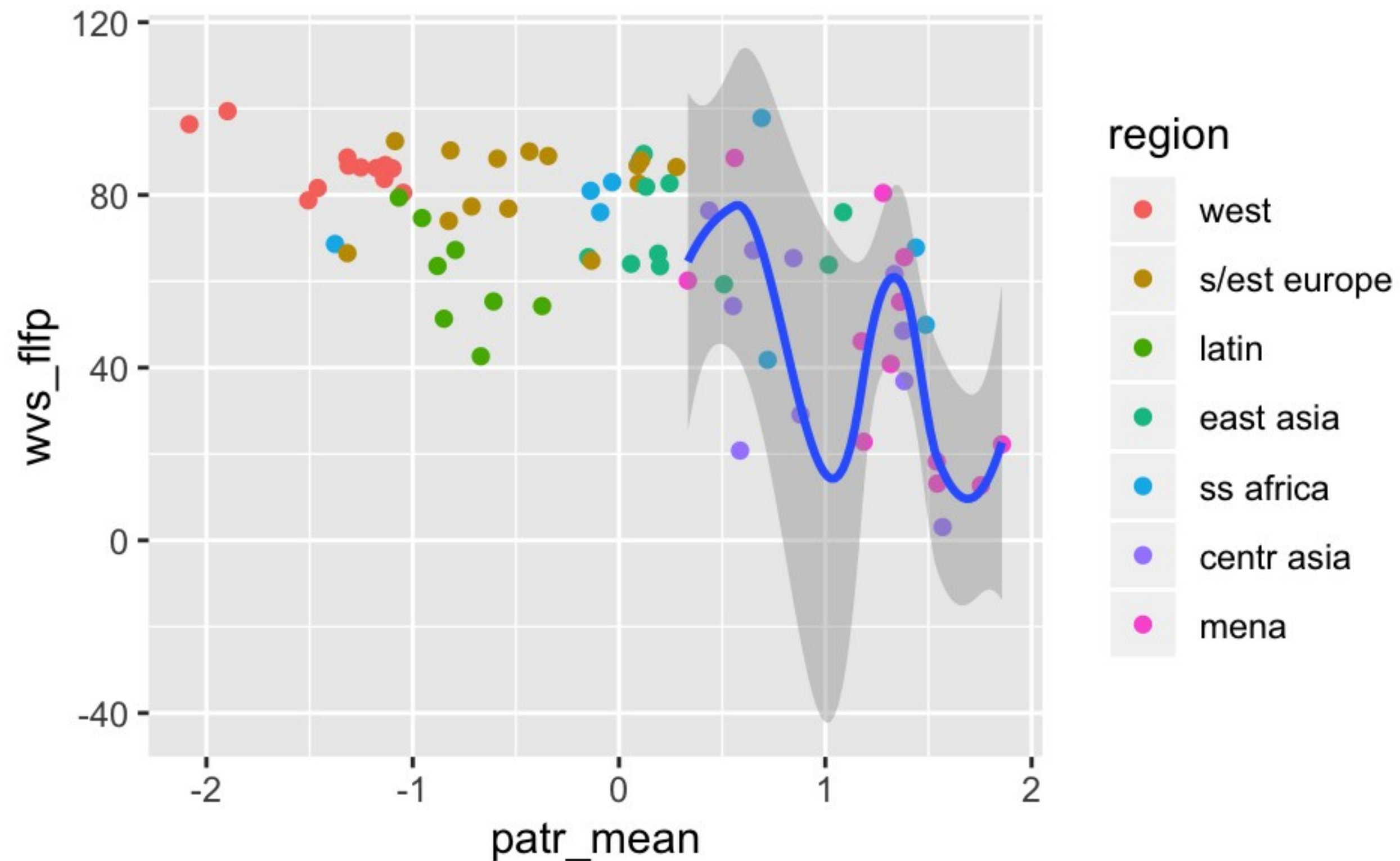
Local mapping



Mappings (and data) that appear in a `geom_function` will add to or override the global mappings for that layer only

```
ggplot(flfp_agg, aes(x = patr_mean, y = wvs_flfp)) +  
  geom_point(aes(colour = region)) +  
  geom_smooth() +  
  theme_bw()
```

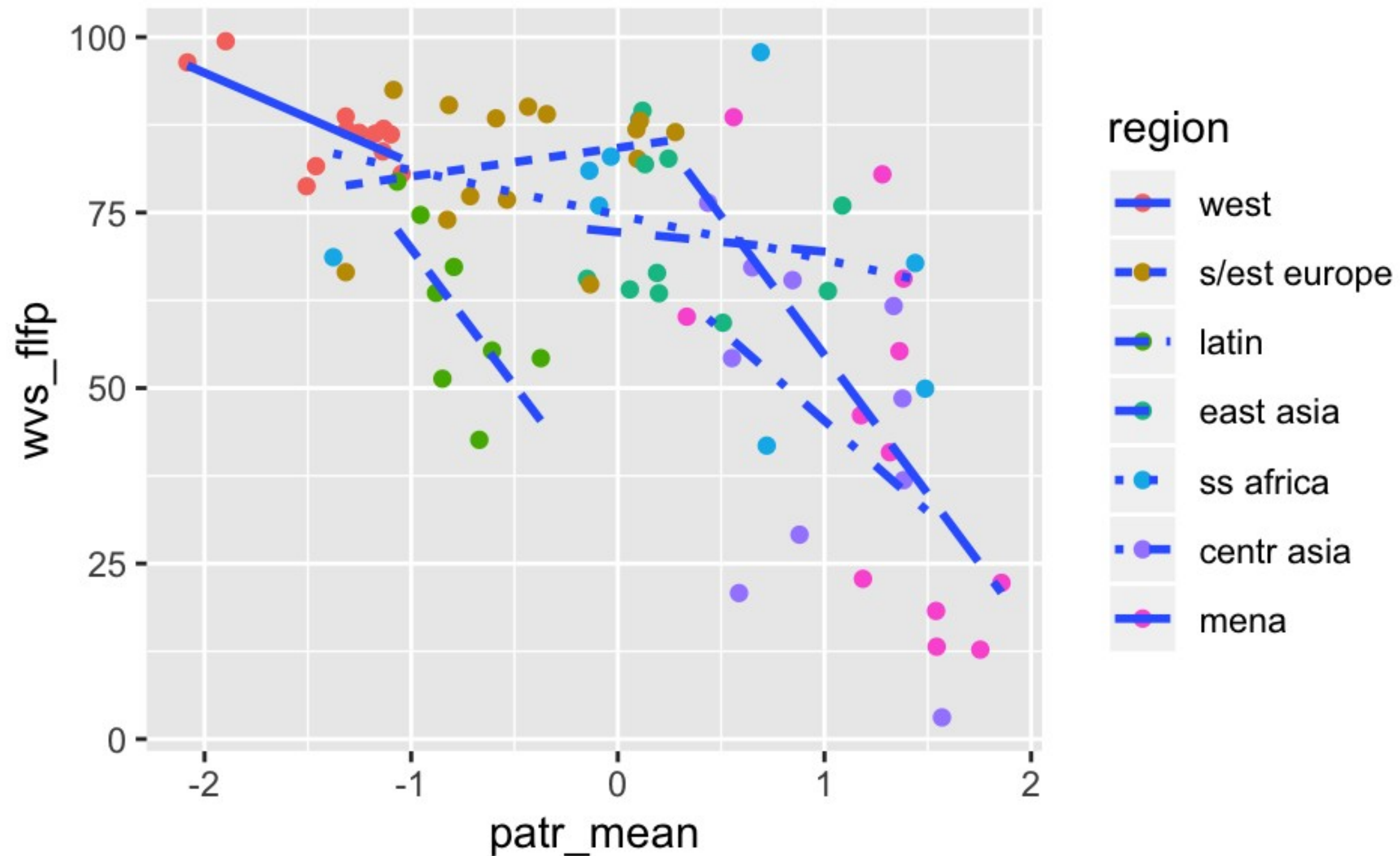
Local mapping



Data can also
be set locally
by geom

```
ggplot(flfp_agg, aes(x = patr_mean, y = wvs_flfp)) +  
  geom_point(aes(colour = region)) +  
  geom_smooth(data = filter(flfp_agg, region == "mena")) +  
  theme_bw()
```

Different geoms use different aesthetics



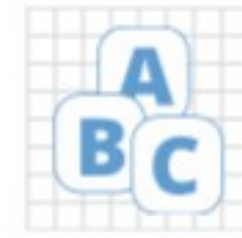
```
ggplot(flfp_agg, aes(x = patr_mean, y = wvs_flfp)) +  
  geom_point(aes(colour = region)) +  
  geom_smooth(aes(linetype = region), se = FALSE, method = "lm") +  
  theme_bw()
```

Different geoms use different aesthetics

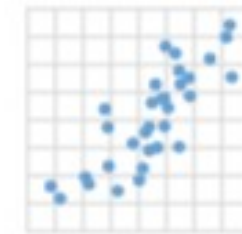
TWO VARIABLES

continuous x , continuous y

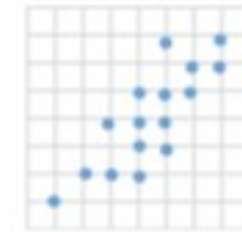
```
e <- ggplot(mpg, aes(cty, hwy))
```



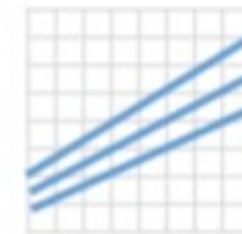
e + geom_label(aes(label = cty), nudge_x = 1, nudge_y = 1, check_overlap = TRUE) x, y, label, alpha, angle, color, family, fontface, hjust, lineheight, size, vjust



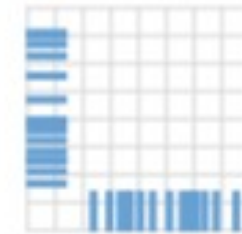
e + geom_jitter(height = 2, width = 2) x, y, alpha, color, fill, shape, size



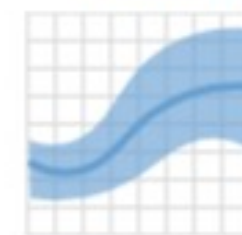
e + geom_point(), x, y, alpha, color, fill, shape, size, stroke



e + geom_quantile(), x, y, alpha, color, group, linetype, size, weight



e + geom_rug(sides = "bl"), x, y, alpha, color, linetype, size



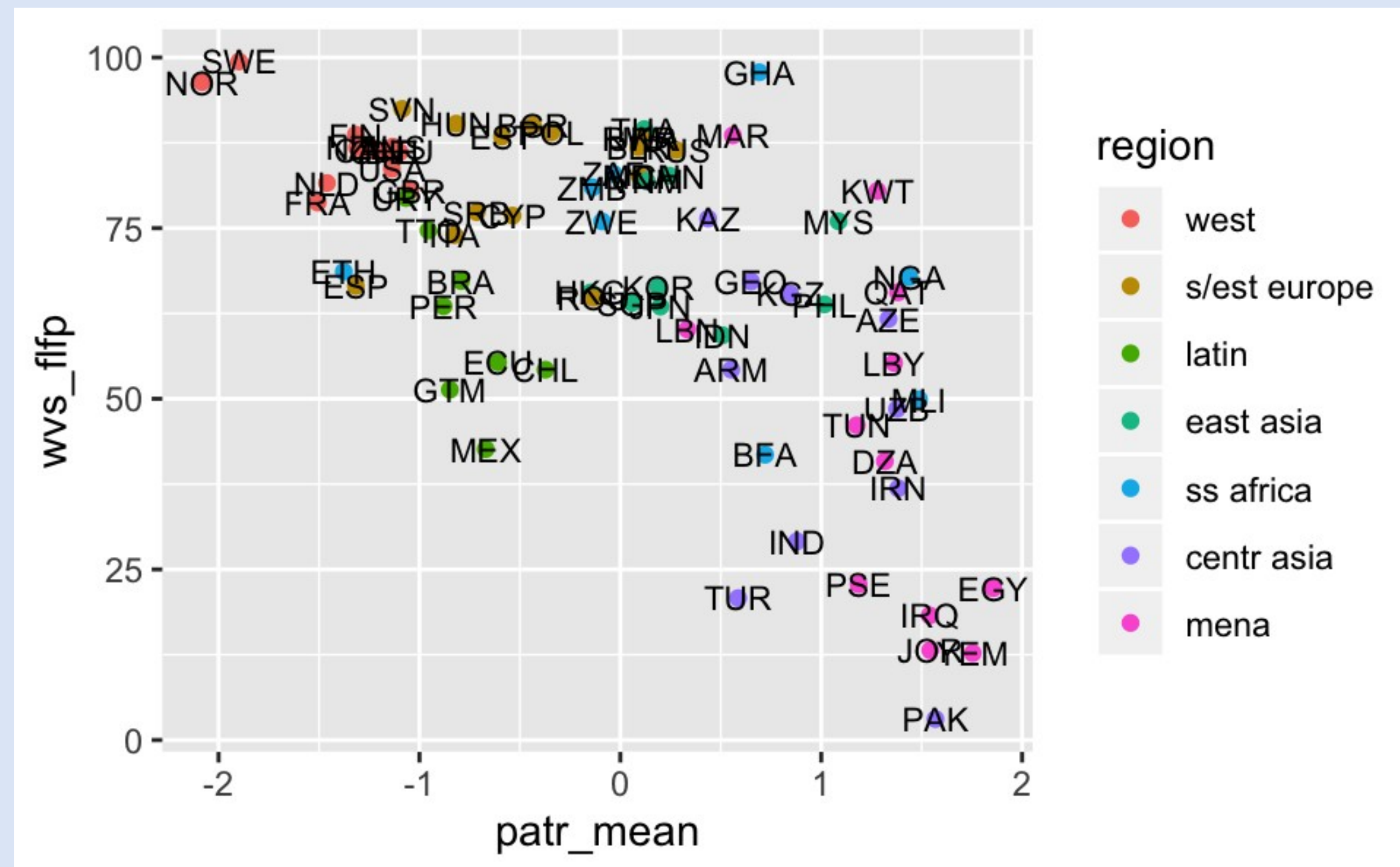
e + geom_smooth(method = lm), x, y, alpha, color, fill, group, linetype, size, weight

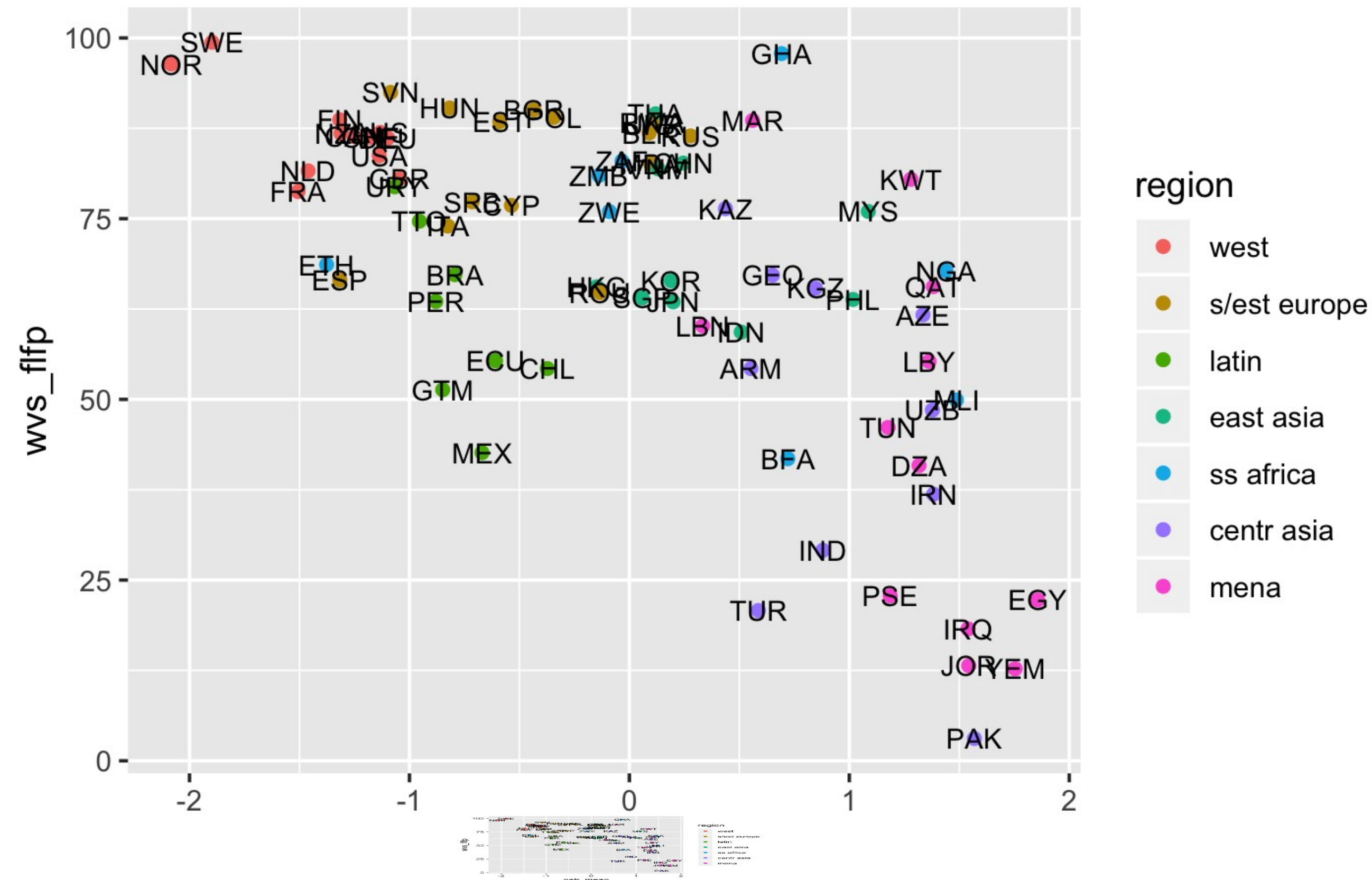


e + geom_text(aes(label = cty), nudge_x = 1, nudge_y = 1, check_overlap = TRUE), x, y, label, alpha, angle, color, family, fontface, hjust, lineheight, size, vjust

Your turn

- Make the scatterplot below with **labels** representing country codes. Use the cheatsheet.



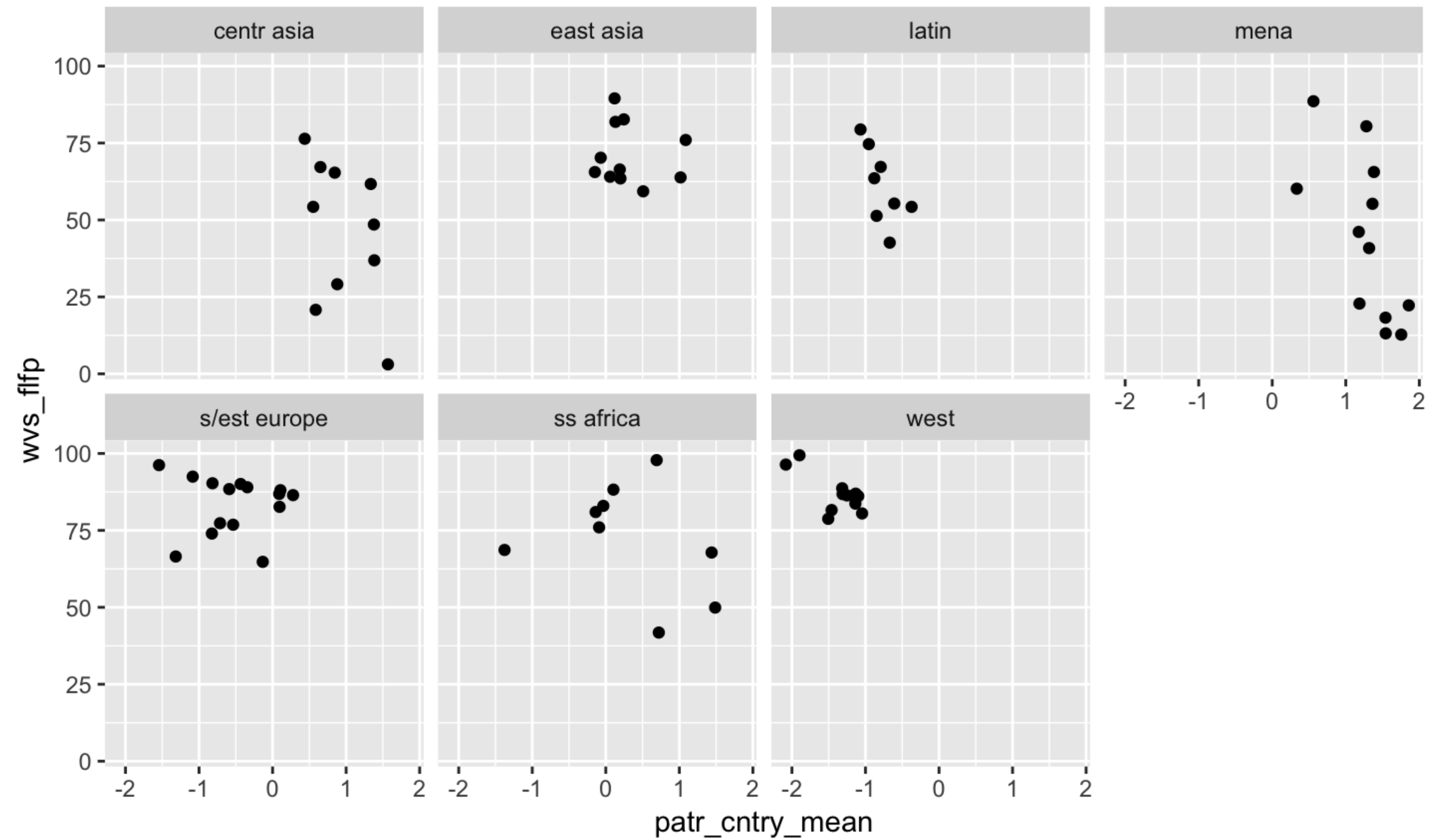


*Of course, one
can also adjust
the labels so
they do not
cover the points.*

```
ggplot(flfp, aes(x = patr_mean, y = wvs_flfp)) +
  geom_point(aes(colour = region)) +
  geom_text(aes(label = code))
```

What else?

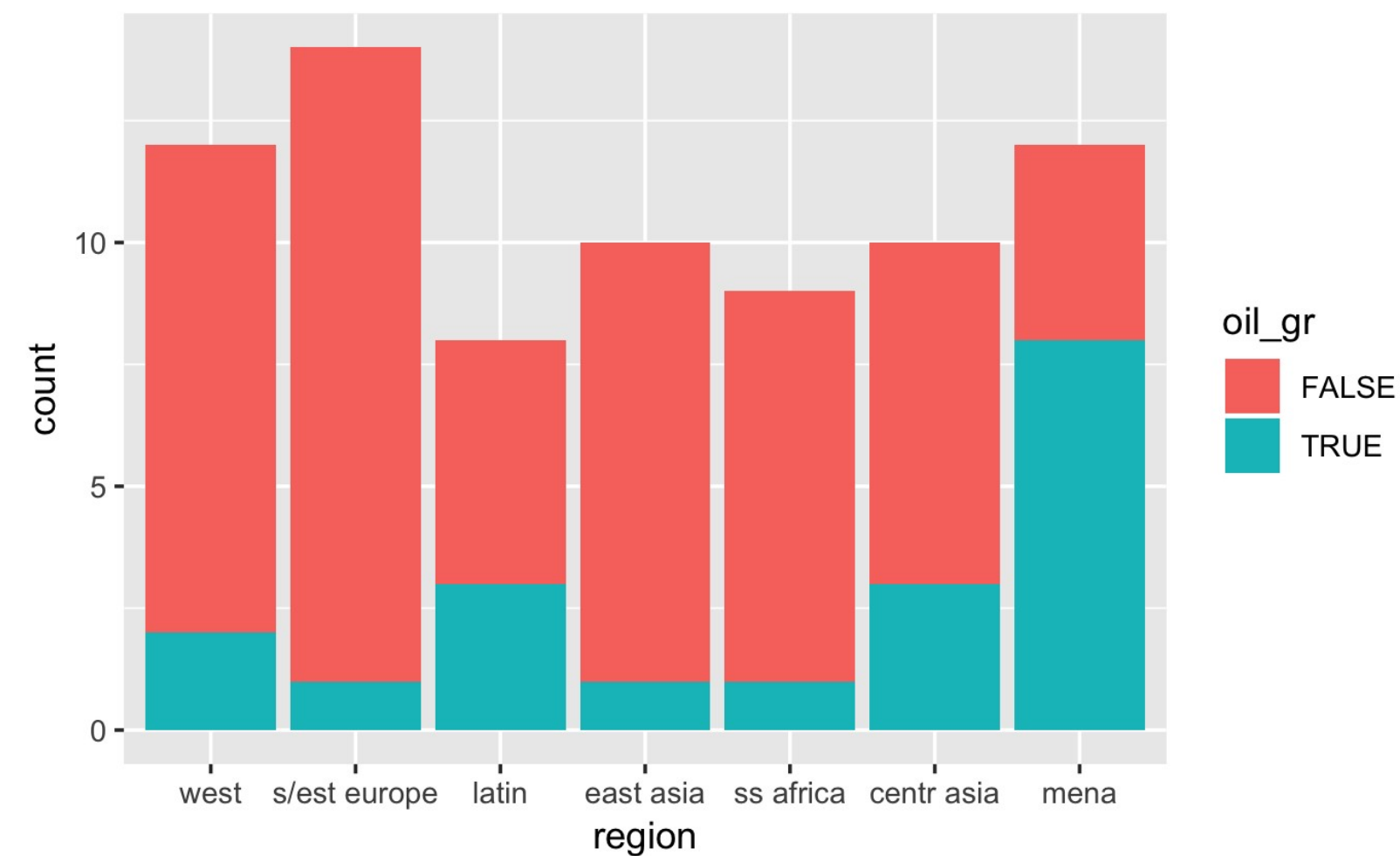
Facets



```
ggplot(flfp_agg) +  
  geom_point(aes(x = patr_mean, y = wvs_flfp)) +  
  facet_wrap(vars(region)) +  
  theme_bw()
```

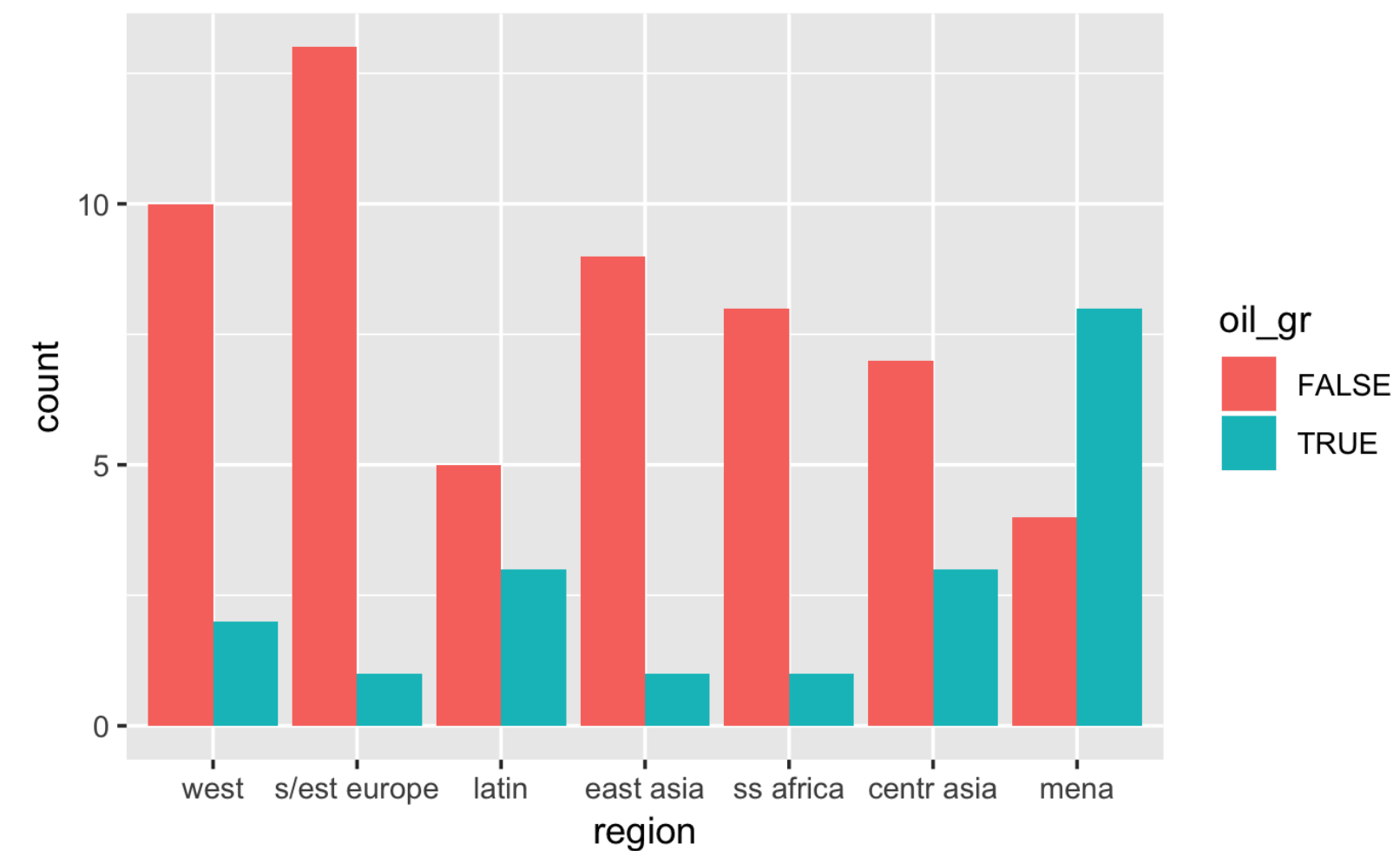
Position adjustments

How overlapping objects are arranged



```
ggplot(flfp_agg) +  
  geom_bar(aes(x = region,  
               fill = oil_gr),  
           position = "stack") +  
  theme_bw()
```

Default position

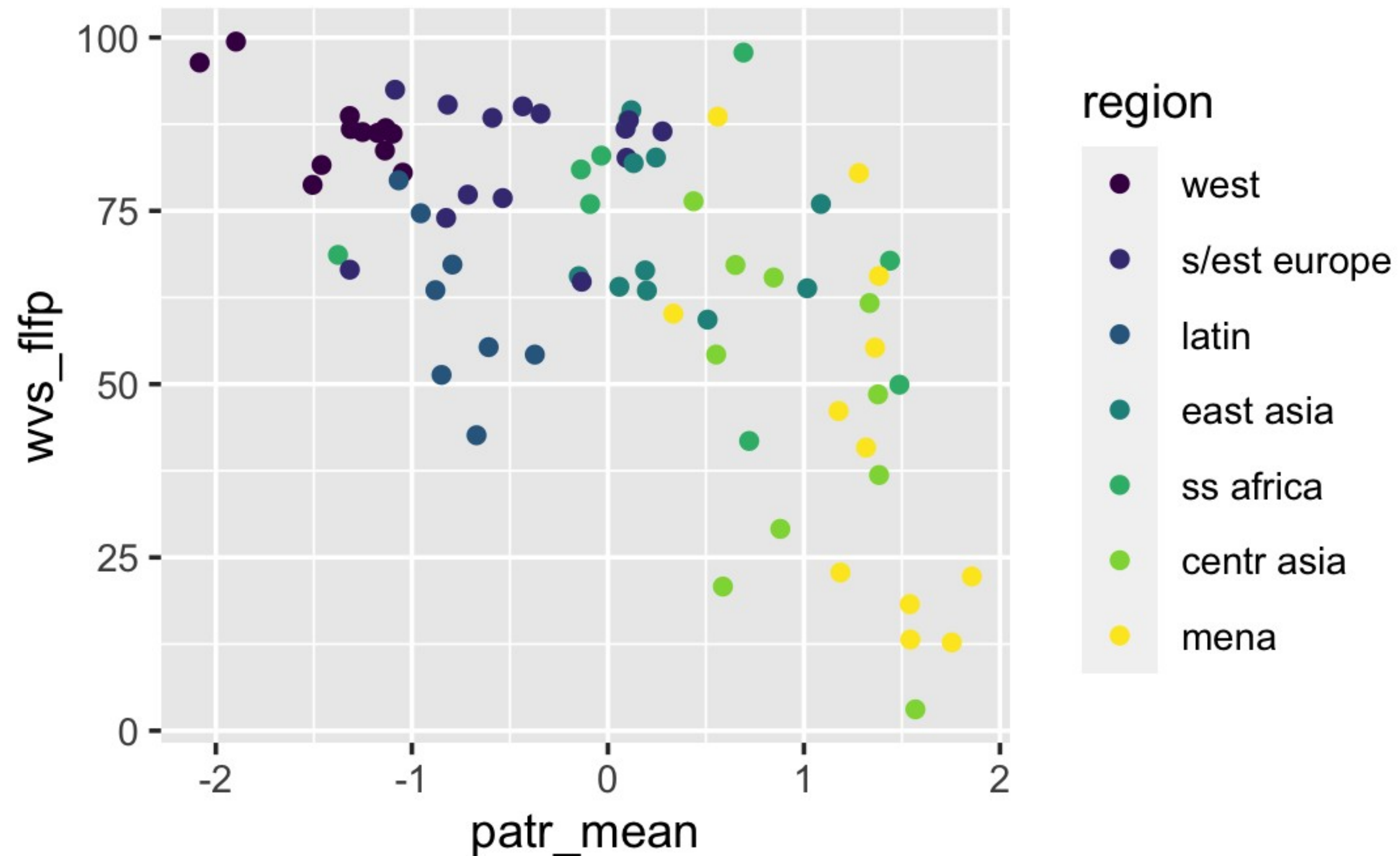


```
ggplot(flfp) +  
  geom_bar(aes(x = region,  
               fill = oil_gr),  
           position = "dodge")
```

Dodged position

Scales

Customise colour scales, other mappings



```
ggplot(flfp_agg) +  
  geom_point(aes(x = patr_mean, y = wvs_flfp, colour = region)) +  
  scale_color_viridis_d() +  
  theme_bw()
```

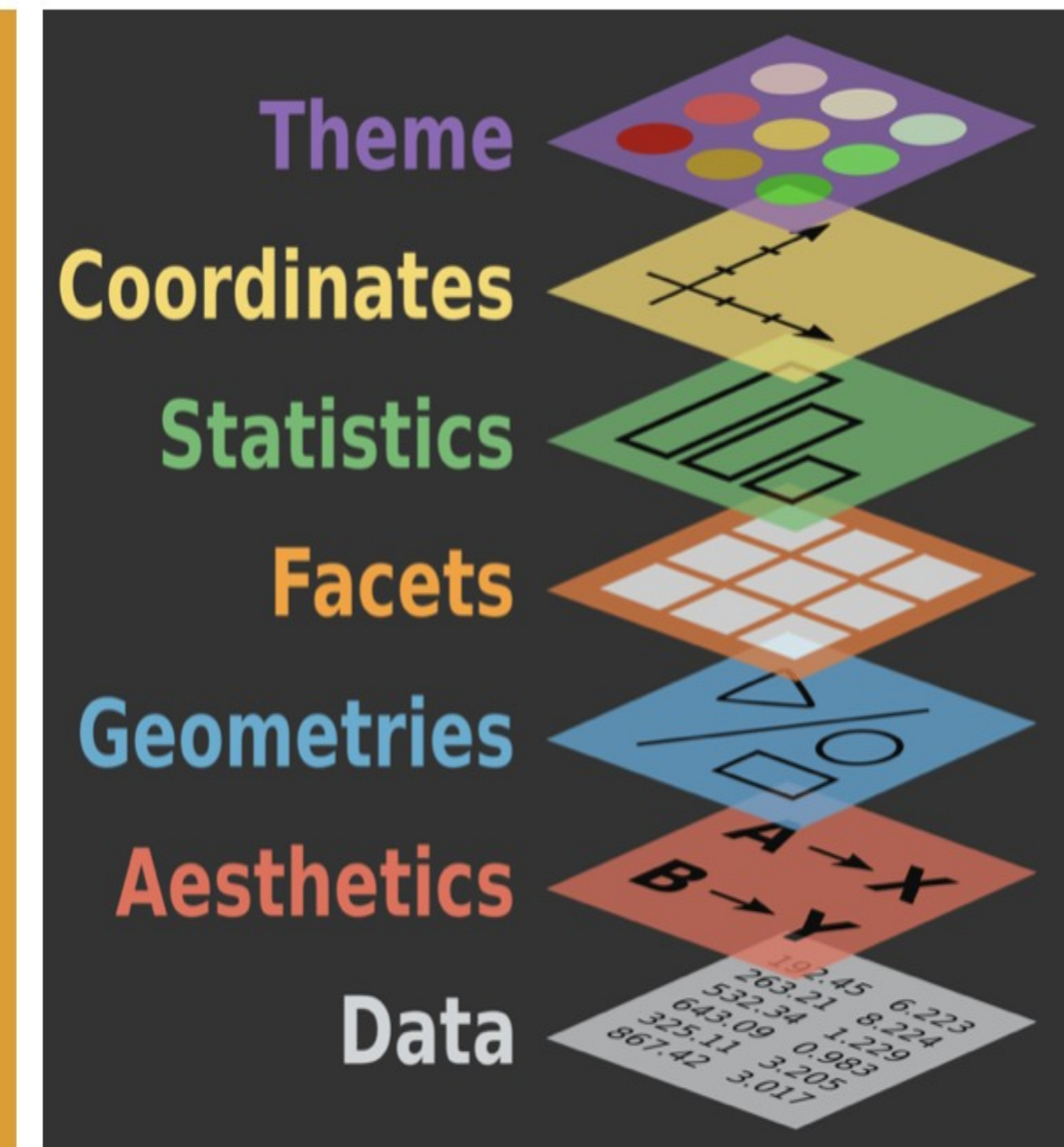
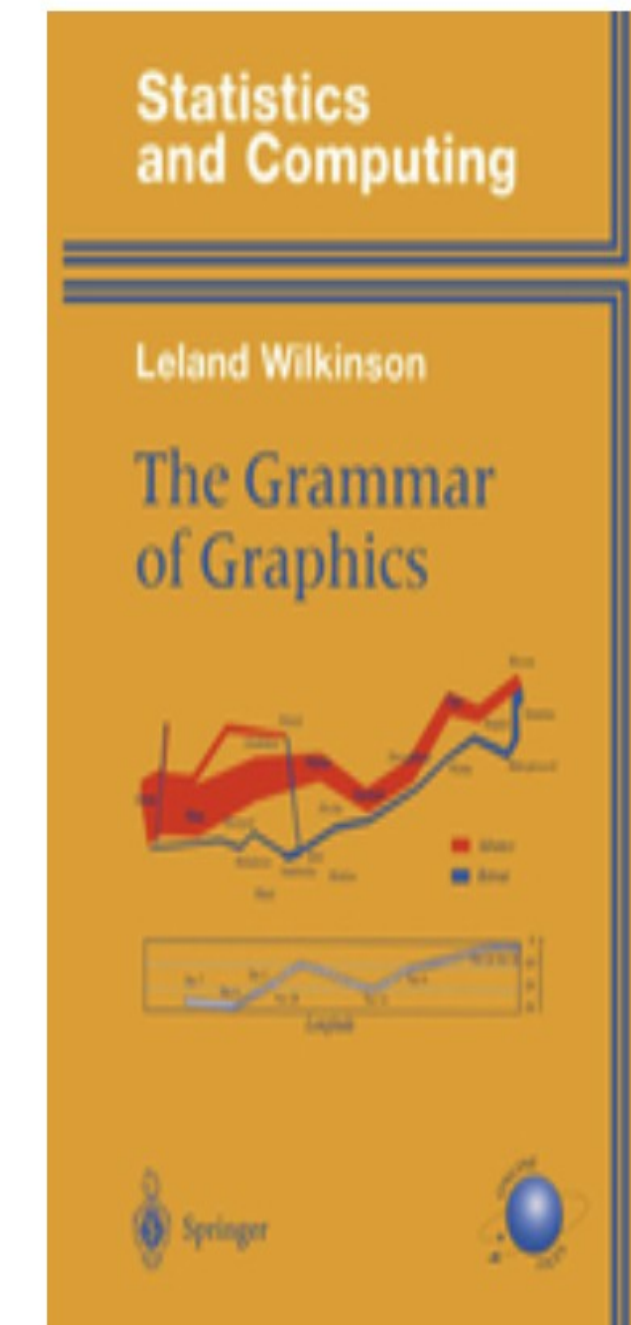
Use the template to make any plot of your choice

Complete the template below to build a graph.

```
ggplot (data = <DATA>) +  
  <GEOM_FUNCTION> (mapping = aes(<MAPPINGS>),  
    stat = <STAT> , position = <POSITION> ) +  
  <COORDINATE_FUNCTION> +  
  <FACET_FUNCTION> +  
  <SCALE_FUNCTION> +  
  <THEME_FUNCTION>
```

required

Not required, sensible defaults supplied



London Cycle Hire Journeys

Thicker, yellower lines mean more journeys

`geom_sf()`
for maps

Data: 3.2 Million Journeys (from TfL)
Routing: Ollie O'Brien (@oobr) + OpenStreetMap cc-by-sa
Buildings: OS Opendata Crown Copyright 2011
Map: James Cheshire (@spatialanalysis)

James Cheshire,

Reference

Plot basics

All ggplot2 plots with a call to `ggplot()`, supplying default data and aesthetic mappings, specified by `aes()`. You then add layers, scales, coords and facets with `+`. To save a plot to disk, use `ggsave()`.

<code>ggplot()</code>	Create a new ggplot
<code>aes()</code>	Construct aesthetic mappings
<code>`+` (<gg>)`</code> <code>`%+%`</code>	Add components to a plot
<code>ggsave()</code>	Save a ggplot (or other grid object) with sensible defaults
<code>qplot()</code> <code>quickplot()</code>	Quick plot

Layer: geoms

A layer combines data, aesthetic mapping, a geom (geometric object), a stat (statistical transformation), and a position

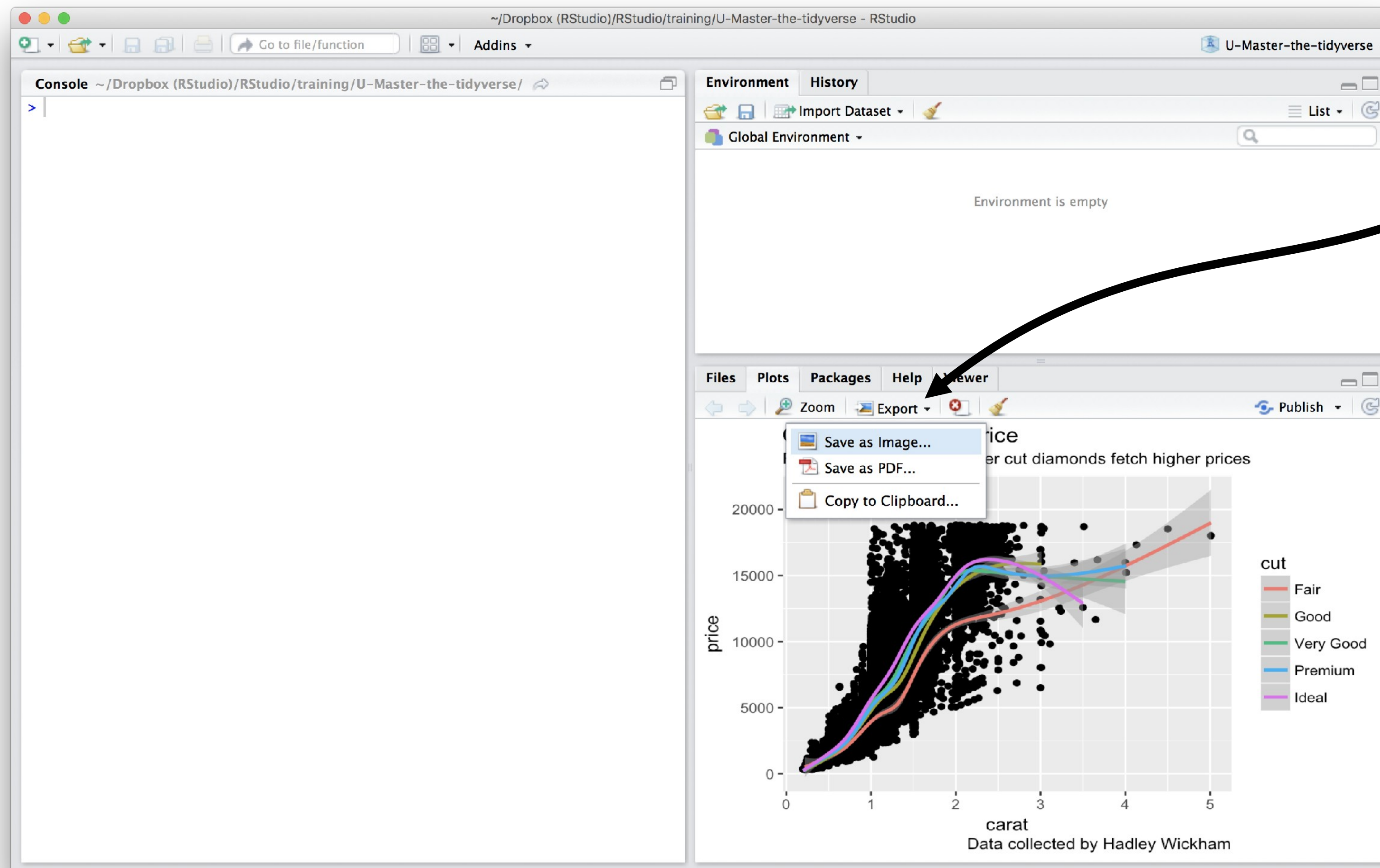
Contents

- [□ Plot basics](#)
- [□ Layer: geoms](#)
- [□ Layer: stats](#)
- [□ Layer: position adjustment](#)
- [□ Layer: annotations](#)
- [□ Aesthetics](#)
- [□ Scales](#)
- [□ Guides: axes and legends](#)
- [□ Facetting](#)
- [□ Facetting: labels](#)
- [□ Coordinate systems](#)
- [□ Themes](#)
- [□ Programming with ggplot2](#)
- [□ Extending ggplot2](#)
- [□ Vector helpers](#)
- [□ Data](#)

Saving graphs

Manually saving plots

Save plots manually with the export menu



Saving plots

ggsave() saves the last plot.

Uses size on screen:

```
ggsave("my-plot.pdf")  
ggsave("my-plot.png")
```

Specify size in inches

```
ggsave("my-plot.pdf", width = 6, height = 6)
```

Your Turn

- Use the `ggsave()` function to save the plot in a pdf file called "01_patr-values-vs-flfp-scatter.pdf".

```
ggsave("01_patr-values-vs-flfp-scatter.pdf")
```

I generally have way better results using "zoom" and then saving manually compared to this function

Occasionally, I have had to do a screen shot to get the right look.

**We can also
make graphics
interactive**

Plotly demonstration

Your turn

- Run the code below.

```
p1 <- ggplot(flfp_agg, aes(x = patr_mean,  
                           y = wvs_flfp,  
                           colour = region,  
                           label = cntry)) +  
  geom_point()
```

```
library(plotly)  
ggplotly(p1)
```

gganimate demonstration

- Line 291 in Day 4 code

Your turn

- There is a `geom_smooth()` argument commented out in the code i have given you. What will this line do if added? Remove the pound signs (don't forget the plus sign above) and re-run the code.
- What is revealed by the slopes here?

rmarkdown



R Markdown

Plain text file with 3 types of content:

The screenshot displays the R Notebook editor interface. The main window shows the R Notebook.Rmd file with the following content:

```
1 ---
2 title: "R Notebook"
3 output: html_notebook
4 ---
5
6 Text written in markdown
7
8 ```{r}
9 # code written in R
10 (x <- rnorm(7))
11 ```
12
13 [1] -1.2  1.0 -0.5  0.9 -0.6 -1.1 -1.5
14
15 Text written in markdown
16
17 ```{r}
18 # code written in R
19 hist(x)
20 ```
```

Three callout boxes highlight specific features:

- A header surrounded by `---`** (Green box): Points to the header section (lines 1-4).
- Text in markdown** (Grey box): Points to the text "Text written in **markdown**" (line 6).
- Code chunks surrounded by `````** (Blue box): Points to the R code chunk (lines 8-11).

The console at the bottom shows the output of the R code: `[1] -1.2 1.0 -0.5 0.9 -0.6 -1.1 -1.5`.

Markdown language

Heading

=====

Sub-heading

Paragraphs are separated
by a blank line.

Two spaces at the end of a
line
produces a line break.

Text attributes *_italic_*,
****bold****, ``monospace``.

Horizontal rule:

Bullet list:

- * apples
- * oranges
- * pears

Numbered list:

1. lather
2. rinse
3. repeat

<h1>Heading**</h1>**

<h2>Sub-heading**</h2>**

<p>Paragraphs are separated
by a blank line.**</p>**

<p>Two spaces at the end of a line**
**
produces a line break.**</p>**

<p>Text attributes ****italic****,
****bold****,
<code>monospace**</code>**.**</p>**

<p>Horizontal rule:**</p>**

<hr />

<p>Bullet list:**</p>**

****apples****
****oranges****
****pears****

<p>Numbered list:**</p>**

****lather****

Heading

Sub-heading

Paragraphs are separated by a blank line.

Two spaces at the end of a line
produces a line break.

Text attributes *italic*, **bold**, `monospace` .

Horizontal rule:

Bullet list:

- apples
- oranges
- pears

Numbered list:

1. lather
2. rinse
3. repeat

An [example](#).



knitr

pandoc



Logistics

1

Knitr runs the document in a fresh R session, which means you need to load the libraries that the document uses *in the document*

Logistics

1

Knitr runs the document in a fresh R session, which means you need to load the libraries that the document uses *in the document*

2

Objects made in one code chunk will be available to code in later code chunks.

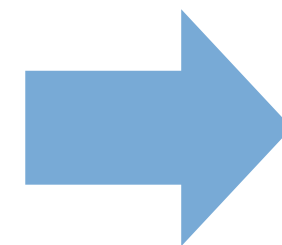
knitr is multi-lingual!

- ☒ SAS
- ☒ python
- ☒ more

python

To embed **non-R** code, change the chunk label from `r` to the language to use.

```
Some python code,  
```{python}  
x = 'hello, python
world!'
print(x)
print(x.split(' '))
```
```

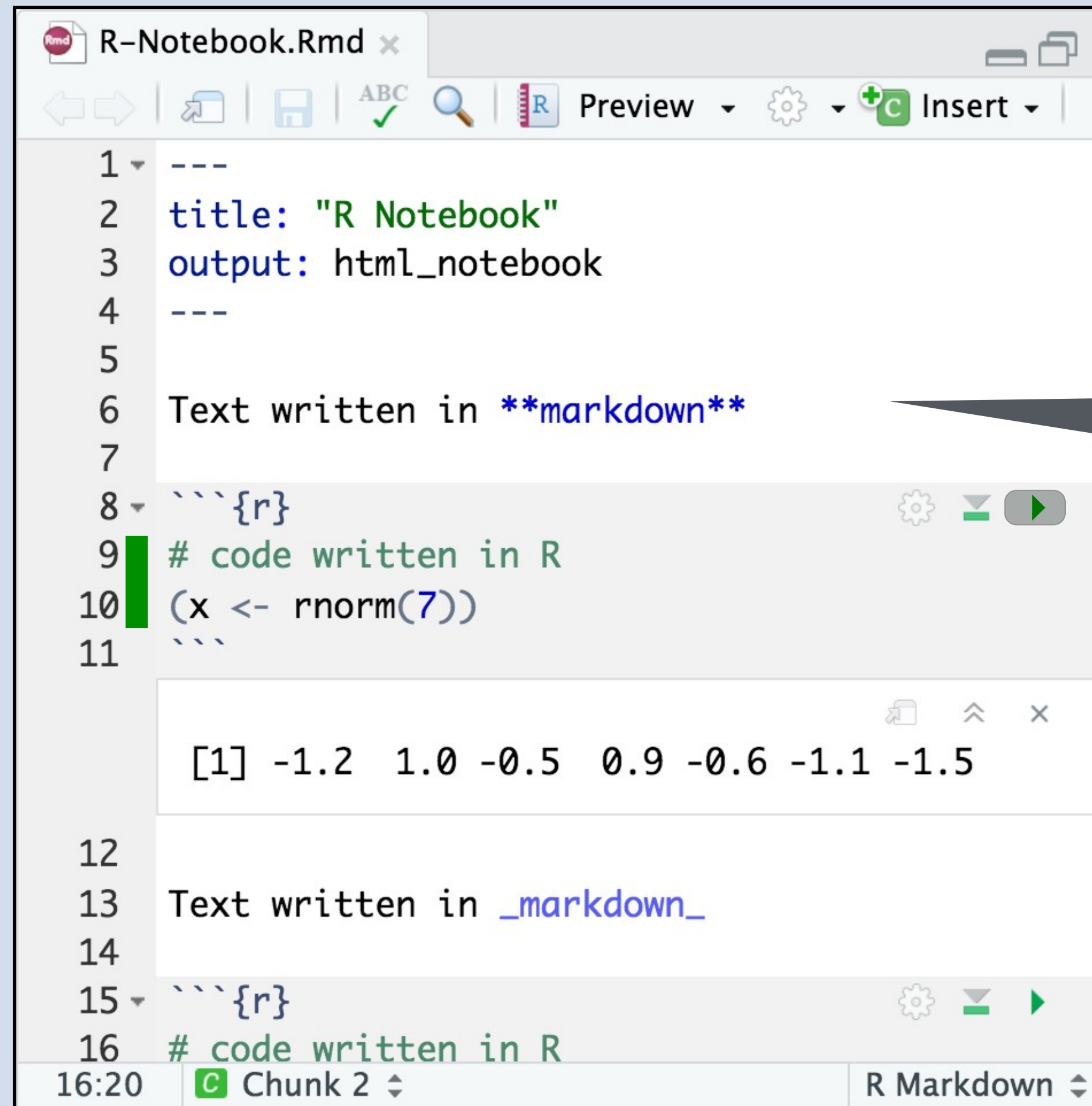


Some python code:

```
x = 'hello, python world!'  
print(x)  
print(x.split(' '))
```

```
## hello, python world!  
## ['hello,', 'python', 'world!']
```

Markdown



```
1 ---
2 title: "R Notebook"
3 output: html_notebook
4 ---
5
6 Text written in markdown
7
8 ```{r}
9 # code written in R
10 (x <- rnorm(7))
11 ```
12
13 [1] -1.2 1.0 -0.5 0.9 -0.6 -1.1 -1.5
14
15 Text written in _markdown_
16
17 ```{r}
18 # code written in R
```

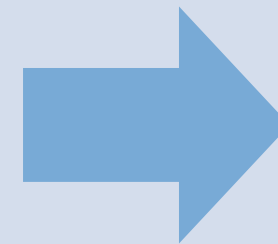
Text in markdown

Headers

Use # to create headers.

Multiple #'s create lower level headers.

```
# Header 1  
## Header 2  
### Header 3  
#### Header 4  
##### Header 5  
##### Header 6
```



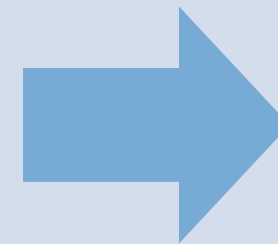
Header 1
Header 2
Header 3
Header 4
Header 5
Header 6

Text

Text is rendered as plain text. Surround text with `_`, `**`, or ``` to format it.

Add two spaces
at the end of a line
to start a new line

Text 
italics
bold
`code`



Text
italics
bold
`code`

Lists

Use asterisks to make bullet points.

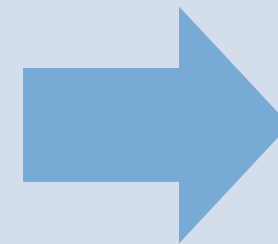
Use numbers to make numbered lists.

Bullets

- * bullet 1
- * bullet 2

Numbered list

1. item 1
2. item 2



Bullets

- bullet 1
- bullet 2

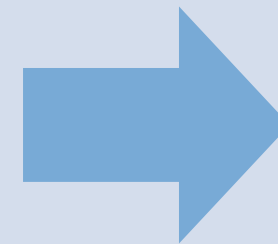
Numbered list

1. item 1
2. item 2

Hyperlinks

Use brackets to denote a link.
Place the URL in parentheses.

```
This is a  
[link] (www.git.com) .
```



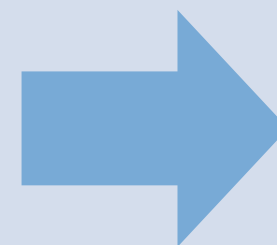
This is a [link](http://www.git.com).

Images

Use a link preceded by an ! to insert an image.

The link text should be a URL (if the image is hosted online), or a file path (if the image is saved as a file)

```
![] (images/1.png)  
The RStudio logo.
```



Markdown

Dictionary of formatting cues.

Pandoc's Markdown

Write with syntax on the left to create effect on right (after render)

| | |
|--|--|
| Plain text
End a line with two spaces to start a new paragraph.
italics and **bold**
`verbatim code`
sub/superscript ² ~2~
~~strikethrough~~
escaped: \^_\\
endash: --, emdash: ---
equation: \$A = \pi * r^2\$
equation block:

$E = mc^2$ | Plain text
End a line with two spaces to start a new paragraph.
/italics and /bold
verbatim code
sub/superscript ²
strikethrough
escaped: ^ _ \
endash: --, emdash: ---
equation: A = π * r ²
equation block:

$E = mc^2$ |
| > block quote | block quote |
| # Header1 {#anchor} | Header1 |
| ## Header 2 {#css_id} | Header 2 |
| ### Header 3 {css_class} | Header 3 |
| #### Header 4 | Header 4 |
| ##### Header 5 | Header 5 |
| ##### Header 6 | Header 6 |
| <!--Text comment--> | HTML ignored in pdfs |
| \textbf{Tex ignored in HTML} | http://www.rstudio.com |
| HTML ignored in pdfs | link |
| <http://www.rstudio.com> | Jump to Header 1 |
| [link](www.rstudio.com) | image: |
| Jump to [Header 1](#anchor) |  |
| image: | Caption |
| ![[Caption]](smallorb.png) | |
| * unordered list
+ sub-item 1
+ sub-item 2
- sub-sub-item 1 | <ul style="list-style-type: none">unordered list<ul style="list-style-type: none">sub-item 1sub-item 2<ul style="list-style-type: none">sub-sub-item 1 |
| * item 2 | <ul style="list-style-type: none">item 2 |
| Continued (indent 4 spaces) | Continued (indent 4 spaces) |
| 1. ordered list
2. item 2
i) sub-item 1
A. sub-sub-item 1 | <ol style="list-style-type: none">ordered listitem 2<ol style="list-style-type: none">sub-item 1A. sub-sub-item 1 |
| (@) A list whose numbering continues after | 1. A list whose numbering continues after |
| (@) an interruption | 2. an interruption |
| Term 1 | Term 1 |
| : Definition 1 | Definition 1 |
| Right Left Default Center | Right Left Default Center |
| ----- ----- ----- ----- | ----- |
| 12 12 12 12 | 12 12 12 12 |
| 123 123 123 123 | 123 123 123 123 |
| 1 1 1 1 | 1 1 1 1 |
| - slide bullet 1
- slide bullet 2 | <ul style="list-style-type: none">slide bullet 1slide bullet 2 |
| (>- to have bullets appear on click) | (>- to have bullets appear on click) |
| horizontal rule/slide break:

*** | horizontal rule/slide break:

A footnote [^1] |
| [^1]: Here is the footnote. | A footnote 1 |
| | 1. Here is the footnote.↩ |

Pandoc's Markdown

Write with syntax on the left to create effect on right (after render)

| | |
|--|--|
| Plain text
End a line with two spaces to start a new paragraph.
italics and **bold**
`verbatim code`
sub/superscript ² ~2~
~~strikethrough~~
escaped: \^_\\
endash: --, emdash: ---
equation: \$A = \pi * r^2\$
equation block:

$E = mc^2$ | Plain text
End a line with two spaces to start a new paragraph.
/italics and /bold
verbatim code
sub/superscript ²
strikethrough
escaped: ^ _ \
endash: --, emdash: ---
equation: A = π * r ²
equation block:

$E = mc^2$ |
| > block quote | block quote |
| # Header1 {#anchor} | Header1 |
| ## Header 2 {#css_id} | Header 2 |
| ### Header 3 {css_class} | Header 3 |
| #### Header 4 | Header 4 |
| ##### Header 5 | Header 5 |
| ##### Header 6 | Header 6 |
| <!--Text comment--> | HTML ignored in pdfs |
| \textbf{Tex ignored in HTML} | http://www.rstudio.com |
| HTML ignored in pdfs | link |
| <http://www.rstudio.com> | Jump to Header 1 |
| [link](www.rstudio.com) | image: |
| Jump to [Header 1](#anchor) |  |
| image: | Caption |
| ![[Caption]](smallorb.png) | |
| * unordered list
+ sub-item 1
+ sub-item 2
- sub-sub-item 1 | <ul style="list-style-type: none">unordered list<ul style="list-style-type: none">sub-item 1sub-item 2<ul style="list-style-type: none">sub-sub-item 1 |
| * item 2 | <ul style="list-style-type: none">item 2 |
| Continued (indent 4 spaces) | Continued (indent 4 spaces) |
| 1. ordered list
2. item 2
i) sub-item 1
A. sub-sub-item 1 | <ol style="list-style-type: none">ordered listitem 2<ol style="list-style-type: none">sub-item 1A. sub-sub-item 1 |
| (@) A list whose numbering continues after | 1. A list whose numbering continues after |
| (@) an interruption | 2. an interruption |
| Term 1 | Term 1 |
| : Definition 1 | Definition 1 |
| Right Left Default Center | Right Left Default Center |
| ----- ----- ----- ----- | ----- |
| 12 12 12 12 | 12 12 12 12 |
| 123 123 123 123 | 123 123 123 123 |
| 1 1 1 1 | 1 1 1 1 |
| - slide bullet 1
- slide bullet 2 | <ul style="list-style-type: none">slide bullet 1slide bullet 2 |
| (>- to have bullets appear on click) | (>- to have bullets appear on click) |
| horizontal rule/slide break:

*** | horizontal rule/slide break:

A footnote [^1] |
| [^1]: Here is the footnote. | A footnote 1 |
| | 1. Here is the footnote.↩ |

Set render options with YAML

- When you render, R Markdown
1. runs the R code, embeds results and text into .md file with knitr
2. then converts the .md file into the finished format with pandoc

| | |
|--|---|
| Set a document's default output format in the YAML header: | output: html_document |
| output value | creates |
| html_document | html (requires Tex) |
| pdf_document | pdf (requires Tex) |
| word_document | Microsoft Word (.docx) |
| odt_document | OpenDocument Text |
| rtf_document | Rich Text Format |
| md_document | Markdown |
| github_document | GitHub compatible markdown |
| ioslides_presentation | ioslides HTML slides |
| slidy_presentation | slidy HTML slides |
| beamer_presentation | Beamer pdf slides (requires Tex) |
| Customize output with sub-options (listed to the right): | output: html_document:
code_folding: hide
toc_float: TRUE |

| |
|---|
| html tabsets |
| Use tabset css class to place sub-headers into tabs |
| # Tabset {tabset,tabset-fade,tabset-pills} |
| ## Tab 1 |
| text 1 |
| ## Tab 2 |
| text 2 |
| ### End tabset |

Create a Reusable Template

1. Create a new package with a inst/markdown/templates directory
2. In the directory, Place a folder that contains: template.yaml (contents of the template) any supporting files
3. Install the package
4. Access template in wizard at File ► New File ► R Markdown template.yaml

| |
|-------------------|
| name: My Template |
|-------------------|

Table Suggestions

Several functions format R data into tables

| | |
|-----------|---------|
| eruptions | walling |
| eruptions | walling |
| eruptions | walling |
| eruptions | walling |
| eruptions | walling |

| | |
|-----------|---------|
| eruptions | walling |
| eruptions | walling |
| eruptions | walling |
| eruptions | walling |
| eruptions | walling |

Citations and Bibliographies

Create citations with .bib, .bibtex, .copac, .enl, .json, .medline, .mods, .ris, .wos, and .xml files

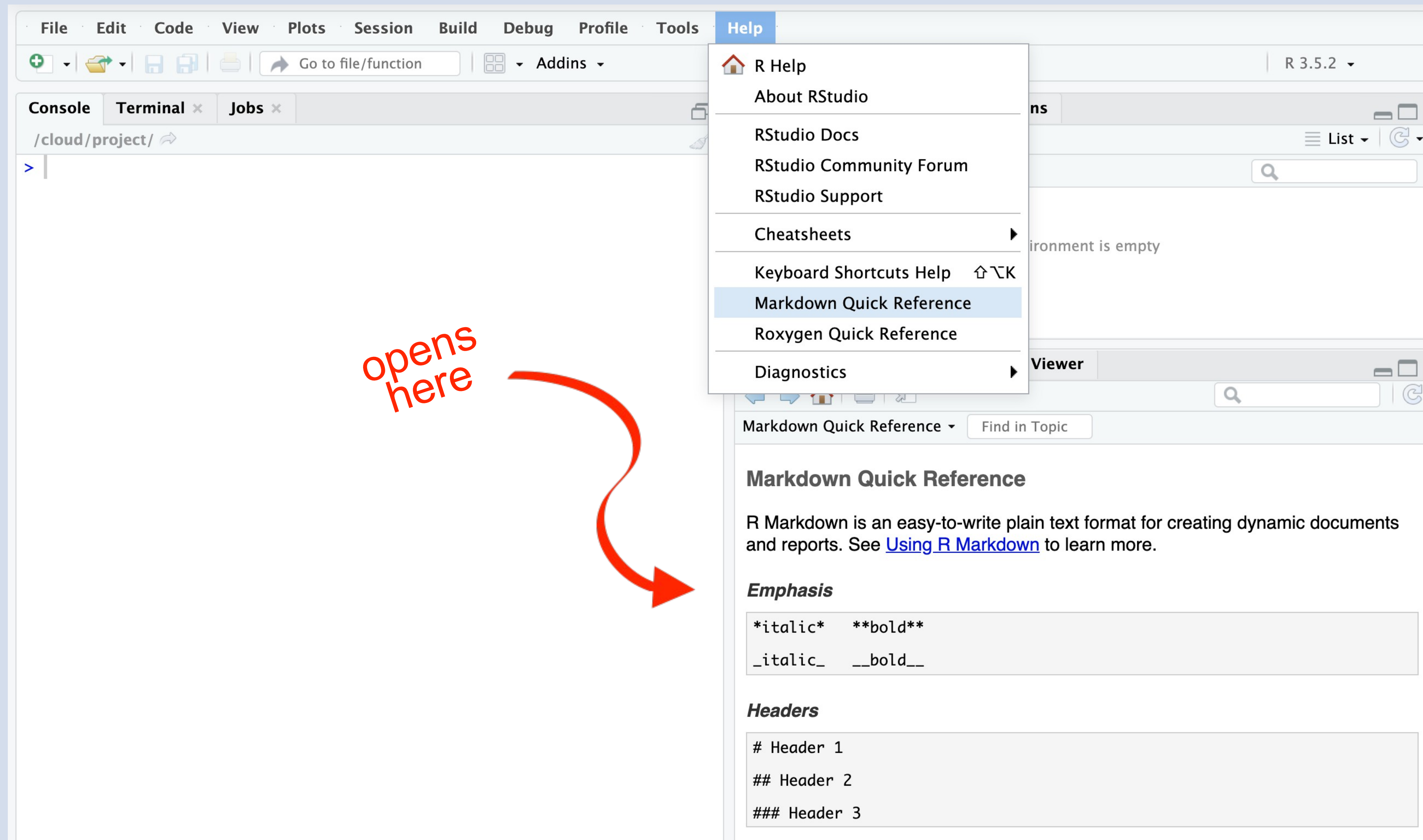
1. Set bibliography file and CSL 1.0 Style file (optional) in the YAML header
2. Use citation keys in text

| |
|---|
| Smith cited [smith04]. |
| Smith cited without author [-@smith04]. |
| @smith04 cited in line. |

3. Render. Bibliography will be added to end of document

IDE Reference

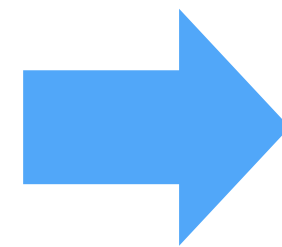
Go to **Help > Markdown Quick Reference**



Inline code

Place code in a sentence with ``r <code>``. R
Markdown will replace the code with its results.

```
Today is  
`r Sys.Date()`.
```



```
Today is 2015-04-16.
```

Inline code

Code whose results are inserted into the text.

```
Today is `r Sys.Date()`.
```

Surround
with `r`

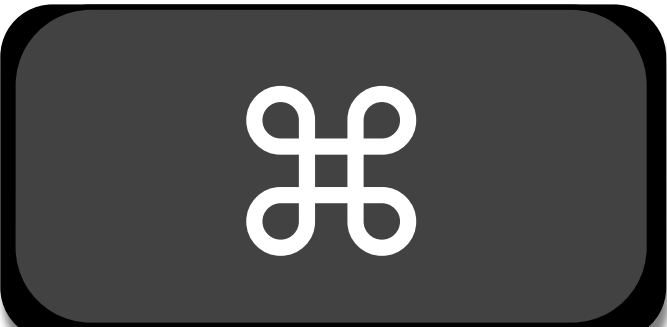
Code to run. Only the
result will be included.

The sample includes ``r nrow(flfp)`` countries. The average FLFP rate is ``r round(mean(flfp$wvs_flfp), 2)``.

Code chunks

Insert a chunk of R code with

```
` `` {r}  
# some code  
` ``
```

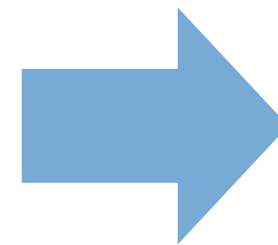
 +  +  (Mac)

 +  +  (PC)

chunk options

By default, R Markdown includes both the code and its results

```
Here's some code  
```${r}  
dim(iris)
```
```



Here's some code

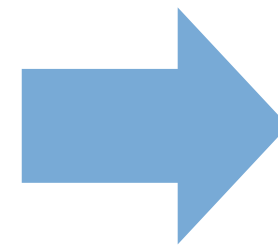
```
dim(iris)
```

```
## [1] 150 5
```

echo

Add options in the brackets after r.
echo = FALSE hides the code.

```
Here's some code  
```{r echo=FALSE}  
dim(iris)
```
```



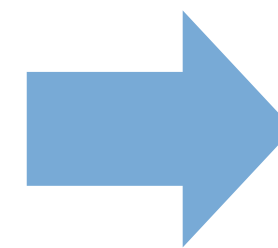
Here's some code

```
## [1] 150 5
```

eval

eval = FALSE prevents the code from being run. As a result, no results will be displayed with the code.

```
Here's some code  
```{r eval=FALSE}  
dim(iris)
```
```



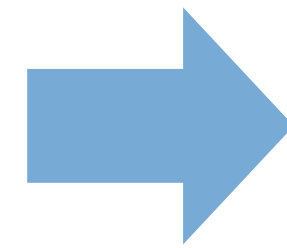
Here's some code

```
dim(iris)
```

include

include = FALSE runs the code, but prevents both the code and the results from appearing (e.g. to setup).

```
Here's some code  
```{r include=FALSE}  
dim(iris)
```
```



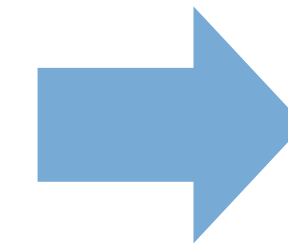
Here's some code

fig.height, fig.width

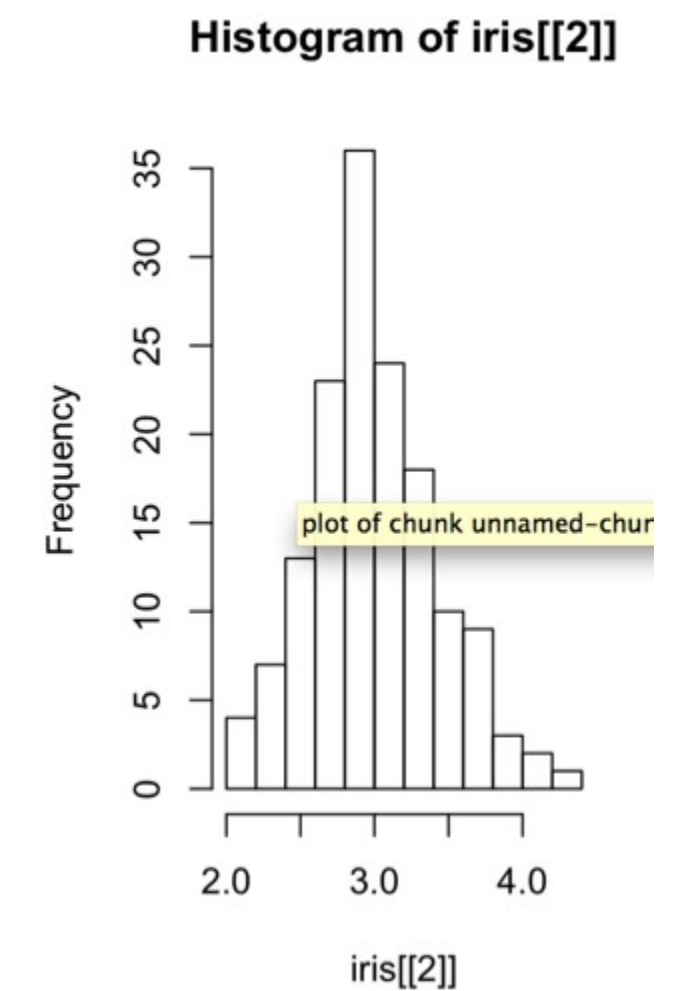
Specify the dimension of plots (in inches) with `fig.width` and `fig.height`. Separate multiple arguments with commas.

Here's a plot

```
```{r echo=FALSE, fig.width=3, fig.height=5}  
hist(iris[[2]])
```
```



Here's a plot

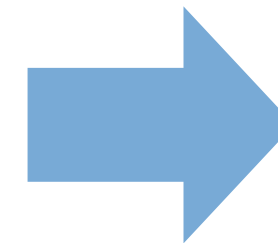


warning

Suppresses warnings. Works similarly with messages.

Here's some code

```
` `{r warning=FALSE}  
warning("Something troublesome I do not want you to see")  
` `
```



Here's some code

global chunk options

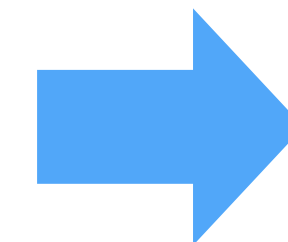
Hides the code in every chunk unless over-written within {r}.

```
` `` {r setup}  
knitr::opts_chunk$set(echo = FALSE)  
` ``
```

Header

A section of *key:value* pairs
separated by dashed lines ———

```
---  
title: "Untitled"  
author: "RStudio"  
date: "February 4, 2015"  
output: html\_document  
---  
  
Text of document
```



Untitled

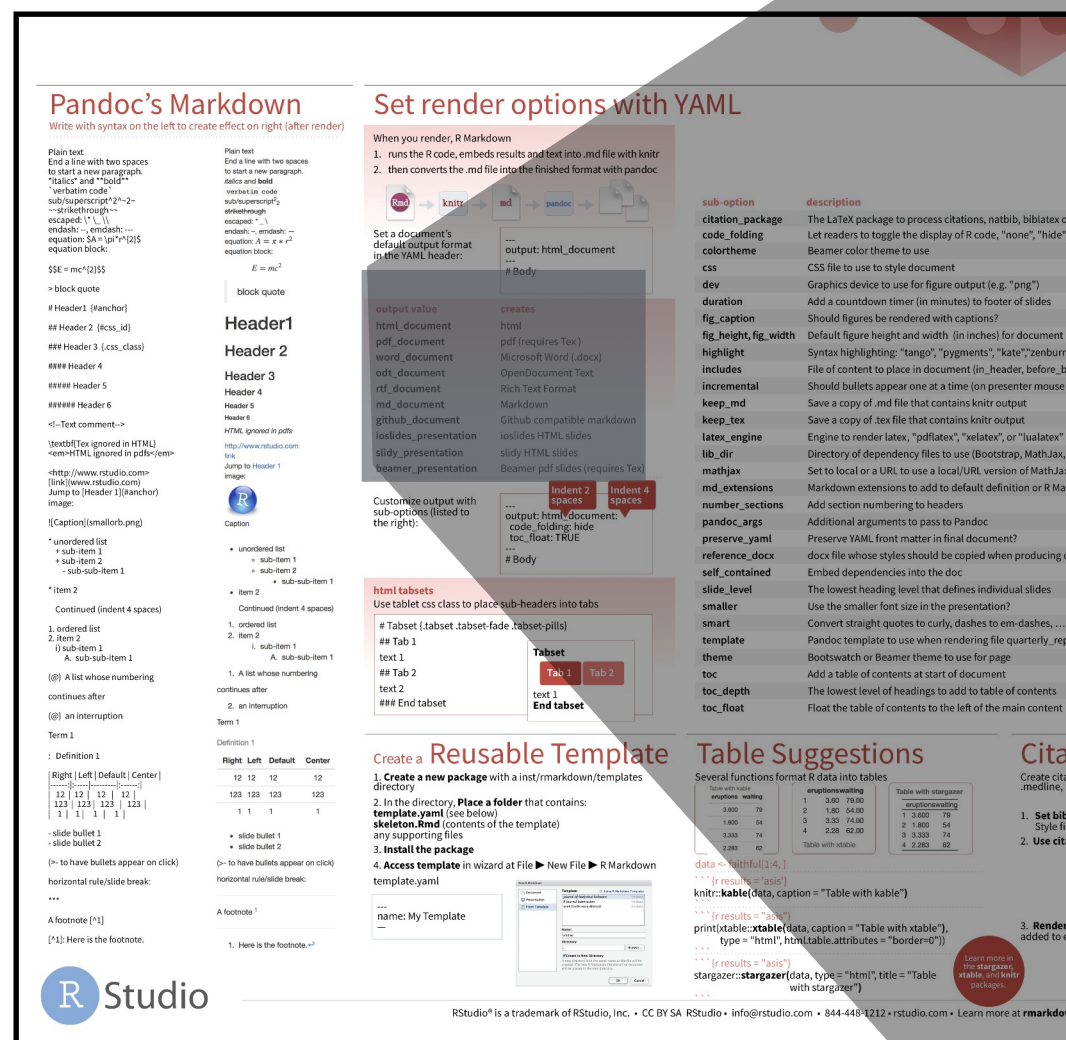
RStudio

February 4, 2015

Text of document

output

The output: field sets the format of the final report



output value

html_document

pdf_document

word_document

odt_document

rtf_document

md_document

github_document

ioslides_presentation

slidy_presentation

beamer_presentation

creates

html

pdf (requires Tex)

Microsoft Word (.docx)

OpenDocument Text

Rich Text Format

Markdown

Github compatible markdown

ioslides HTML slides

slidy HTML slides

Beamer pdf slides (requires Tex)



Now...

- Let's demonstrate the utility of rmarkdown by using rmarkdown to generate a powerpoint presentation about... rmarkdown.
- (by the way, **last year** I tried to get bing copilot AI to help me generate such a presentation, but it got a bit confused with the meta aspects of the task and failed).
- Open up the script called "rmarkdown introduction.R"